

# Tile IR 规范（中文翻译）

2025 年 12 月

## Contents

|          |                             |           |
|----------|-----------------------------|-----------|
| <b>1</b> | <b>简介</b>                   | <b>6</b>  |
| 1.1      | 在 GPU 上使用 tile 进行可扩展的数据并行计算 | 6         |
| 1.2      | 目标与范围                       | 6         |
| 1.3      | 文档结构                        | 6         |
| <b>2</b> | <b>编程模型</b>                 | <b>8</b>  |
| 2.1      | tile 内核                     | 8         |
| 2.1.1    | tile 编程有何不同?                | 8         |
| 2.1.2    | 内核输入/输出                     | 8         |
| 2.2      | tile 网格                     | 9         |
| 2.2.1    | 实现 GEMM                     | 10        |
| 2.2.2    | 使用单个块进行 GEMM                | 10        |
| 2.2.3    | 逐块的 GEMM                    | 11        |
| 2.2.4    | 遍历图块                        | 13        |
| 2.3      | 结构化指针                       | 14        |
| 2.4      | Tiling & Views              | 15        |
| 2.4.1    | 使用视图进行向量加法                  | 15        |
| 2.4.2    | 使用视图的动态 GEMM                | 17        |
| 2.5      | 跨 TileBlock 通信              | 18        |
| <b>3</b> | <b>语法</b>                   | <b>20</b> |
| 3.1      | 模块                          | 20        |
| 3.2      | 条目                          | 20        |
| 3.3      | 全局变量                        | 20        |
| 3.4      | 内核                          | 20        |
| 3.5      | 类型                          | 20        |
| <b>4</b> | <b>二进制格式</b>                | <b>21</b> |
| 4.1      | 原语                          | 21        |
| 4.1.1    | 固定宽度整数                      | 21        |
| 4.1.2    | 可变宽度整数                      | 21        |
| 4.2      | 文件结构                        | 21        |
| 4.2.1    | 魔数                          | 22        |
| 4.2.2    | 版本                          | 22        |
| 4.2.3    | Sections                    | 22        |
| 4.3      | 操作码与编码                      | 29        |
| 4.4      | 操作编码详情                      | 29        |
| 4.5      | 节顺序                         | 31        |
| <b>5</b> | <b>类型系统</b>                 | <b>32</b> |
| 5.1      | 元素类型                        | 32        |
| 5.1.1    | 基本类型                        | 32        |
| 5.1.2    | 备选类型                        | 32        |
| 5.2      | 指针                          | 32        |
| 5.3      | 张量类型                        | 33        |
| 5.3.1    | 瓦片类型                        | 33        |

|          |                     |           |
|----------|---------------------|-----------|
| 5.3.2    | 张量视图                | 33        |
| 5.3.3    | 子视图类型               | 34        |
| 5.4      | 类型等价                | 35        |
| <b>6</b> | <b>语义</b>           | <b>36</b> |
| 6.1      | 抽象机器                | 36        |
| 6.2      | 模块                  | 36        |
| 6.2.1    | 全局变量                | 36        |
| 6.2.2    | Tile Kernel         | 36        |
| 6.2.3    | Tile 函数             | 37        |
| 6.2.4    | 函数体                 | 37        |
| 6.2.5    | Well-Formedness     | 37        |
| 6.3      | 值                   | 37        |
| 6.3.1    | 指针                  | 37        |
| 6.3.2    | Tiles               | 38        |
| 6.3.3    | Views               | 38        |
| 6.4      | 瓦片网格                | 39        |
| 6.5      | Register File       | 39        |
| 6.6      | 全局内存                | 39        |
| 6.7      | 瓦片块                 | 39        |
| 6.8      | 执行语义                | 40        |
| 6.8.1    | 初始化                 | 40        |
| 6.8.2    | 前向进展                | 40        |
| 6.8.3    | 图块块执行               | 40        |
| 6.8.4    | 终止                  | 40        |
| <b>7</b> | <b>内存模型</b>         | <b>41</b> |
| 7.1      | 内存模型操作              | 41        |
| 7.2      | 作用域                 | 41        |
| 7.3      | 内存排序                | 41        |
| 7.4      | Moral Strength      | 41        |
| 7.5      | 令牌与令牌顺序             | 42        |
| 7.6      | 基础关系                | 42        |
| 7.7      | No-thin-air         | 42        |
| 7.8      | 数据竞争                | 42        |
| 7.9      | PTX 互操作性            | 42        |
| 7.10     | Tile IR 内存模型中的风险与直觉 | 43        |
| 7.10.1   | 从未来读取的令牌顺序操作        | 43        |
| 7.10.2   | 单个瓦片块线程内的竞争风险       | 43        |
| 7.10.3   | 关于如何使用令牌排序的一些直观理解   | 43        |
| <b>8</b> | <b>运算操作</b>         | <b>44</b> |
| 8.1      | 元类型                 | 44        |
| 8.1.1    | 符号                  | 44        |
| 8.1.2    | 标志                  | 44        |
| 8.1.3    | 令牌                  | 44        |
| 8.1.4    | 可变参数                | 44        |
| 8.1.5    | 任意                  | 44        |
| 8.1.6    | 名称                  | 44        |
| 8.1.7    | 类型                  | 44        |
| 8.1.8    | 数组                  | 44        |
| 8.1.9    | 字符串                 | 44        |
| 8.1.10   | bool                | 44        |
| 8.1.11   | DenseConstant       | 44        |
| 8.1.12   | view_type           | 44        |
| 8.2      | 操作设计注意事项            | 45        |
| 8.2.1    | 显式广播                | 45        |
| 8.2.2    | 独立的浮点与整数运算          | 45        |
| 8.2.3    | 显式溢出标注              | 45        |

|        |                               |    |
|--------|-------------------------------|----|
| 8.3    | Core                          | 45 |
| 8.3.1  | cuda_tile.broadcast           | 45 |
| 8.3.2  | cuda_tile.cat                 | 45 |
| 8.3.3  | cuda_tile.cmpf                | 46 |
| 8.3.4  | cuda_tile.cmpi                | 48 |
| 8.3.5  | cuda_tile.constant            | 49 |
| 8.3.6  | cuda_tile.entry               | 50 |
| 8.3.7  | cuda_tile.extract             | 51 |
| 8.3.8  | cuda_tile.get_global          | 51 |
| 8.3.9  | cuda_tile.get_num_tile_blocks | 52 |
| 8.3.10 | cuda_tile.get_tile_block_id   | 53 |
| 8.3.11 | cuda_tile.global              | 53 |
| 8.3.12 | cuda_tile.iota                | 54 |
| 8.3.13 | cuda_tile.mmaf                | 54 |
| 8.3.14 | cuda_tile.mmai                | 56 |
| 8.3.15 | cuda_tile.module              | 57 |
| 8.3.16 | cuda_tile.offset              | 57 |
| 8.3.17 | cuda_tile.pack                | 58 |
| 8.3.18 | cuda_tile.permute             | 59 |
| 8.3.19 | cuda_tile.reduce              | 59 |
| 8.3.20 | cuda_tile.reshape             | 61 |
| 8.3.21 | cuda_tile.scan                | 61 |
| 8.3.22 | cuda_tile.select              | 63 |
| 8.3.23 | cuda_tile.unpack              | 63 |
| 8.4    | 类型转换                          | 64 |
| 8.4.1  | cuda_tile.bitcast             | 64 |
| 8.4.2  | cuda_tile.exti                | 65 |
| 8.4.3  | cuda_tile.ftof                | 65 |
| 8.4.4  | cuda_tile.ftoi                | 66 |
| 8.4.5  | cuda_tile.itof                | 67 |
| 8.4.6  | cuda_tile.int_to_ptr          | 68 |
| 8.4.7  | cuda_tile.ptr_to_int          | 68 |
| 8.4.8  | cuda_tile.ptr_to_ptr          | 69 |
| 8.4.9  | cuda_tile.trunci              | 69 |
| 8.5    | 控制流                           | 70 |
| 8.5.1  | cuda_tile.assert              | 70 |
| 8.5.2  | cuda_tile.break               | 71 |
| 8.5.3  | cuda_tile.continue            | 72 |
| 8.5.4  | cuda_tile.for                 | 72 |
| 8.5.5  | cuda_tile.if                  | 74 |
| 8.5.6  | cuda_tile.loop                | 75 |
| 8.5.7  | cuda_tile.return              | 76 |
| 8.5.8  | cuda_tile.yield               | 77 |
| 8.6    | 内存                            | 78 |
| 8.6.1  | cuda_tile.join_tokens         | 78 |
| 8.6.2  | cuda_tile.load_ptr_tko        | 79 |
| 8.6.3  | cuda_tile.make_token          | 81 |
| 8.6.4  | cuda_tile.store_ptr_tko       | 81 |
| 8.7    | 浮点数                           | 83 |
| 8.7.1  | 浮点运算                          | 83 |
| 8.7.2  | 浮点运算                          | 83 |
| 8.7.3  | cuda_tile.absf                | 83 |
| 8.7.4  | cuda_tile.addf                | 83 |
| 8.7.5  | cuda_tile.atan2               | 84 |
| 8.7.6  | cuda_tile.ceil                | 85 |
| 8.7.7  | cuda_tile.cosh                | 86 |
| 8.7.8  | cuda_tile.cos                 | 86 |
| 8.7.9  | cuda_tile.divf                | 87 |

|          |                                 |            |
|----------|---------------------------------|------------|
| 8.7.10   | cuda_tile.exp2                  | 88         |
| 8.7.11   | cuda_tile.exp                   | 89         |
| 8.7.12   | cuda_tile.floor                 | 90         |
| 8.7.13   | cuda_tile.fma                   | 90         |
| 8.7.14   | cuda_tile.log2                  | 91         |
| 8.7.15   | cuda_tile.log                   | 92         |
| 8.7.16   | cuda_tile.maxf                  | 93         |
| 8.7.17   | cuda_tile.minf                  | 94         |
| 8.7.18   | cuda_tile.mulf                  | 95         |
| 8.7.19   | cuda_tile.negf                  | 97         |
| 8.7.20   | cuda_tile.pow                   | 97         |
| 8.7.21   | cuda_tile.remf                  | 98         |
| 8.7.22   | cuda_tile.rsqrt                 | 99         |
| 8.7.23   | cuda_tile.sinh                  | 100        |
| 8.7.24   | cuda_tile.sin                   | 100        |
| 8.7.25   | cuda_tile.sqrt                  | 101        |
| 8.7.26   | cuda_tile.subf                  | 102        |
| 8.7.27   | cuda_tile.tanh                  | 103        |
| 8.7.28   | cuda_tile.tan                   | 104        |
| 8.8      | 整数                              | 105        |
| 8.8.1    | 整数运算                            | 105        |
| 8.8.2    | cuda_tile.absi                  | 105        |
| 8.8.3    | cuda_tile.addi                  | 106        |
| 8.8.4    | cuda_tile.divi                  | 106        |
| 8.8.5    | cuda_tile.maxi                  | 107        |
| 8.8.6    | cuda_tile.mini                  | 109        |
| 8.8.7    | cuda_tile.muli                  | 110        |
| 8.8.8    | cuda_tile.mulhii                | 111        |
| 8.8.9    | cuda_tile.negi                  | 112        |
| 8.8.10   | cuda_tile.ori                   | 113        |
| 8.8.11   | cuda_tile.remi                  | 113        |
| 8.8.12   | cuda_tile.shli                  | 114        |
| 8.8.13   | cuda_tile.shri                  | 115        |
| 8.8.14   | cuda_tile.subi                  | 116        |
| 8.8.15   | cuda_tile.xori                  | 117        |
| 8.9      | 位运算                             | 118        |
| 8.9.1    | cuda_tile.andi                  | 118        |
| 8.10     | 原子操作                            | 118        |
| 8.10.1   | cuda_tile.atomic_cas_tko        | 118        |
| 8.10.2   | cuda_tile.atomic_rmw_tko        | 121        |
| 8.11     | 视图                              | 123        |
| 8.11.1   | cuda_tile.get_index_space_shape | 124        |
| 8.11.2   | cuda_tile.get_tensor_shape      | 124        |
| 8.11.3   | cuda_tile.load_view_tko         | 125        |
| 8.11.4   | cuda_tile.make_partition_view   | 127        |
| 8.11.5   | cuda_tile.make_tensor_view      | 128        |
| 8.11.6   | cuda_tile.store_view_tko        | 130        |
| 8.12     | 杂项                              | 132        |
| 8.12.1   | cuda_tile.assume                | 132        |
| 8.12.2   | cuda_tile.print_tko             | 136        |
| <b>9</b> | <b>调试信息</b>                     | <b>137</b> |
| 9.1      | 位置信息                            | 137        |
| 9.1.1    | 位置类型                            | 137        |
| 9.1.2    | 生成作用域元数据                        | 137        |
| 9.1.3    | 选项                              | 137        |
| 9.1.4    | 权衡                              | 138        |
| 9.1.5    | 机制                              | 138        |

|       |                  |     |
|-------|------------------|-----|
| 9.2   | 作用域元数据 . . . . . | 138 |
| 9.2.1 | 编译单元 . . . . .   | 138 |
| 9.2.2 | 文件 . . . . .     | 138 |
| 9.2.3 | 词法块 . . . . .    | 138 |
| 9.2.4 | 子程序 . . . . .    | 139 |
| 9.3   | 示例用法 . . . . .   | 139 |

# 1 简介

本文档描述了 **Tile IR**，一种可移植的低层级图块虚拟机和指令集。

与将 GPU 建模为数据并行单指令多线程（SIMT）处理器的 PTX 不同，**Tile IR** 将 GPU 建模为基于 tile 的处理器。

在 **Tile IR** 中，每个逻辑线程（tile 块）在多维数组（张量）的部分片段（tile）上进行计算。

## 1.1 在 GPU 上使用 tile 进行可扩展的数据并行计算

CUDA 平台通过 CUDA C++ 和 PTX 提供可移植性，使得为旧款 GPU 编写的代码能在新款 GPU 上高效运行。这些机制使开发者能够为某一系列 GPU 编写软件，并持续部署到未来的代际产品上。

硬件特性的快速演进（如张量核心）增加了编程模型的复杂性，要求开发者具备更高的专业知识和硬件理解能力，才能编写出可移植且高性能的代码。

自 Volta 以来，每一代新的 GPU 都引入了强大的新硬件特性，催生了新的编程范式。

为了解决这些挑战，**Tile IR** 引入了一套虚拟指令集，支持以 tile 为单位对硬件进行原生编程，使开发者能够编写更高级别的代码，并能在多个 GPU 平台上以最小改动高效执行。

虽然 PTX 确保了 SIMT 程序的可移植性，但 **Tile IR** 通过原生支持基于分块（tile）的程序扩展了 CUDA 平台，使开发者能够专注于将数据并行程序划分为分块和分块组，而由 **Tile IR** 负责将其映射到线程、内存层次结构和张量核心等硬件资源上。

通过提高抽象层次，**Tile IR** 使用户能够为 NVIDIA 硬件构建新的、更高层次的硬件特定领域专用语言（DSL）、编译器和框架。

## 1.2 目标与范围

**Tile IR** 旨在为基于图块的并行编程提供一个性能可移植的编程模型和指令集。

核心目标：

- 引入一种数据并行的 tile 编程抽象，该抽象与程序员意图保持一致，并可作为领域特定语言和编译器的代码生成目标。
- 抽象张量核心及其编程模型，以在不破坏 NVIDIA 生态系统或客户现有软件投资的前提下推动硬件创新。
- 抽象底层、架构特定的细节，如 CUDA 线程和内存层次结构，以确保程序能够为不同的 GPU 高效地重新编译。
- 最小化抽象开销。虽然可移植性有其代价，但 **Tile IR** 旨在将这一代价保持在较低水平，仅引入适度的性能开销。
- 提供用户控制和优化提示，以便在需要时恢复峰值性能。
- 与现有 CUDA 编程模型（如 CUDA C++ 和 PTX）实现无缝互操作性。

**Tile IR** 通过以下关键组件实现这些核心目标，按重要性排序：

1. **Tile IR** abstract machine 的版本化规范，包括版本化且可移植的字节码表示形式。
2. 一个用于 **Tile IR** 程序的优化编译器，作为 CUDA 驱动程序的一部分提供，也可作为 CUDA 工具包中的独立工具使用，类似于 PTX。
3. 一个现有编译器可用的 MLIR 方言，用于将 **Tile IR** 作为后端编译器目标。

## 1.3 文档结构

4. 简介 本节。
5. 编程模型 描述了 **Tile IR** 的编程模型。
6. 语法定义了本规范中使用的 **Tile IR** 程序的语法。
7. 二进制格式 描述了 **Tile IR** 字节码及其二进制格式。
8. 类型系统定义了 **Tile IR** 的类型系统。

9. 语义 描述了 **Tile IR** 的半形式化语义。
10. 内存模型 描述了 **Tile IR** 内存模型。
11. 操作 **Tile IR** 完整操作集的列表。
12. 调试信息 描述了 **Tile IR** 程序的调试信息格式。
13. 稳定性 描述了 **Tile IR** 的稳定性保证。
14. 附录 包含相关的参考资料，包括完整程序和参考文献。

## 2 编程模型

**Tile IR** 通过引入与 CUDA C++ 或 PTX 中现有抽象不同的新抽象，扩展了 CUDA 的低级编程模型。

本节介绍 **Tile IR** 的编程模型，并引导读者熟悉其核心概念与抽象。我们将通过一系列实际程序示例逐步展开，最终构建一个动态、高性能的 GEMM 实现，该实现充分利用了 **Tile IR** 的所有主要特性。

**Tile IR** 拥有一个富有表现力的基于 tile 的编程模型。我们通过逐步调整示例程序，以更好地利用 **Tile IR** 的特性（这些特性有助于简化程序，并使编译器能够提供性能可移植性），向用户介绍表示张量计算的各种方法。我们首先从读者可能已从现有技术中熟悉的概念开始介绍。

### 2.1 tile 内核

**Tile IR** 程序被称为 *tile kernels*，类似于 CUDA C++ 或 PTX，这些函数在调用时以 N 个副本并行运行。主要区别在于执行的基本单位：一个 *tile-block*，它表示单个逻辑 tile 线程在多维数据块上执行的计算。

在执行过程中，每个 tile kernel 都被称为一个 tile kernel instance。

下面是一个简单的 **Tile IR** 内核，用于打印 “Hello World!”。

```
cuda_tile.module @hello_world_module {  
  entry @hello_world_kernel() {  
    print "Hello World!\n"  
  }  
}
```

对于熟悉 CUDA 线程的用户，需要注意的是 **Tile IR** 的线程有所不同。

在我们进一步深入形式化之前，有必要先强调这些差异。

tile kernels 是程序的入口点，作为 tile 块的并行实例执行。

#### 2.1.1 tile 编程有何不同？

**Tile IR** 是对 CUDA 编程模型的扩展，它为 tile 编程提供了一流的支持。

Tiled kernels 将程序表达为在 tiles 上操作的逻辑 tiled 线程网格。网格和单个 tiled 线程到底层硬件线程的映射从编程模型中抽象出来，并由编译器处理。

NVIDIA 流式多处理器 (SM) 的 SIMT 编程模型是一种线程在（相对）小数据块上运行的模型，用户需负责将线程划分并调度到合适的块中，以便高效地对输入数据进行计算。这种模型为程序员提供了如何将线程映射到数据（或反之）的灵活性。SIMT 是 CUDA 和 PTX 所呈现的编程模型，自 2006 年推出以来，一直为 NVIDIA GPU 提供良好支持。

深度学习重要性的提升，既为用户工作负载引入了更强的规律性，也对这些工作负载的性能交付提出了日益增长的需求。正如引言中所讨论的，这催生了以张量核心形式出现的新型专用硬件。

张量核心为 SIMT 编程模型引入了新的维度。现在，SM 线程必须与张量核心协同工作，才能达到峰值性能。随着每一代新硬件的推出，这两部分硅芯片之间的相互作用释放了惊人的新性能，但也带来了日益增加的编程复杂性。

**Tile IR** 的构建旨在帮助实现高性能算法，这些算法能够充分利用底层硬件的性能，同时减轻编程复杂性的增加。

通过抽象线程到数据的映射，**Tile IR** 相比传统的 SIMT 模型，简化了 Tensor Cores 等专用硬件的使用。

#### 2.1.2 内核输入/输出

为了说明 **Tile IR** 的设计，我们将从简单的 hello world 内核转向一个实现一维张量（即向量）加法且具有固定块大小 128 的内核。

本节中展示的所有示例均可在编程模型示例程序中找到。

Tile kernels 接受输入和输出作为参数；这是消费和产生数据的唯一机制，因此我们首先定义 kernel 参数。

```
entry @vector_block_add_128x1_kernel(  
  %a_ptr_base_scalar : !cuda_tile.tile<ptr<f32>>,  
  %b_ptr_base_scalar : !cuda_tile.tile<ptr<f32>>,  
  %c_ptr_base_scalar : !cuda_tile.tile<ptr<f32>>)
```



上述代码片段定义了一个名为 `vector_block_add_128x1_kernel` 的内核，该内核接受三个参数，分别代表两个输入缓冲区 `a` 和 `b`，以及输出缓冲区 `c`。所有参数的类型均为标量指针，表示为包含单个指针的零维张量。

**Tile IR** 中的所有值要么是张量，要么是张量视图（参见张量视图）。张量是一个  $n$  维矩形数组，由其秩（维度数量）、形状（每个维度上的大小）及其基本元素类型描述。

张量可以具有 0 阶或更高阶。0 阶张量是标量。阶数、维度和元素类型都是张量类型的一部分，并且是静态已知的。张量类型被分配给表示全局内存中包含的多维数组逻辑视图的值。全局内存总是通过张量访问，因此指针参数始终指向全局设备内存中的 CUDA 设备分配。Tile 内核没有返回值，因此省略了返回类型注释（注意，Tile 函数可能有返回类型，稍后将讨论）。

一位敏锐的读者现在可能会疑惑，为什么我们给输入和输出使用了标量指针类型，而不是使用具有静态已知秩、形状和步长的张量类型。在 **Tile IR** 编程模型中，一个常见的模式是让内核接收非结构化的基指针作为参数，然后可以用这些指针来构建所需的张量。

这种灵活性使得根据期望的程序行为有多种使用指针的方式，但我们将首先通过将基指针转换为任意指针的张量来关注最灵活表示方式。

一个任意指针的张量是一种灵活的抽象，它允许您一次性从一组地址执行分散/聚集式加载，并将一组地址视为逻辑 tile。

我们需要采取几个步骤，将基础指针 `%a_ptr_base_scalar` 转换为表示我们要计算的 128x1 图块的指针张量，其中包含从  $(base + 0, \dots, base + 127)$  的地址。

我们首先创建一个偏移张量，它表示包含性的  $(0, 127)$  区间。

我们使用 `cuda_tile.iota`，它构造一个范围张量，从 0 计数到  $n - 1$ ，形成一个大小为  $n$  的向量。

```
%offset = iota : tile<128xi32>
```

然后我们将从标量 `ptr<f32>` 重塑为一维张量 `1xptr<f32>`，以获得正确的秩。

```
%a_ptr_base_tensor = reshape %a_ptr_base_scalar :  
  tile<ptr<f32>> -> tile<1xptr<f32>>
```

然后我们广播指针，这样我们就得到了一个包含 128 个元素的一维张量  $(base, \dots, base)$ 。

```
%a_ptr = broadcast %a_ptr_base_tensor : tile<1xptr<f32>> -> tile<128xptr<f32>>
```

将偏移张量加到指针张量上，得到一个 `tile<128xptr<f32>>`，其值包含指针  $(base + 0, \dots, base + 127)$ 。现在我们有了一组指针张量，代表我们想要在其上进行计算的图块。我们对 `a`、`b` 和 `c` 执行相同的步骤。

```
%a_tensor = offset %a_ptr, %offset :  
  tile<128xptr<f32>>, tile<128xi32> -> tile<128xptr<f32>>
```

最后，我们加载两个操作数，执行加法，并将结果存储到输出中。

```
%a_val, %token_a = load_ptr_tko weak %a_tensor : tile<128xptr<f32>> -> tile<128xf32>, token  
%b_val, %token_b = load_ptr_tko weak %b_tensor : tile<128xptr<f32>> -> tile<128xf32>, token  
%c_val = addf %a_val, %b_val rounding<nearest_even> : tile<128xf32>  
store_ptr_tko weak %c_tensor, %c_val : tile<128xptr<f32>>, tile<128xf32> -> token
```

我们现在拥有一个完整的单 tile 块内核，用于对 128 个元素的向量执行加法操作。如您所见，这段代码是从单个控制线程编写的，但其并行化水平将由编译器决定。

内核通过指针参数与全局内存通信，这些参数可以通过形状和步幅操作转换为张量，以实现灵活的数据访问。

## 2.2 tile 网格

到目前为止，我们只研究了为单个 tile 块编写的内核。**Tile IR** 允许将 tile 块分组为 **tile 网格**，类似于 CUDA C++，使用户能够启动并行执行的 tile 块集合。与 PTX 一样，tile 内核在 tile 块坐标上隐式参数化（可通过 `cuda_tile.get_tile_block_id` 查询），并且可以是一维、二维或三维的。

当启动一个 tile 内核时，用户指定网格大小，这决定了启动的 tile 块数量。启动的 tile 块数量等于网格的大小。例如，如果我们使用一个  $(1, 1, 2)$  网格启动之前的示例，我们将运行两次相同的计算。

我们现在可以看看一个改进的 hello world 程序，它展示了如何查询网格大小和坐标。

```

cuda_tile.module @hello_world_module {
  // TileIR kernel function
  entry @hello_world_kernel() {
    // Step 1. Get the tile block ID
    %block_x_index, %block_y_index, %block_z_index = cuda_tile.
      get_tile_block_id : tile<i32>
    // Step 2. Get the tile block dimensions
    %block_dim_x, %block_dim_y, %block_dim_z = cuda_tile.get_num_tile_blocks
      : tile<i32>
    // Step 3. Print the tile block ID and dimensions. Each tile executes
    the
    // following print statement and prints a single line.
    cuda_tile.print "Hello, I am tile <%, %, %> in a kernel with <%, %, %>
      tiles.\n",
      %block_x_index, %block_y_index, %block_z_index, %block_dim_x, %
        block_dim_y, %block_dim_z
      : tile<i32>, tile<i32>, tile<i32>,
        tile<i32>, tile<i32>, tile<i32>
  }
}

```

每个 tile 内核都可以使用一维、二维或三维网格启动。每个 tile 块可以通过 `cuda_tile.get_tile_block_id` 查询其在网格中的位置，并通过改变参数来查询第一、第二或第三维度的索引。tile 内核也可以使用 `cuda_tile.get_num_tile_blocks` 以类似的方式观察网格维度。如果我们使用一个 (1, 1, 2) 的网格，我们将看到两个打印：

```

1      "Hello Tile-Grid. I am tile <1, 1, 1> in a kernel with <1, 1, 2> tiles."
2      "Hello Tile-Grid. I am tile <1, 1, 2> in a kernel with <1, 1, 2> tiles."

```

tile 网格组织 tile 块的并行执行，允许内核通过查询其在网格内的坐标来根据问题规模进行扩展。

### 2.2.1 实现 GEMM

既然我们已经理解了 **Tile IR** 编程模型的基本概念，接下来我们将介绍如何为单个块计算二维 GEMM，然后通过利用 tile 网格、引入控制流和手动分块，逐步将其推广到完整的 GEMM 计算。

这种进展展示了如何将基本的 tile 操作组合成一个复杂的高性能并行算法。

### 2.2.2 使用单个块进行 GEMM

让我们首先自然地从一个静态块向量加法过渡到单个静态方块矩阵乘法。

This example proceeds much as before:

```

entry @gemm_block_64x64_kernel(
  %a_ptr_base_scalar: !cuda_tile.tile<!cuda_tile.ptr<f32>>,
  %b_ptr_base_scalar: !cuda_tile.tile<!cuda_tile.ptr<f32>>,
  %c_ptr_base_scalar: !cuda_tile.tile<!cuda_tile.ptr<f32>>

```

我们从一组标量指针开始。为了简化这个例子，我们假设每个指针都指向长度至少为 64 个元素且步幅为 1 的分配内存。

然后，就像之前一样，我们声明一个方形偏移张量。

```

%offset_flat = iota : tile<4096xi32>
%offset = reshape %offset_flat :
  tile<4096xi32> -> tile<64x64xi32>

```

我们像之前为 A、B、C 声明底层图块的指针一样进行声明。

```

%a_ptr_base_tensor = reshape %a_ptr_base_scalar :
  tile<ptr<f32>> -> tile<1x1xptr<f32>>
%a_ptr = broadcast %a_ptr_base_tensor : tile<1x1xptr<f32>> -> tile<64x64xptr<f32>>
%a_tensor = offset %a_ptr, %offset :
  tile<64x64xptr<f32>>, tile<64x64xi32> -> tile<64x64xptr<f32>>

```

这里的唯一区别是我们在二维而非一维中构建方形图块。

我们随后加载输入参数, 计算 MMA 操作(浮点数计算参见cuda\_tile.mmaf, 整数计算参见cuda\_tile.mmai) 以计算输出图块, 然后将其存储。

```
// Load a single 64x64 matrix from the tile.
%A_block, %token_a = load_ptr_tko weak %a_tensor :
    tile<64x64xptr<f32>> -> tile<64x64xf32>, token
// Load a single 64x64 matrix from the tile.
%B_block, %token_b = load_ptr_tko weak %b_tensor :
    tile<64x64xptr<f32>> -> tile<64x64xf32>, token
%C_block = mmf %A_block, %B_block, %init_accum: tile<64x64xf32>, tile<64x64xf32>
    , tile<64x64xf32>
store_ptr_tko weak %c_tensor, %C_block :
    tile<64x64xptr<f32>>, tile<64x64xf32> -> token
```

如果你在实现矩阵乘法方面有经验, 你会看到这对于单个 64x64 方块乘法效果很好, 但如果我们想要在并行的大型输入问题规模上运行分块内核, 会发生什么呢?

基础矩阵乘法通过从指针构建二维分块, 并在单个块内执行矩阵乘加 (MMA) 操作来实现。

### 2.2.3 逐块的 GEMM

让我们看看如何将其推广到大型矩阵尺寸——例如 4096x4096, 其中每个 tile 块将计算最终矩阵的单个输出 tile。请注意, 我们仍然基于问题形状是静态的假设进行工作, 本节稍后我们将回到处理动态形状输入的问题。

内核声明看起来与我们迄今为止所采用的完全相同, 将输入和输出的标量指针作为参数。

```
entry @gemm_square_4096_tile_64x64_kernel(
    %a_ptr_base_scalar: tile<ptr<f32>>,
    %b_ptr_base_scalar: tile<ptr<f32>>,
    %c_ptr_base_scalar: tile<ptr<f32>>
```

为了在大矩阵上进行完整的矩阵乘法, 我们需要使用 tile 网格将问题分割到多个 tile 块中。我们将使用二维网格, 因此每个线程首先需要读取其二维块坐标。

```
// Read Tile block id's.
%block_x_index, %block_y_index, %block_z_index = get_tile_block_id : tile<i32>
```

块坐标决定了我们将在该块上计算输出矩阵的哪个图块。内核的结构是在矩阵的 K 维度上进行归约, 生成独立但完整的输出图块。它循环遍历归约维度, 对每个贡献于输出图块的输入图块执行矩阵乘加 (MMA) 操作。

正如我们在内核开始之前所做的那样, 首先设置初始状态。由于此内核被构造为一个循环, 我们将首先设置循环状态。

```
%m_tile_size = constant <i32: 64> : tile<i32>
%m_stride_factor = cuda_tile.constant <i32: 64> : tile<64x64xi32> // todo fix
    the line # and restore this %n_tile_size = cuda_tile.constant <i32: 64> :
    tile<i32>
%k_tile_size = cuda_tile.constant <i32: 64> : tile<i32>
```

首先我们将 tile 大小指定为常量。在这种情况下, 由于我们使用的是方阵的方形 tile, 所有值都等同于 64。Tile IR 中的常量可以是张量值, 这里我们创建包含单个标量值的 0 维张量。

```
%range_start = cuda_tile.constant <i32: 0> : tile<i32>
%range_step = cuda_tile.constant <i32: 1> : tile<i32>
%init_accum = cuda_tile.constant <f32: 0.000000e+00> : tile<64x64xf32>
```

接下来我们为循环初始化常量, 稍后将详细讨论, 并为 MMA (矩阵乘法累加) 零初始化一个累加器值。值得注意的是, 这里的张量是一个值, Tile IR 编译器将处理分配和执行。

```
%tile_size_range = cuda_tile.iota : tile<64xi32>
```

我们还使用cuda\_tile.iota为归约维度再次创建了一个从 (0, 63) 的共享范围。

```
%a_tile_base = cuda_tile.muli %block_x_index, %k_tile_size : tile<i32>
%a_tile_base_reshape = cuda_tile.reshape %a_tile_base : tile<i32> -> tile<1xi32>
```

```
%a_tile_base_tensor = cuda_tile.broadcast %a_tile_base_reshape :
    tile<1xi32> -> tile<64xi32>
%m_offsets_vec = cuda_tile.addi %a_tile_base_tensor, %tile_size_range : tile<64
xi32>
```

我们必须首先计算 A 和 B 的偏移张量，以便获取每个张量的指针张量，从而从内存中加载它们。概念上，我们首先使用下面的 Python 伪代码计算 M 维度的偏移量：

```
m_offsets = block_x_index * k_tile_size + arange(0, k_tile_size)
```

这会生成一个从 tile“顶部角落”开始的向量，该角落位于  $(\text{block\_x\_index} * \text{k\_tile\_size}, \text{block\_x\_index} * \text{k\_tile\_size} + \text{k\_tile\_size})$ 。

A 矩阵的步幅是 (64, 1)，这意味着我们可以省略 K 维度上额外的乘以 1 操作，因为偏移矩阵中每一行的值都是连续的。

对于这个，K 维度上的偏移量将是  $\text{k\_offs} = \text{arange}(0, \text{k\_tile\_size})$ ，因此我们可以在下面重用  $\text{\%tile\_size\_range}$ 。

我们可以再次使用以下 Python 伪代码来计算偏移矩阵：

```
a_tile = reshape(m_offsets, (64, 1)) * m_stride + reshape(k_offsets, (1, 64)) *
    k_stride
```

在像 Python 的 NumPy 这样的库中，这种计算隐藏了必须在 **Tile IR** 中展开的隐式广播。

我们将分两步完成：首先计算沿 M 维度的偏移量，再计算沿 K 维度的相对偏移量，然后将它们相加，生成正确的偏移张量。

我们将  $\text{m\_offsets}$  广播成一个矩阵，其中每一列都相同并按步长进行缩放。

```
%m_offsets_matrix = cuda_tile.reshape %m_offsets_vec :
    tile<64xi32> -> tile<64x1xi32>
%m_offsets_broadcast = cuda_tile.broadcast %m_offsets_matrix :
    tile<64x1xi32> -> tile<64x64xi32>
%m_offsets = cuda_tile.muli %m_offsets_broadcast, %m_stride_factor : tile<64
x64xi32>
```

现在按 64 进行缩放的矩阵将包含以下值：

```
[[ 0, 0, 0, ..., 0, 0, 0],
 [ 64, 64, 64, ..., 64, 64, 64],
 [ 128, 128, 128, ..., 128, 128, 128],
 ...,
 [3904, 3904, 3904, ..., 3904, 3904, 3904],
 [3968, 3968, 3968, ..., 3968, 3968, 3968],
 [4032, 4032, 4032, ..., 4032, 4032, 4032]]
```

然后将  $\text{k\_offsets}$  广播成一个矩阵，其中每一行都相同，并按步长进行缩放。

```
%ak_offsets_matrix = cuda_tile.reshape %tile_size_range :
    tile<64xi32> -> tile<1x64xi32>
%ak_offsets_broadcast = cuda_tile.broadcast %ak_offsets_matrix :
    tile<1x64xi32> -> tile<64x64xi32>
%ak_offsets = cuda_tile.muli %ak_offsets_broadcast, %m_stride_factor : tile<64
x64xi32>
```

A 是一个行主序矩阵（即 K 维的步长为 1），因此 K 偏移量包含连续的值。

```
[[ 0, 1, 2, ..., 61, 62, 63],
 [ 0, 1, 2, ..., 61, 62, 63],
 [ 0, 1, 2, ..., 61, 62, 63],
 ...,
 [ 0, 1, 2, ..., 61, 62, 63],
 [ 0, 1, 2, ..., 61, 62, 63],
 [ 0, 1, 2, ..., 61, 62, 63]]
```

我们将它们相加，得到的结果中，第 i 行从  $\text{m\_offsets}$  的第 i 个值开始，每列包含接下来的 63 个整数。

```
%a_tile_offsets = cuda_tile.addi %m_offsets, %ak_offsets : tile<64x64xi32>
```

```
[[ 0, 1, 2, ..., 61, 62, 63],
 [ 64, 65, 66, ..., 125, 126, 127],
 [ 128, 129, 130, ..., 189, 190, 191],
 ...,
 [3904, 3905, 3906, ..., 3965, 3966, 3967],
 [3968, 3969, 3970, ..., 4029, 4030, 4031],
 [4032, 4033, 4034, ..., 4093, 4094, 4095]]
```

请注意，由于这是输出矩阵的第 0 个 tile，它以 (0, 0) 为中心，下一个 tile 将以 (64, 0) 为中心，然后是 (128, 0)，依此类推。基于网格坐标，上述计算将在该点生成 64 x 64 的网格。  
现在我们同样操作 B。

```
n_offs = j * n_tile_size + torch.arange(0, n_tile_size)
b_tile = k_offs[:, None] * k_stride + n_offs[None, :] * n_stride
```

```
%b_tile_base = cuda_tile.muli %block_y_index, %k_tile_size : tile<i32>
%b_tile_base_reshape = cuda_tile.reshape %b_tile_base :
    tile<i32> -> tile<1xi32>
%b_tile_base_tensor = cuda_tile.broadcast %b_tile_base_reshape :
    tile<1xi32> -> tile<64xi32>
%n_offsets_vec = cuda_tile.addi %b_tile_base_tensor, %tile_size_range : tile<64
xi32>
```

```
%bk_offsets_matrix = cuda_tile.reshape %tile_size_range : tile<64xi32> -> tile
<64x1xi32>
%bk_offsets = cuda_tile.broadcast %bk_offsets_matrix : tile<64x1xi32> -> tile<64
x64xi32>
```

```
%n_offsets_matrix = cuda_tile.reshape %n_offsets_vec : tile<64xi32> -> tile<1
x64xi32>
%n_offsets_broadcast = cuda_tile.broadcast %n_offsets_matrix : tile<1x64xi32>
-> tile<64x64xi32>
%n_offsets = cuda_tile.muli %n_offsets_broadcast, %m_stride_factor : tile<64
x64xi32>
%b_tile_offsets = cuda_tile.muli %bk_offsets, %n_offsets : tile<64x64xi32>
```

现在我们已经计算了指针的初始偏移量，我们只需像之前一样转换基指针并加上偏移量。

```
%a_ptr_base_tensor = cuda_tile.reshape %a_ptr_base_scalar :
tile<ptr<f32>> -> tile<1x1xptr<f32>>
%a_ptr = cuda_tile.broadcast %a_ptr_base_tensor : tile<1x1xptr<f32>> -> tile<64
x64xptr<f32>>
%a_tile_ptr = offset %a_ptr, %a_tile_offsets :
tile<64x64xptr<f32>>, tile<64x64xi32> -> tile<64x64xptr<f32>>
```

大型矩阵运算被分解为由块网格处理的较小图块，需要仔细计算每个块的全局内存偏移量。

## 2.2.4 遍历图块

现在，在完成所有准备工作之后，我们可以执行内核的核心计算。

```
%C_tile, %a_ptr_final, %b_ptr_final = for %k in (%range_start to %k_tile_size,
step %range_start) : tile<i32>
    iter_values(
        %acc_prev = %init_accum,
        %a_tile_ptr_prev = %a_tile_ptr,
        %b_tile_ptr_prev = %b_tile_ptr
    ) -> (tile<64x64xf32>, tile<64x64xptr<f32>>, tile<64x64xptr<f32>>)
{
    // Load a single 64x64 matrix from the tile.
    %A_tile, %token_a = load_ptr_tko weak %a_tile_ptr :
        tile<64x64xptr<f32>> -> tile<64x64xf32>, token
    // Load a single 64x64 matrix from the tile.
```

```

%B_tile, %token_b = load_ptr_tko weak %b_tile_ptr :
    tile<64x64xptr<f32>> -> tile<64x64xf32>, token
%C_tile_acc = mmf %A_tile, %B_tile, %acc_prev: tile<64x64xf32>, tile<64
x64xf32>, tile<64x64xf32>

// Advance by K block size.
%block_size = constant <i32: 64> : tile<64x64xi32>
%a_tile_ptr_next = offset %a_tile_ptr_prev, %block_size
    : tile<64x64xptr<f32>>, tile<64x64xi32>
    -> tile<64x64xptr<f32>>
%b_tile_ptr_next = offset %b_tile_ptr_prev, %block_size
    : tile<64x64xptr<f32>>, tile<64x64xi32>
    -> tile<64x64xptr<f32>>
// Store the partial sum to the 64x64 accumulator.
continue %C_tile_acc, %a_tile_ptr_next, %b_tile_ptr_next : tile<64x64xf32>,
    tile<64x64xptr<f32>>, tile<64x64xptr<f32>>
}

```

在完成对 K 维度的归约后，我们需要将输出块存储到 C 矩阵中。

我们采用与处理 A 和 B 时相同的方法，但这次计算略有不同，因为我们可以仅使用块坐标来计算 X 和 Y 两个维度。直观地理解，平铺的“列”

“rows” produce a single output tile.

```

cm_offsets = block_x_index * BLOCK_SIZE_M + arange(0, BLOCK_SIZE_M)
cn_offsets = pid_n * BLOCK_SIZE_N + arange(0, BLOCK_SIZE_N)
c_tile = c_ptr + stride_cm * reshape(cm_offsets, (64, 1)) + stride_cn * reshape(
    cn_offsets, (64, 1))

```

我们省略了 %c\_tile\_offset 的索引计算，然后像之前一样将其添加到基指针中。

```

%c_tile_offsets = muli %c_tile_x_offsets, %c_tile_y_offsets : tile<64x64xi32>
%c_ptr_base_tensor = reshape %c_ptr_base_scalar :
    tile<ptr<f32>> -> tile<1x1xptr<f32>>
%c_ptr = broadcast %c_ptr_base_tensor :
    tile<1x1xptr<f32>> -> tile<64x64xptr<f32>>
%c_tile_ptr = offset %c_ptr, %c_tile_offsets :
    tile<64x64xptr<f32>>, tile<64x64xi32> -> tile<64x64xptr<f32>>

```

然后我们可以像在所有先前内核中一样存储该值。

```

store_ptr_tko weak %c_tile_ptr, %C_tile :
    tile<64x64xptr<f32>>, tile<64x64xf32> -> token

```

结构化控制流允许在归约维度上进行高效迭代，在将最终输出块写入内存之前，将结果累积在寄存器中。

## 2.3 结构化指针

之前的例子已经探讨了如何使用指针张量和分散/聚集风格的加载与存储来定义矩阵乘法，这些操作接受任意的指针张量进行操作。这些操作为程序员提供了最大的表达能力，但以潜在的性能为代价。

这些对于理解很有价值；正如我们在上一节中所看到的，你可以构建任意的指针张量，然后灵活地对它们进行加载和存储。

例如，如果用户创建了一个完全不相交的指针张量，那么无论是人类还是编译器都很难从该程序中获得有意义的性能。在最坏的情况下，每个元素都将成为一个完全不相交的内存操作，从而阻碍向量化或张量化操作，也无法利用缓存或线程局部性的代码。

这种灵活性是以需要更多设置和手动计算来实现即使相对简单的算法（如分块 GEMM）为代价的，并且可能效率低下。由于加载/存储操作在退化情况下可以处理任意张量，它们很可能被分解为一系列任意的加载和存储操作，这无法充分利用内存局部性或底层硬件。

**Tile IR** 将尽力通过这些存储获得良好性能，但使用结构化指针，在 **Tile IR** 中称为 **tensor view**，可以更轻松地实现最佳性能。张量视图使我们能够简化 **Tile IR** 的编程模型，并提高用户程序的效率。

正如我们在之前的例子中所见，当内核接收到一个张量时，它是一个指向全局内存的标量基指针。

指向 A 的基指针指向一个位于全局内存中的分配。#

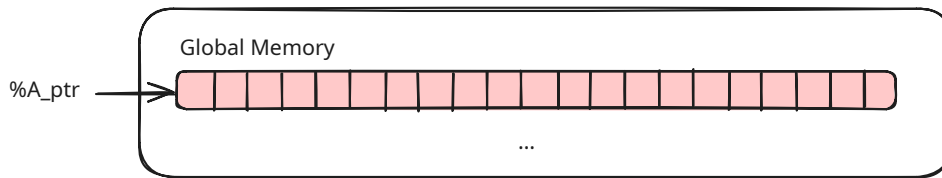


Figure 1: 全局内存视图

通过用正确的尺寸重新表述我们之前的问题，我们完全绕过了复杂的偏移计算，并避免了处理不完美的平铺或存储/加载掩码。更多掩码的细节见 XX。

在构建张量视图时，我们使用静态或动态的形状和步幅信息将原始指针转换为张量。

为了简化 **Tile IR** 的编程模型并提高用户程序的效率，**Tile IR** 还引入了一种称为张量视图的结构化指针类型。张量视图可以通过 `cuda_tile.make_tensor_view` 从原始指针构建，它将形状和步长信息附加到指针上，有效地表示一个指向张量的类型化指针。

在构建张量视图时，我们使用静态或动态的形状和步幅信息将原始指针转换为张量，这是通过 `cuda_tile.make_tensor_view` 完成的。此操作将形状和步幅信息附加到指针上，有效地将类型化指针转换为张量。

```
%A_ref = make_memref %ptr, shape = [%M, %K], strides = [%stride_am, 1]
```

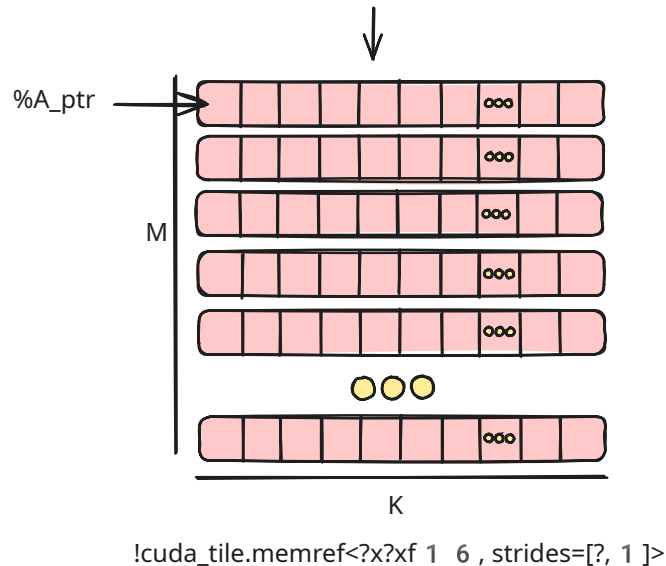


Figure 2: 全局内存视图

指向 A 的基指针指向一个位于全局内存中的分配。#

结构化指针，或称张量视图，封装了形状和步幅信息，使编译器能够优化内存访问并简化用户的代码。

## 2.4 Tiling & Views

一旦你构建了张量视图，我们就可以利用 `cuda_tile.make_partition_view` 来对底层张量进行分块处理。

指向 A 的基指针指向一个位于全局内存中的分配。#

在上一节中，我们通过选择使问题完全正方形的 tile 尺寸、使用相同的布局、简单的 tile 划分和静态输入维度等，简化了问题。

我们将放宽这些限制，以在探索视图和分区时展示更复杂的 tile 映射。

### 2.4.1 使用视图进行向量加法

我们现在可以实现一个更复杂的向量操作，即 SAXPY 或“单精度  $A \cdot X$  加  $Y$ ”（一种常见的

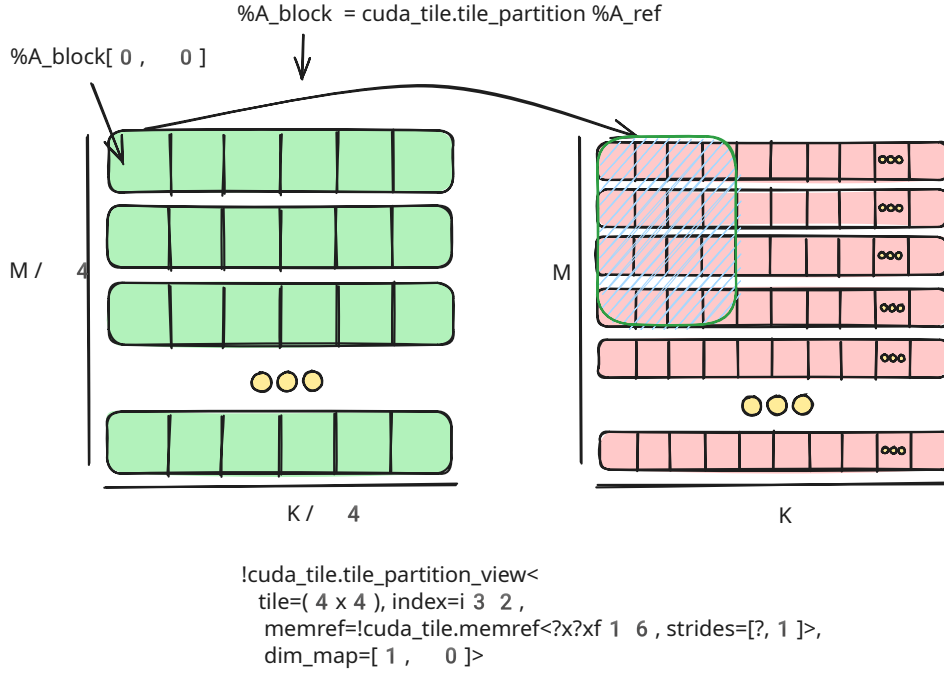


Figure 3: A view of global memory

BLAS 操作)。我们可以使用 `tensor view` 和 `cuda_tile.make_partition_view` 来为任意大小的向量实现此操作。

内核首先定义其参数。我们现在将输入 `%X`、`%Y`、`%alpha` 以及完整向量的维度作为参数。

```
entry @saxpy_memref(%X: tile<ptr<f32>>,
                    %Y: tile<ptr<f32>>,
                    %alpha: tile<f32>,
                    %M : tile<i32>,
                    %N : tile<i32>) {
```

对于那些熟悉其他平铺编程模型的人来说，你可能会好奇为什么我们需要将输入张量的大小作为参数，而不是用它们来控制块的数量，并在程序中保持隐式。

张量视图模型与一些现有的 tile 编程模型不同，在那些模型中，每个内核除了需要屏蔽加载和存储操作外，对整个问题规模的维度一无所知。相比之下，`tensor view` 需要知道张量的整体维度，以便能够自动高效且正确地降级处理 tile 及其相关操作。

```
%x_memref = make_tensor_view %X, shape = [%M, %N], strides = [%M, 1] : tile<i32>
-> tensor_view<?x?xf32, strides=[?,1]>
%y_memref = make_tensor_view %Y, shape = [%M, %N], strides = [%M, 1] : tile<i32>
-> tensor_view<?x?xf32, strides=[?,1]>
```

张量视图的构造函数动态接收形状和步长，从而生成一个动态形状的张量视图，这在类型中由 `?` 表示，例如：`!cuda_tile.tile view<?x?xf32, strides=[?,1]>`。

我们使用 `cuda_tile.make_partition_view` 为 `%x` 和 `%y` 创建视图，这些视图表示一个  $(M/128 \times N/256)$  的 tile 张量。每个 tile 的大小由分区类型的 tile 参数指定。

```
%x_view = make_partition_view %x_memref : partition_view<tile=(128x256),
tensor_view<?x?xf32, strides=[?,1]>>
%y_view = make_partition_view %y_memref : partition_view<tile=(128x256),
tensor_view<?x?xf32, strides=[?,1]>>
```

`cuda_tile.load_view_tko` 允许我们为 `%X` 和 `%Y` 加载由 `%view[%x, %y]` 指定的图块。

```
%x_tile, %token_x = load_view_tko weak %x_view[%tileIdX, %tileIdY] :
partition_view<tile=(128x256), tensor_view<?x?xf32, strides=[?,1]>>, tile<i32>
-> tile<128x256xf32>, token
```



然后我们可以直接使用这些 tile 进行计算，以获得我们的结果 tile。

```
// Step 6. Compute sAXPY: y = alpha * A + y
```

最后我们将结果直接累加到 %Y 中。在这里，我们将其视为该内核中的输入和输出参数。

将张量视图与分区结合简化了内核实现，允许直接加载和操作逻辑分块，无需手动计算偏移量。

## 2.4.2 使用视图的动态 GEMM

我们现在已经介绍了 **Tile IR** 的核心概念。我们可以将这些想法结合起来，利用结构化指针来支持动态的 GEMM 内核。首先，我们对此 GEMM 进行两处修改：

输入实际上被转置为列主序布局，且输入为 fp16 格式，而输出为 fp32 格式。

我们将输入和输出张量（作为指针）、所有维度和所有步长作为参数。

```
entry @gemm_kloop_kernel(  
    %A_ptr: !cuda_tile.tile<!cuda_tile.ptr<f16>>,  
    %B_ptr: !cuda_tile.tile<!cuda_tile.ptr<f16>>,  
    %C_ptr: !cuda_tile.tile<!cuda_tile.ptr<f32>>,  
    %M: !cuda_tile.tile<i32>, %N: !cuda_tile.tile<i32>, %K: !cuda_tile.tile<i32>,  
    %stride_ak: !cuda_tile.tile<i32>, %stride_bn: !cuda_tile.tile<i32>, %  
    stride_cm: !cuda_tile.tile<i32>
```

**Tile IR** 编译器可以在已知底层指针和步幅的对齐方式时，优化 Tensor View 的内存加载和存储。如果这些信息是静态已知的，我们可以直接推断对齐方式；但如果它们是动态的（如此例所示），我们可以使用 `cuda_tile.assume` 操作来告知编译器指针已正确对齐。

这里我们使用 `div_by` 谓词来告知编译器这些值的可除性，这可用于直接推断对齐约束。

```
%A_ptr_assume = assume #cuda_tile.div_by<16>, %A_ptr : tile<ptr<f16>>  
%B_ptr_assume = assume #cuda_tile.div_by<16>, %B_ptr : tile<ptr<f16>>  
%C_ptr_assume = assume #cuda_tile.div_by<16>, %C_ptr : tile<ptr<f32>>  
%stride_ak_assume = assume #cuda_tile.div_by<8>, %stride_ak : tile<i32>  
%stride_bn_assume = assume #cuda_tile.div_by<8>, %stride_bn : tile<i32>  
%stride_cm_assume = assume #cuda_tile.div_by<8>, %stride_cm : tile<i32>
```

我们为 %A、%B 和 %C 创建张量视图。

```
// A reference to the A tensor pointed to by A_ptr, (K x M)  
%A = make_tensor_view %A_ptr_assume, shape = [%K, %M], strides = [%stride_ak, 1]  
: tile<i32> -> tensor_view<?x?xf16, strides=[?,1]>  
// A reference to the B tensor pointed to by B_ptr, (N x K)  
%B = make_tensor_view %B_ptr_assume, shape = [%N, %K], strides = [%stride_bn, 1]  
: tile<i32> -> tensor_view<?x?xf16, strides=[?,1]>  
// A reference to the C tensor pointed to by C_ptr, (M x N)  
%C = make_tensor_view %C_ptr_assume, shape = [%M, %N], strides = [%stride_cm, 1]  
: tile<i32> -> tensor_view<?x?xf32, strides=[?,1]>
```

我们为每个张量创建一个分区视图。首先是张量 A：

```
%A_block = make_partition_view %A : partition_view<tile=(128x64), tensor_view<?  
x?xf16, strides=[?,1]>, dim_map=[1, 0]>
```

Then B:

```
%B_block = make_partition_view %B : partition_view<tile=(64x128), tensor_view<?  
x?xf16, strides=[?,1]>, dim_map=[1, 0]>
```

And last C:

```
%C_block = make_partition_view %C : partition_view<tile=(128x128), tensor_view  
<?x?xf32, strides=[?,1]>, dim_map=[0, 1]>
```

```
// Load a single 64x128 matrix from the tile.
%B_frag, %t2 = load_view_tko weak %B_block [%k, %bidx] : partition_view<tile=(64
x128), tensor_view<?x?xf16, strides=[?,1]>, dim_map=[1, 0]>, tile<i32> ->
tile<64x128xf16>, token
```

然后我们使用 `cuda_tile.get_tile_block_id` 读取图块网格坐标。

```
%bidx, %bidx, %bidx = get_tile_block_id : tile<i32>
```

由于 K 维度是动态的，我们本可以自行计算归约索引空间的大小，但 **Tile IR** 提供了一个操作来为我们完成这项工作。`cuda_tile.get_index_space_shape` 可用于获取索引空间的大小，从而确定归约循环的规模。

```
%mk_len_i32:2 = get_index_space_shape %A_block : partition_view<tile=(128x64),
tensor_view<?x?xf16, strides=[?,1]>, dim_map=[1, 0]> -> tile<i32>
```

现在，我们可以使用 `cuda_tile.for` 循环来遍历图块。

`for` 循环是 **Tile IR** 中的一种结构化控制流操作，它使循环变量在（起始值，结束值，步长）定义的范围内逐步遍历，并为范围内的每个值执行循环体。

`iter_values` 是循环体的循环传递变量，它们在循环头中被初始化，并通过使用 `cuda_tile.continue` 操作产生它们来在每次迭代中更新。

为了实现归约，我们从 0 开始，遍历平铺后的 K 维度大小，每一步均匀地增加 1，并使用一个单循环携带变量来表示累加器平铺块。

```
%result = for %k in (%i0 to %mk_len_i32#1, step %i1): tile<i32>
iter_values(%acc_prev = %cst) -> (tile<128x128xf32>)
```

We load tiles from A and B.

```
// Load a single 128x64 matrix from the tile.
%A_frag, %t1 = load_view_tko weak %A_block [%bidx, %k] : partition_view<tile=(128
x64), tensor_view<?x?xf16, strides=[?,1]>, dim_map=[1, 0]>, tile<i32> ->
tile<128x64xf16>, token
// Load a single 64x128 matrix from the tile.
%B_frag, %t2 = load_view_tko weak %B_block [%k, %bidx] : partition_view<tile=(64
x128), tensor_view<?x?xf16, strides=[?,1]>, dim_map=[1, 0]>, tile<i32> ->
tile<64x128xf16>, token
```

我们计算 MMA，然后使用该值继续下一个循环迭代。

```
%acc = mmf %A_frag, %B_frag, %acc_prev: tile<128x64xf16>, tile<64x128xf16>,
tile<128x128xf32>
continue %acc : tile<128x128xf32>
```

最后，与我们之前的实现类似，在循环外部，我们这次通过视图将 tile 存储回 %C，从而避免了再次计算偏移量的需要。

```
%t3 = store_view_tko weak %result, %C_block [%bidx, %bidx] : tile<128x128xf32>,
partition_view<tile=(128x128), tensor_view<?x?xf32, strides=[?,1]>, dim_map
=[0, 1]>, tile<i32> -> token
```

Tensor 视图支持动态形状和步幅，能够实现稳健、灵活的内核，以处理不同输入尺寸的同时保持高性能。

## 2.5 跨 TileBlock 通信

```
cuda_tile.module @hello_cross_block {
  global @_global_printf_mutex <i32: 1> : tile<1xi32>
  entry @hello_cross_block_kernel() {
    %idx, %idy, %idz = get_tile_block_id : tile<i32>
    %tilex, %tiley, %tilez = get_num_tile_blocks : tile<i32>
    %2 = get_global @_global_printf_mutex : tile<ptr<i32>>
    %3 = cuda_tile.constant <i32: 0> : tile<i32>
    %4 = cuda_tile.constant <i32: 1> : tile<i32>
```

```

loop {
    %t1 = make_token : token
    %6, %t2 = atomic_cas_tko relaxed device %2, %4, %3 token=%t1: tile<!
        cuda_tile.ptr<i32>>, tile<i32> -> tile<i32>, !cuda_tile.token
    %7 = trunci %6 : tile<i32> -> tile<i1>
    if %7 {
        break
    }
}
print "current tile: %i / %i\0A", %idx, %tilex : tile<i32>, tile<i32>
%5, %t4 = atomic_rmw_tko relaxed device %2, xchg, %4: tile<ptr<i32>>,
    tile<i32> -> tile<i32>, !cuda_tile.token
return
}
}

```

### 3 语法

**Tile IR** 使用 **Tile IR** MLIR 方言构建，并以字节码形式存储。

为了让人类能够理解 **Tile IR** 字节码程序，我们提供了一种基于 MLIR 方言的**不稳定**文本表示形式。该文本表示没有稳定性保证，不适用于编写 **Tile IR** 程序。

#### 3.1 模块

一个 **Tile IR** 程序由一个包含一系列项目的 **Tile IR** 模块组成。

```
symbol_name := `@` identifier
cuda_tile.module @symbol_name {
    <items>*
}
```

#### 3.2 条目

一个条目是一个内核定义或全局变量定义。

```
<items> ::= <kernel_definition> | <global_variable_definition>
```

#### 3.3 全局变量

全局变量定义是在内核外部定义的变量。

```
global_variable_definition ::= `global` <symbol_name> `:` <type> `=` <value>
```

#### 3.4 内核

内核定义是一个在 **Tile IR** 模块内部定义的函数。

```
ssa_name := `%` identifier
function_signature ::= <function_parameter>*
function_parameter ::= <ssa_name> `:` <type>
<kernel_definition> ::= `func` @kernel_name `(` <function_signature> `)` -> <
    function_type> {
    <kernel_body>
}
```

内核定义的主体是一系列操作。

```
kernel_body ::= <operation>*
operation ::= (ssa_name `,`?)* `=` <operation_name> <ssa_name>* attribute=
    attribute_value : type ...
```

#### 3.5 类型

**Tile IR** 中的类型是一组固定的预定义类型。

```
element_type ::= `f32` | `f64` | `i8` | `i16` | `i32` | `i64` | `b8` | `b16` | `
    b32` | `b64`
type ::= `tile` `<` shape `x` element_type `>`
shape ::= `[` integer_literal (`x` integer_literal)* `]`
```

## 4 二进制格式

**Tile IR** 字节码是 **Tile IR** 模块的二进制表示，包含所有项目（全局变量、内核、函数）、属性和指令。

字节码格式是稳定的、版本化的，并提供 **Tile IR** 可移植性保证。

旧版 **Tile IR** 编译器/驱动程序生成的字节码格式可以被新版编译器/驱动程序读取。

编译器/驱动程序接受其支持的 **Tile IR** 版本以下的字节码。

本节剩余部分描述了 **Tile IR** 字节码格式及其编码方式。

该字节码的设计遵循若干顶层目标：

- 表示一个有限但可随时间扩展的操作集合。
- 使用最小类型集合进行编码。
- 为了支持人工手动构建和检查。
- 支持函数的延迟加载。
- 支持编码的前向和后向兼容性。

单个操作的编码在操作中有详细描述。

### 4.1 原语

字节码由少量基本类型组成，每种类型都有其自身的编码方式，如下所述。该格式利用这些基本类型来编码更复杂的数据结构，以表示完整的 **Tile IR** 项目集。

#### 4.1.1 固定宽度整数

固定宽度整数是已知大小（以字节为单位）的无符号整数。这些值以小端字节序存储。

**注意：** **Tile IR** 字节码格式中的所有多字节值均采用小端编码，包括固定宽度整数、数组偏移量和类型索引。

```
byte ::= `0x00`...`0xFF`
```

#### 4.1.2 可变宽度整数

可变宽度整数，或 **VarInt**，为整数提供了一种紧凑的表示方式。

每个编码的 **VarInt** 由一到九个字节组成，这些字节共同表示一个单一的

64 位值。**Tile IR** 字节码对 **VarInt** 使用 **PrefixVarInt** 编码。

这种编码是 **LEB128**（“小端基数 128”）编码的一种变体，其中编码的每个字节最多提供 7 位用于表示数值，剩余位用于存储一个标记，指示编码所使用的字节数。小的无符号整数

（小于  $2^7$  的）可以存储在一个字节中，较大的无符号整数（最多  $2^{14}$ ）可以存储在两个字节中，依此类推。

```
varint ::= `0x00`...`0xFF`
```

### 4.2 文件结构

**Tile IR** 字节码文件由字节流组成；字节流由头部和多个节（section）构成。头部包含 8 字节的“魔数”和一个版本号。

每个节区预计在字节码文件中出现一次，并通过类型代码、长度、对齐方式、填充和有效载荷来标识。这种设计实现了前向兼容性和选择性解析（工具可以跳过未知节区或不需要的节区）。

```
bytecode {  
  magic: "\x7FTileIR\x00",  
  version: varint,  
  sections: section[]  
}
```

### 4.2.1 魔数

**Tile IR** 字节码的魔数占用 8 字节，为 `\x7FTileIR\x00`。该魔数必须出现在字节码文件的开头，才能被驱动程序接受。

### 4.2.2 版本

在魔数之后是字节码格式的版本号。版本号允许格式向前和向后兼容。

版本号编码为紧跟在魔数之后的三个小端固定宽度字段：

```
version {  
    major: uint8_t,      // Major version number  
    minor: uint8_t,      // Minor version number  
    tag: uint16_t         // Version tag (little-endian)  
}
```

每个文件都编码了版本信息，这使得解析器能够适应特定的版本。

我们不会破坏旧版本的格式，如果新文件使用了旧解析器无法解释的功能，它会跳过未知的可选功能，但在不支持新功能的情况下会优雅地失败。新的生产者也可以直接针对旧版本的字节码，以确保与旧驱动程序的兼容性。

### 版本兼容性保证

1. 向后兼容性（新的反序列化器能够读取旧字节码）：
  - 反序列化器必须支持字节码格式的所有先前版本。
  - 反序列化器必须处理所有先前存在的操作码、类型和节段。
  - 反序列化器必须保持原始程序语义。
2. 前向兼容性（旧的反序列化器能够读取新的字节码）：
  - 反序列化器仅在遇到未知必需功能时必须失败。
  - 反序列化器必须仅读取其版本范围内的字节码。
  - 若存在版本不匹配，反序列化器必须通过清晰消息报错。
3. 版本定向（序列化器可以针对特定的旧版本）：
  - 序列化器可以针对一个或多个特定的旧版本。
  - 序列化器必须验证所请求的目标版本支持所有功能。
  - 如果尝试序列化不支持的功能，序列化器必须失败。
4. 可选与必需特性（必需变更已明确记录）：
  - 必需变更必须在新字节码版本中引入，且必须设计为优雅降级。
  - 可选变更必须明确记录，并必须保留上述保证。

### Examples :

- For new ops → Add new opcodes
- 对于现有操作的更改 → 通常保留旧格式以保持向后兼容性，并添加新的操作码而非修改现有的操作码

### 4.2.3 Sections

字节码流的其余部分由多个节组成。节用于在字节码内对数据进行分组，并允许对流进行按节操作，从而实现数据的乱序处理和/或延迟加载。

每个部分包含一个部分 ID（其高位指示该部分是否有对齐要求）、长度以及一个可选的对齐要求。部分 ID 是一个 7 位整数，其高位指示该部分是否有对齐要求。当存在对齐要求时，会有可变数量的填充字节（每个字节 = `0xCB`）。

可能出现在节数据之前。节的对齐必须是 2 的幂。对齐以 `VarInt` 表示。填充确保节数据从指定的对齐边界开始。

```

section {
    idAndIsAligned: byte    // low 7 bits = section ID, high bit = alignment bit
    length: varint,
    alignment: varint?,     // present only if high bit was set
    padding: byte[],       // bytes of 0xCB as needed
    data: byte[]
}

```

**反序列化** 以下是章节格式对反序列化过程的影响。

1. 反序列化器必须将段 ID 和长度 (idAndIsAligned) 作为单个字节读取。
2. 如果节 ID 的高位被设置, 反序列化器必须读取对齐值 (alignment) 并根据需要跳过 0xCB 字节来应用对齐。
3. 反序列化器必须读取载荷的长度 (length) 字节。该长度是一个 VarInt。
4. 反序列化器必须继续前进到下一部分, 直到遇到 EOF (文件结束符)。

字符串部分保存了模块使用的所有文本名称, 以避免在行内重复它们。

该部分编码包含字符串总数, 随后是每个独立字符串的起始索引。其余编码包含一个拼接所有字符串的单一数据块。这种设计允许在不读取整个部分的情况下加载特定字符串。字节码中的字符串以原始字节序列 (采用 UTF-8 编码) 形式存储, 并附带关联的大小信息。查找第 *i* 个字符串需要跳转到 `stringStartIndex[i]` 并读取至下一个偏移量或数据块末尾。

```

strings {
    numStrings: varint,
    stringStartIndex: uint32[],
    stringData: byte[]
}

```

**函数表部分** 函数表部分枚举了模块的函数并内联嵌入其代码。首先, `numFunctions` 指示后续有多少个函数; 对于每个函数 *i*, `nameIndex[i]` 是指向字符串部分中函数名称的索引, `signatureIndex[i]` 是指向类型部分中指定其参数、返回类型、全局标志等的索引 (因此具有相同签名的多个函数可以共享同一个类型条目)。字段 `functionLocIndex[i]` 是一个 VarInt, 引用调试部分中描述函数定义位置 (例如源文件和行号) 的条目。如果 `functionLocIndex[i]` 为零, 则表示该函数的定义范围没有关联的调试元数据。 `lengthOfFunction[i]` 说明函数 *i* 有多少字节的代码, 而 `functionBody[i]` 则包含恰好这些字节的指令编码。这种布局避免了单独的代码部分, 并使解析每个函数变得简单: 一旦读取了函数 *i* 的元数据, 您可以直接解析其指令, 或者通过跳过 `lengthOfFunction[i]` 字节来忽略它们。指令编码本身允许每个操作包含其源位置的槽位, 引用调试部分中的条目。

指令编码 (操作码、操作数等) 稍后在 Operation Opcodes and Encodings Section 进行描述  
函数表编码已更新, 以包含函数标志和可选的优化提示:

```

functionTable {
    numFunctions : varint
    // for each function i in [0..numFunctions-1]:
    nameIndex[i] : varint                // References the function's name
                                         in the StringSec
    signatureIndex[i] : varint           // References the extended function
                                         signature in the TypeSec
    entryFlag[i] : byte                  // Function flags (visibility, kind
                                         , optimization hints)
    functionLocIndex[i] : varint          // 0 means no debug location
    optimizationHints[i]? : self-contained // Present only if
                                         HasOptimizationHints flag is set
    lengthOfFunction[i] : varint
    functionBody[i] : byte[lengthOfFunction[i]]
}

```

entryFlag 字节编码了以下信息：

- **Bit 0 (0x01)** : 可见性标志 (0 = 公开, 1 = 私有)
- **Bit 1 (0x02)** : 功能类型标志 (0 = 设备功能, 1 = 内核入口点)
- **Bit 2 (0x04)** : 优化提示标志位 (0 = 否, 1 = 是)
- **Bits 3-7** : Reserved for future extensions

常量数据区将大型常量（例如，密集张量、大型数组）与代码分开存储。

当前实现未对单个常量施加明确的大小限制。

常量使用 `uint64_t` 偏移量存储，允许存储非常大的常量数据。

然而，实际限制可能受到可用内存以及任何下游编译器或运行时限制（例如 CUBIN 生成约束）的影响。

正如我们对字符串部分所做的那样，我们将常量移动到它们自己的部分，以避免函数表部分膨胀，并允许在需要时惰性加载特定常量。

单个操作可以通过索引此部分来引用常量。

该部分编码包含常量总数，随后是每个独立常量的起始索引。其余编码包含一个单一的二进制大对象，其中所有常量被串联在一起。

```
constant {
    numConstants: varint,
    constantStartIndex: uint64_t[],
    constantData: byte[]
}
```

**类型节** 类型段存储模块中使用的所有类型定义（标量类型、函数签名、参数化类型等）。该段以 `numTypes` 开头，指定后续类型的数量，然后是一个偏移量数组 (`typeStartIndex`)，指向编码数据块 (`typeData`)。要加载第 `i` 个类型定义，您使用 `typeStartIndex[i]` 中包含的偏移量来索引 `typeData` 数据块，并从那里解析定义。`typeData` 数据块是一个单一的编码二进制数据块，其中包含连接在一起的所有类型定义。

```
type {
    numTypes: varint
    typeStartIndex: uint32_t[] // array of offsets, length = numTypes
    typeData: byte[]           // concatenated bytes for all type definitions
}
```

每个独立的类型定义将由一个 `typeTag` 及其后跟的特定于该 `typeTag` 的预定义载荷结构组成。这种确定性的方法使得解析器能够仅基于 `typeTag` 来理解 `payload` 的布局。

```
typeTag : byte
payload : byte[lengthOfPayload]
```

- `typeTag` 表示类型的“种类”。它可以是简单的标量、函数签名，或是更复杂的结构（如张量）。
- `payload` 根据 `typeTag` 进行解析。例如，函数签名可能存储参数数量、对参数类型的引用等，而张量类型可能存储维度和元素类型。

Example `typeTag` Values:

| typeTag | Meaning          | Expected Payload                                    |
|---------|------------------|-----------------------------------------------------|
| 0x01    | i1 (1-bit int)   | None                                                |
| 0x02    | i32 (32-bit int) | None                                                |
| 0x03    | i64 (64-bit int) | None                                                |
| 0x04    | tensor           | <code>elementTypeIndex + rank + dimensions[]</code> |
| 0x10    | Function         | Extended function signature                         |
| ...     | Future types     | Depends on the type                                 |



**详细类型编码** 以下部分描述了每种类型标签的具体编码格式：

**Integer Types ( typeTag = 0x00-0x04)**

整数类型除了类型标签外不需要额外的有效载荷数据：

```
I1:    typeTag = 0x00 // 1-bit boolean
I8:    typeTag = 0x01 // 8-bit integer
I16:   typeTag = 0x02 // 16-bit integer
I32:   typeTag = 0x03 // 32-bit integer
I64:   typeTag = 0x04 // 64-bit integer
```

**Float Types ( typeTag = 0x05-0x0B)**

浮点类型除了类型标签外不需要额外的负载数据：

```
F16:    typeTag = 0x05 // 16-bit IEEE float
BF16:   typeTag = 0x06 // 16-bit bfloat
F32:    typeTag = 0x07 // 32-bit IEEE float
TF32:   typeTag = 0x08 // TensorFlow-32
F64:    typeTag = 0x09 // 64-bit IEEE float
F8E4M3FN: typeTag = 0x0A // 8-bit float (E4M3FN format)
F8E5M2:  typeTag = 0x0B // 8-bit float (E5M2 format)
```

**Pointer Type ( typeTag = 0x0C)**

```
typeTag : byte = 0x0C // Pointer type
pointeeTypeIndex : varint // Index of the pointee type
```

**Tile Type ( typeTag = 0x0D)**

```
typeTag : byte = 0x0D // Tile type
elementTypeIndex : varint // Index of the element type
shape : int64_t[] // Shape dimensions (var-length array)
```

**TensorView Type ( typeTag = 0x0E)**

```
typeTag : byte = 0x0E // TensorView type
elementTypeIndex : varint // Index of the element type
shape : int64_t[] // Shape dimensions (var-length array)
strides : int64_t[] // Stride values (var-length array)
indexTypeTag : byte // Index type (I32=0x03 or I64=0x04)
```

**PartitionView Type ( typeTag = 0x0F)**

```
typeTag : byte = 0x0F // PartitionView type
tileShape : int32_t[] // Tile shape (var-length array)
tensorViewTypeIndex : varint // Index of the TensorView type
dimMap : int32_t[] // Dimension mapping (var-length array)
masked : byte // Masked flag (0x00=false, 0x01=true)
```

**Function Type ( typeTag = 0x10)**

```
typeTag : byte = 0x10 // Function Type
numParams : varint // Number of input parameters
paramTypeIndices : varint[] // Array of type indices for each parameter
numResults : varint // Number of results
resultTypeIndices : varint[] // Array of type indices for each return value
```

函数类型编码仅存储必要的类型信息（输入和结果类型）。

参数属性和其他函数元数据单独存储在函数表部分中。

**Token Type ( typeTag = 0x11)**

```
typeTag : byte = 0x11 // Token type (no additional payload)
```

**属性编码** Tile IR 字节码中的属性可以通过两种方式编码：

1. **内联编码** - 简单属性直接在指令流中编码
2. **自包含编码** - 复杂属性包含一个类型标签，后跟其数据

自包含属性使用以下格式：

```
attributeTag : byte
attributeData : byte[] // Format depends on attributeTag
```

The following attribute tags are supported:

**Integer Attribute** ( attributeTag = 0x01)

```
attributeTag : byte = 0x01 // Integer attribute
typeIndex : varint         // Type index for the integer type
value : varint             // Integer value (zero-extended)
```

**Float Attribute** ( attributeTag = 0x02)

```
attributeTag : byte = 0x02 // Float attribute
typeIndex : varint         // Type index for the float type
value : byte[]            // APFloat representation (variable length)
```

**Bool Attribute** ( attributeTag = 0x03)

```
attributeTag : byte = 0x03 // Bool attribute
value : byte           // 0x00=false, 0x01=true
```

**Type Attribute** ( attributeTag = 0x04)

```
attributeTag : byte = 0x04 // Type attribute
typeIndex : varint         // Index of the referenced type
```

**String Attribute** ( attributeTag = 0x05)

```
attributeTag : byte = 0x05 // String attribute
stringIndex : varint       // Index into the string section
```

**Array Attribute** ( attributeTag = 0x06)

```
attributeTag : byte = 0x06 // Array attribute
numElements : varint       // Number of elements
elements : self-contained[] // Array of self-contained attributes
```

**DenseElements 属性** ( attributeTag = 0x07)

```
attributeTag : byte = 0x07 // DenseElements attribute
typeIndex : varint         // Type index for the shaped type
constantIndex : varint     // Index into constant section (for numeric data)
// OR for string data:
numStrings : varint        // Number of string elements
stringIndices : varint[]   // Indices into string section
```

**DivBy Attribute** ( attributeTag = 0x08)

```
attributeTag : byte = 0x08 // DivBy attribute
divisor : varint           // Divisor value
flags : byte               // Bit 0: unsignedInt, Bit 1: hasEvery, Bit 2:
                           // hasAlong
every : signed_varint?     // Present if Bit 1 set
along : signed_varint?     // Present if Bit 2 set
```

**相同元素属性** ( attributeTag = 0x09)

```
attributeTag : byte = 0x09 // SameElements attribute
values : int64_t[]         // Array of int64 values (var-length)
```

### Dictionary Attribute ( attributeTag = 0x0A)

```
attributeTag : byte = 0x0A      // Dictionary attribute
numEntries : varint             // Number of key-value pairs
entries : dictEntry[]           // Array of dictionary entries
dictEntry {
  keyStringIndex : varint        // Index of key string
  value : self-contained        // Self-contained attribute value
}
```

### 优化提示属性 (attributeTag = 0x0B)

```
attributeTag : byte = 0x0B      // OptimizationHints attribute
dictionary : dictionary         // Dictionary attribute (without tag)
```

### 非负属性 (attributeTag = 0x0C)

```
attributeTag : byte = 0x0C      // NonNegative attribute
// No additional payload - presence indicates non-negative constraint
```

**全局节** 全局部分存储模块级别的全局变量。此部分是可选的，仅当模块包含 `cuda_tile.global` 操作时才会出现。

```
global {
  numGlobals: varint
  // for each global i in [0..numGlobals-1]:
  symbolNameIndex[i] : varint    // References the global's symbol name in the
    StringSec
  valueTypeIndex[i] : varint      // Type index for the global's value type
  constantValueIndex[i] : varint // Index into constant section for the global
    's initial value
}
```

Each global variable is encoded with:

- **symbolNameIndex**: 指向全局变量符号名称在字符串段中的索引
- **valueTypeIndex**: 指向全局变量类型在类型区段中的索引（通常是一个有形状的类型，如张量）
- **constantValueIndex**: 指向包含全局变量初始值的常量区索引

debug 部分存储了序列化的调试信息（有关调试信息的更多详情，请参阅 调试信息）。

此部分是可选的，因为某些工具可能会忽略它，且序列化器可能会在发布版本中将其留空。

```
debug {
  diOpsNum: varint              // Total number of operations with debug info
  padding: bytes                 // Align to 4 bytes
  diIndexOffsets: uint32_t[]     // Per op offset into the debug info indices
  diIndicesNum: varint           // Total number of debug info indices
  padding: bytes                 // Align to 4 bytes
  diIndices: uint64_t[]          // Array of debug indices to debug info attributes
  diAttrNum: varint              // Total number of debug info attributes
  padding: bytes                 // Align to 4 bytes
  diOffsets: uint32_t[]          // Per debug info attribute offset into the debug
    info data
  diData: bytes                  // Data for each debug info attribute
}
```

调试部分采用多级间接寻址方案：

1. **操作**: `diOpsNum` 个操作包含调试信息，其中 `diIndexOffsets` 指向索引数组
2. **索引**: `diIndices` 中共有 `diIndicesNum` 个索引，引用调试信息属性

3. **属性**: 存储在 `diData` 中的 `diAttrNum` 个调试信息属性, 其偏移量位于 `diOffsets` 中  
每个调试信息属性都以一个 `debugEntryType` 开头, 表明它是何种调试信息:

- 0x00 = “Unknown”
- 0x01 = “DIPCompileUnit”
- 0x02 = “DIFFile”
- 0x03 = “DILexicalBlock”
- 0x04 = “DILoc”
- 0x05 = “DISubprogram”
- 0x06 = “CallSite”

`debugEntryPayload` 描述了行、文件、变量名、函数索引、指令偏移等。每个 `debugEntryType` 都有固定的已知结构。如果 `functionLocIndex[i]` 或指令的 `locationIndex` 非零, 则引用本节中的 `debugEntryOffset[...]`, 其有效载荷可以存储文件/行信息或其他元数据。

**调试条目载荷格式** `diData` 中的每个调试条目都以一个 `debugEntryType` 字节开头, 后跟特定类型的数据:

```
debugEntry {
    debugEntryType : byte      // Identifies the debug info type
    entryData : byte[]        // Type-specific payload (format below)
}
```

The following debug entry types are supported:

**Unknown Debug Info** ( `debugEntryType` = 0x00)

```
debugEntryType : varint = 0x00 // Unknown debug info
// No additional payload
```

**DIPCompileUnit** ( `debugEntryType` = 0x01)

```
debugEntryType : varint = 0x01 // DIPCompileUnit
language : varint              // Source language identifier
fileIndex : varint             // Index of associated DIPFile
producer : varint              // String index for compiler producer
optimized : byte               // 0x00=false, 0x01=true
emissionKind : varint          // Emission kind enumeration
```

**DIFFile** ( `debugEntryType` = 0x02)

```
debugEntryType : varint = 0x02 // DIPFile
filename : varint               // String index for filename
directory : varint              // String index for directory
```

**DILexicalBlock** ( `debugEntryType` = 0x03)

```
debugEntryType : varint = 0x03 // DILexicalBlock
line : varint          // Line number
column : varint         // Column number
scopeIndex : varint    // Index of parent scope (DIPFile or DISubprogram)
)
```

**DILoc** ( `debugEntryType` = 0x04)

```
debugEntryType : varint = 0x04 // DILoc (source location)
line : varint          // Line number
column : varint         // Column number
scopeIndex : varint    // Index of scope (DISubprogram, DILexicalBlock,
    etc.)
inlinedAtIndex : varint // Index of inlined location (0 if not inlined)
```

**DISubprogram** ( debugEntryType = 0x05)

```
debugEntryType : varint = 0x05 // DISubprogram (function debug info)
name : varint // String index for function name
linkageName : varint // String index for linkage name
fileIndex : varint // Index of associated DIFile
line : varint // Line number where function is defined
typeIndex : varint // Index of function type
scopeLineIndex : varint // Line number where scope begins
flags : varint // Function flags (visibility, etc.)
unitIndex : varint // Index of associated DIBasicUnit
```

**CallSite** ( debugEntryType = 0x06)

```
debugEntryType : varint = 0x06 // CallSite location
calleeIndex : varint // Index of called location
callerIndex : varint // Index of calling location
```

### 4.3 操作码与编码

**Tile IR** 字节码中的每条指令表示为：

```
opcode : byte
locationIndex : varint
instructionSpecificFields : byte[]
```

在这种表示中，opcode 唯一标识操作。这与 **Tile IR** 方言中定义的操作之一相匹配。locationIndex 始终存在；如果指令不携带调试信息，此字段为 0。非零值指向调试节中包含文件、行号或其他源级元数据的条目。指令特定字段（操作数、属性等）遵循由操作码定义的布局。

此外，我们不为每个生产指令存储 resultIndex。相反，我们依赖顺序遍历在解析时分配局部值索引。

较旧的解析器应跳过或拒绝未知的操作码。随着时间的推移，可以通过分配新的操作码并定义其二进制负载格式来简单地添加新操作。

### 4.4 操作编码详情

操作编码的通用结构遵循一致的模式，但根据操作特性有所不同：

**General Operation Structure**

```
opcode : byte // Operation identifier
locationIndex : varint // Debug location (0 = no debug info)
resultTypes : typeIndex[]? // Present for variadic result operations
flags : varint? // Optional flags for operations with optional
    fields
attributes : encoded_attr[] // Operation-specific attributes
operands : operand_encoding // Operation-specific operand encoding
regions : region_encoding[]? // Present for operations with regions
```

**Flags Field Encoding**

对于具有可选属性或操作数的操作，使用一个标志字段：

```
flags : varint // Bitfield encoding optional presence
```

flags 字段使用单独的位来表示可选属性和操作数的存在：

- 位 0-N：可选属性（按声明顺序）
- Bits N+1-M：可选操作数（按声明顺序）

**UnitAttr** 属性仅编码在标志字段中——不写入额外数据。

**Operand Encoding Patterns**

操作数根据操作数的结构以不同方式编码：

1. **固定操作数**：写作连续的操作数索引

2. 可变操作数：以操作数数量为前缀，后跟索引
3. AttrSizedOperandSegments：每个操作数组单独编码

```
// Fixed operands (e.g., binary operations)
operand1Index : varint
operand2Index : varint
// Variadic operands (e.g., function calls)
numOperands : varint
operandIndices : varint[numOperands]
// AttrSizedOperandSegments (e.g., operations with optional operand groups)
group1 : optional_operand_group
group2 : optional_operand_group
...
```

## Result Type Encoding

- 固定结果：未编码结果类型（从操作中推断）
- 可变结果：编码的结果数量，后跟类型索引

```
// Variadic results
numResults : varint
resultTypeIndices : varint[numResults]
```

## Region Encoding

区域操作在操作数之后对它们进行编码：

```
numRegions : varint
regions : region[numRegions]
region {
    numBlocks : varint
    blocks : block[numBlocks]
}
block {
    numArgs : varint
    argTypeIndices : varint[numArgs]
    numOps : varint
    operations : operation[numOps]
}
```

## Common Operation Examples

算术运算（例如，cuda\_tile.add）

```
opcode : byte // e.g., 0x15 for add
locationIndex : varint // Debug location
lhs : varint // Left operand index
rhs : varint // Right operand index
// Result type inferred from operands
```

Memory Operations (e.g., cuda\_tile.load)

```
opcode : byte // e.g., 0x20 for load
locationIndex : varint // Debug location
resultType : varint // Type index for loaded value
address : varint // Address operand index
// Optional attributes encoded via flags
```

控制流操作（例如：cuda\_tile.if）

```
opcode : byte // e.g., 0x30 for if
locationIndex : varint // Debug location
condition : varint // Condition operand index
numRegions : varint // Always 2 for if (then, else)
thenRegion : region // Then block
elseRegion : region // Else block (may be empty)
```

## 4.5 节顺序

**Tile IR** 字节码读取器凭借其灵活的解析设计，能够处理任意顺序的节区。

读者首先发现所有部分并存储它们的有效载荷，然后按照依赖顺序处理它们。

### Default Writing Order

作者通常按以下顺序发出章节：

1. **Header** (magic number + version)
2. **Global Section** (Section ID: 0x06) - Optional, only if globals exist
3. **Function Table Section** (Section ID: 0x02) - Required
4. **Constant Data Section** (Section ID: 0x04) - Optional, only if constants exist
5. **Debug Section** (Section ID: 0x03) - Optional
6. **Type Section** (Section ID: 0x05) - Required
7. **String Section** (Section ID: 0x01) - Required
8. **End-of-Bytecode Marker** (0x00) - Required

然而，读者并不依赖于这种顺序，无论各节在文件中的排列如何，他们都能处理这些部分。这种灵活性使得未来的优化和不同的写作策略成为可能。

### Reader Implementation

The reader implements this flexibility by:

1. **发现阶段**：读取所有章节标题并将其有效载荷存储在内存中
2. **处理阶段**：按依赖顺序处理各节，与文件顺序无关
3. **惰性解析**：按需解析前向引用（例如，类型、字符串）

这种设计允许高效地随机访问任何部分，并支持未来的文件格式优化。

### Section Dependencies

- Function table references: types, strings, constants, debug info
- Global section references: types, strings, constants
- Debug section references: strings
- All sections may reference the string section

## 5 类型系统

**Tile IR** 中的所有值和操作都是静态类型的。本节定义了 **Tile IR** 的类型，以及它们的等价性、布局和其他可能与 DSL 和编译器作者相关的类型系统细节。

值得注意的是，**Tile IR** 是张量值的：所有值都是张量。我们有两种具体的张量类型：tile（纯张量值）和 view（指向内存中张量的结构化指针）。此外，我们在类型系统的表述中使用了元素类型。它们本身并不描述一个值。

### 5.1 元素类型

元素类型是 **Tile IR** 支持的原生数据类型。它们本身并不描述一个值。由于 **Tile IR** 在张量上操作，这些类型描述了张量可以包含的硬件加速的原始值。它们指定了如何解释一个位序列。每种元素类型都有一个与之关联的大小，表示表示它所需的位数。

Note

> 注意，这与潜在存储大小不同，后者由包含这些值的张量的数据布局指定。

元素类型分为两种，一种是通用基础类型，没有任何限制；另一种是专用替代类型，每种都附带一系列限制。

#### 5.1.1 基本类型

**Tile IR** 支持一组通用整数和浮点类型，这些类型被所有操作支持且没有任何限制。

这些可以包含在任意秩和形状的张量中，且此类型的 0 秩值可被视为标量。

| Type                  | Sizes            | Description                                |
|-----------------------|------------------|--------------------------------------------|
| i1, i8, i16, i32, i64 | 1, 8, 16, 32, 64 | signless integer type of specified size    |
| f16, f32, f64         | 16, 32, 64       | IEEE floating-point type of specified size |

所有算术运算都支持基本元素类型。

Warning

> 整数类型是无符号的，即该类型不编码所表示的值应被解释为有符号还是无符号值。对于在语义上需要区分有符号和无符号的操作，通过每个算术运算上的标志来控制符号性。

#### 5.1.2 备选类型

**Tile IR** 还支持一组非标准但硬件加速的浮点类型。由于这些类型和硬件的特性，每种类型都附带一系列限制。

| Type | Size | Description                                                                                                             |
|------|------|-------------------------------------------------------------------------------------------------------------------------|
| tf32 | 32   | floating-point format with 8 bits for exponent and 10 bits for mantissa. Storage size is 4 bytes with 4-bytes alignment |
| bf16 | 16   | floating-point format with 8 bits for exponent and 7 bits for mantissa                                                  |
| e3m4 | 8    | floating-point format with 3 bits for exponent and 4 bits for mantissa                                                  |
| e5m2 | 8    | floating-point format with 5 bits for exponent and 2 bits for mantissa                                                  |

这些类型的张量可以被创建、操作，并从全局内存中加载和存储，但对它们的某些计算是受限制的。

### 5.2 指针

指针，即包含内存地址的值，被类型化为指向特定被指类型的指针。

| Type   | Size | Description                                                                                                  |
|--------|------|--------------------------------------------------------------------------------------------------------------|
| ptr<E> | 64   | a pointer to a location in memory; the data at the location represents a value of non-pointer element type E |

指针指向内存中的一个位置。加载时，该位置的数据将被解释为元素类型 E。

指针算术同样假设类型 E 的存储大小用于偏移计算。指针类型通过元素类型进行参数化。

（即不支持嵌套指针类型）。关于不同指针类型之间或与整数之间的转换详情，请参阅 `cuda_tile.bitcast`。



## 5.3 张量类型

张量是一种由形状和元素类型描述的多维矩形数组。形状是一个向量，描述了张量每个轴上的元素数量。该向量的长度描述了张量的秩，即其维度数量。

**Tile IR** 中的所有张量都具有静态已知的秩。**Tile IR** 包含两种张量类型：tile 和 view。

### 5.3.1 瓦片类型

**Tile IR** 中的所有值都是 tile 类型。一个 tile 是一个具有静态形状的张量，即每个维度的范围在编译时已知。每个范围必须是 2 的幂。

我们使用 `tile<MxNxKxE>` 作为瓦片类型的语法，其中 M、N、K 是整数，E 是元素类型。更多详细信息请参阅语法。

| Example Tile Type                           | Description                                                                        |
|---------------------------------------------|------------------------------------------------------------------------------------|
| <code>tile&lt;2x8xf32&gt;</code>            | a 2-dimensional tensor with 2 rows and 8 columns of f32 values                     |
| <code>tile&lt;f32&gt;</code>                | a single value of type f32                                                         |
| <code>tile&lt;ptr&lt;f32&gt;&gt;</code>     | a pointer to a value of type f32                                                   |
| <code>tile&lt;4x8xptr&lt;f32&gt;&gt;</code> | a 2-dimensional tensor with 4 rows and 8 columns of pointers to values of type f32 |

Note

> 指针张量的形状定义了加载或存储的值的形状。它并不暗示位置本身有任何结构。例如，张量中两个连续的元素在内存中可能不是连续的位置。更进一步，同一个位置可能在一个指针张量中出现多次。有关影响的讨论，请参见内存模型。

### 5.3.2 张量视图

全局内存中的数据通常遵循跨步结构。例如，广泛采用的行优先或列优先布局就是跨步的。编译器了解内存中数据的跨步布局是有益的。**Tile IR** 具有张量视图类型，用于描述全局内存中的此类结构。

从概念上讲，张量视图类型描述了一个指针的抽象化张量。与常规张量类似，它由形状和所指向元素的类型来描述。此外，它还有一个步长因子向量。

步进因子描述了张量视图元素所指向位置的相对关系。如果一个元素在张量视图的第 i 维上相距 d 个元素，则内存中对应的位置将相距  $d * \text{stride}_i$  个元素。编译器可利用此信息推断内存中的数据访问模式和布局。

类型为张量视图的值通常永远不会在内存中实现。

相反，它们被存储为对基础位置、形状和步进因子的紧凑描述。根据这些信息，可以使用以下公式计算对应于完整视图值的指针张量

$$\text{elem}_{[i_0, \dots, i_n]} = \text{baseptr} + \sum_{m=0}^n i_m * s_m$$

其中  $s_m$  是张量视图的步长因子，而  $\text{baseptr}$  是全局内存中的起始地址。

除了瓦片类型外，张量视图还支持形状向量中的动态范围。我们使用 ? 字符来表示这些动态范围。动态范围在运行时通过 `cuda_tile.make_tensor_view` 操作构造视图类型时绑定到具体值。

一个动态形状的张量视图不能直接用于访问内存。它首先需要被分割成静态大小的块。

张量视图通过 `cuda_tile.make_tensor_view` 操作构建，并包含以下组成部分：

- `elementType`，视图中元素的类型
- `shape`，一个描述每个维度大小的 64 位整数数组
- `strides`，一个描述每个维度步长的 64 位整数数组

示例

```
!cuda_tile.tensor_view<1024x1024xf16, strides=[1024,1]>
!cuda_tile.tensor_view<32x16x32xf16, strides=[512,1,16]>
!cuda_tile.tensor_view<?x?xf16, strides=[?,1]>
!cuda_tile.tensor_view<?x16xf32, strides=[1,?]>
```

### 5.3.3 子视图类型

张量视图通常太大，无法作为单个图块加载进行处理。相反，它通常首先被细分为更小的图块。在 **Tile IR** 中，这是通过子视图类型来表达的。

子视图描述了从索引空间到从张量视图加载的静态大小图块空间的映射。它们定义了从张量视图访问元素时，由 `cuda_tile.load_view_tko` 和 `cuda_tile.store_view_tko` 执行的必要索引计算。子视图的大小由其索引空间的基数定义。

**Tile IR** 目前提供单一的子视图，用于将视图划分为非重叠的网格块，但其设计旨在未来支持更多子视图类型，以适配不同的索引模式。

- 带回网格视图。
- 将视图划分为不重叠的网格图块。
- 将视图分割成重叠图块的网格。

`partition_view` 是一种子视图类型，表示被划分为非重叠瓦片网格的视图。在这种情况下，索引空间是瓦片在网格中的位置。瓦片分区视图通过指定加载瓦片的大小来定义。与所有张量一样，瓦片大小必须是 2 的幂。

分区视图在矩阵乘法等模式中特别有用，其中全局内存中的大型张量被遍历为不重叠的图块以形成最终结果。

分区视图结构是使用 `cuda_tile.make_partition_view` 构造函数创建的。

It consists of:

- `tileShape`，视图中单个图块的形状
- `view`，构建子视图的视图类型
- `dimMap`，一个 32 位整数数组，用于将图块的维度映射到视图的维度。
- `masked`，一个定义视图边界外内存操作的标志

These fields are all encoded as part of the type.

具体来说，一个张量视图类型 `tensor_view<8192x128xf32, strides=[128,1]>` 可以被划分为一个 (64x32) 的网格，每个网格块的大小为 (128x4)，加载时会产生类型为 `tile<128x4xf32>` 的块。从这样的分区视图中加载一个块时，索引是 (64, 32) 网格中的位置。因此，加载元素 (4, 2) 将返回基础视图中起始位置为 (256, 64) 的块。

形式上，给定一个形状为  $[S_0, \dots, S_n]$  和步幅为  $[st_0, \dots, st_n]$  的张量视图，以及一个分块大小为  $[T_0, \dots, T_n]$  的分块视图，在位置  $[I_0, \dots, I_n]$  的加载或存储操作将加载位于以下位置的元素：

$$location[i_0, \dots, i_n] = baseptr + \sum_{m=0}^n I_m \cdot \text{ceildiv}(S_m, T_m) \cdot st_m$$

#### Warning

> 当底层视图的形状无法被瓦片大小整除时，边界上的瓦片可能是部分的。对部分瓦片进行读取或写入是未定义行为。通过为分区视图指定 `masked` 限定符，边界可以选择性地被屏蔽。在这种情况下，越界读取将返回填充值，相应的写入操作将被跳过。

#### 示例

```
!cuda_tile.partition_view<
  tile=(2),
  view=!cuda_tile.tensor_view<16xf32, strides=[1]>
>
!cuda_tile.partition_view<
  tile=(2x2),
  view=!cuda_tile.tensor_view<16x16xf32, strides=[16,1]>
>
!cuda_tile.partition_view<
  tile=(2x2),
  view=!cuda_tile.tensor_view<16x16xf32, strides=[16,1]>,
  dim_map=[1, 0]
```

```
>  
!cuda_tile.partition_view<  
  masked tile=(2x2),  
  view=!cuda_tile.tensor_view<16x16xf32, strides=[16,1]>,  
>
```

## 5.4 类型等价

**Tile IR** 不提供命名类型的方式。因此，类型的等价性纯粹是一种结构属性：如果两个类型在结构上相同，则它们被视为相等。

请注意，某些类型会形成自然的子类型关系。具有动态形状和步幅的类型（如视图）覆盖了将动态形状和步幅全部替换为静态值的相同类型所覆盖的值。然而，在 **Tile IR** 中，我们将这些类型视为不同的类型。

## 6 语义

本节以书面英语形式呈现 **Tile IR** 的操作语义。这些语义旨在帮助以下两类人员理解 **Tile IR**：1) 有意将 **Tile IR** 作为代码生成目标的人；2) 有意阅读他人编写的 **Tile IR** 程序的人。

它并不试图形式化所有可能被公理化表述或小步操作语义所允许的行为。要了解关于该语言及其核心概念更为非正式的介绍，请参阅编程模型部分。

我们首先介绍抽象机器状态和语言定义，然后描述各个内核和程序的语义。

然后我们也会讨论广泛操作类别的语义，至于各个操作及其行为的更详细描述，请参阅操作以获取完整列表。

### 6.1 抽象机器

**Tile IR** 抽象机状态  $S$  是一个由以下组件组成的元组，每个组件的解释如下：

- 一个格式良好的模块  $\mathcal{M}[\cdot]$ ，它存储一个或多个项目，将在下面的模块部分进行讨论。
- 一个瓦片块  $\mathcal{TB}$ （或逻辑瓦片线程）的网格，每个代表一个单独的瓦片内核实例。
- 一个每瓦片块无限寄存器文件 (per-tile-block infinite register file)  $R$ ，它将命名的“寄存器”映射到值。
- 一个全局内存存储， $M$ ，将地址映射到标量值。
- 一组待处理的内存访问操作  $P$ ，它们相对于 **Tile IR** 操作的执行是异步进行的。

### 6.2 模块

**Tile IR** 中的程序以模块的形式表示。模块是一个单独的翻译单元，包含零个或多个项。一个项可以是以下之一：

- 一个全局变量定义
- 一个瓦片内核
- 一个瓦片函数

对于那些熟悉 CUDA 的人来说，tile kernel 是 tile 程序的全局入口点，就像在 CUDA C++ 或 PTX 中 kernel 的作用一样。

Tile 函数代表可从当前 tile 内核调用的设备端函数，但存在一些限制。

#### 6.2.1 全局变量

全局变量是一个存储在全局设备内存中的命名全局变量，所有线程块均可访问。

全局变量使用 `cuda_tile.global` 操作进行声明。

全局变量必须在声明时初始化，并且只会被初始化一次。

全局变量必须包含一个 Tile 类型的值。

可以使用 `cuda_tile.get_global` 操作来获取一个指针，通过该指针可以读写全局变量，从而修改全局变量。

#### 6.2.2 Tile Kernel

```
entry @tile_func(%A0: T0, ..., %AN: TN) {
    %0 = op %P0, %P1, ... %PN -> R0
    ...
    return
}
```

**Tile IR** 中的基本执行单元是瓦片内核。瓦片内核是一个瓦片函数，它充当瓦片程序的入口点。瓦片内核表示一个由一组网格坐标参数化的函数。

在内核运行时，每个唯一的网格坐标对每个内核实例（瓦片块）都是可用的。

一个瓦片内核可以通过 `cuda_tile.get_tile_block_id` 查询其网格坐标，坐标可以是一维、二维或三维，具体取决于启动内核时使用的网格。内核也可以通过 `cuda_tile.get_num_tile_blocks` 查询每个维度上的瓦片块总数。

一个瓦片内核是一个带有额外限制的瓦片函数：

- 只能具有标量（即 0 秩）张量类型的参数
- 要求所有输入张量都以标量指针的形式提供（即 `tile<ptr<E>>`）
- 不产生返回值
- 内核的执行仅针对其对全局设备内存的影响

瓦片内核本质上是一个瓦片函数，瓦片函数的所有属性同样适用于瓦片内核。

### 6.2.3 Tile 函数

一个瓦片函数由名称、形式参数列表、返回类型和函数体组成。

瓦片函数的主体包含一个单线程的瓦片程序（称为瓦片块），该程序由形式参数参数化。

一个瓦片函数有  $N$  个形式参数并产生  $M$  个返回值。参数的类型可以是类型系统中描述的有效类型之一。

Note

> 目前定义非内核瓦片函数的功能已被禁用，计划在未来的版本中提供支持。

### 6.2.4 函数体

函数体由一系列静态单赋值（SSA）形式的语句组成。

每个语句将单个操作的结果分配给一组唯一的结果变量。

**Tile IR** 中的所有操作都以这种方式统一表示，包括控制流和内存操作。

### 6.2.5 Well-Formedness

一个格式良好的模块是满足以下属性的模块：

- 模块至少包含一个项 (item)。
- 每个项在模块中具有唯一的名称。
- 每个瓦片内核和瓦片函数的主体都是一个由有效静态单赋值（SSA）形式的语句序列构成。
- 程序根据类型系统中指定的规则进行类型检查，且运算符类型签名在操作中指定。

程序良构性被要求作为优化和操作语义规则的前置条件与后置条件。

## 6.3 值

**Tile IR** 拥有一小组类型，如类型系统中所述，但我们只有两种类型的值。

- Pointers，表示一个内存地址。
- Tiles，或标量的  $N$  维数组。
- Views，表示内存的结构化视图。

### 6.3.1 指针

指针是一个 64 位整数内存地址，用于引用全局设备内存中的位置。

指针被类型化为 `ptr<E>`，其中  $E$  是它所引用的内存位置类型。

指针必须对齐到它们所指向的基础数据类型的大小，具体编码和分配布局信息请参见元素类型编码。

指针是指向全局内存中某个位置的内存地址。

**数据布局** 输入指针值所指向的分配，以及由此扩展的视图（参见下文），必须符合指定的数据布局。

我们期望由 `ptr<E>` 指向的分配是一个具有元素类型  $E$  的标量值的大小已知的连续分配。

分配的元素之间没有填充，我们预计大小为  $N$  的分配将等同于  $N * \text{sizeof}(E)$  字节。元素的大小和编码由其类型  $E$  决定，并在下表中定义。

顺便提一下，数据类型编码与 DLPack 兼容，这是大多数深度学习框架和数组库采用的标准。我们为每种数据类型提供了等效 PyTorch 和 NumPy 编码。

对于 NumPy 低精度类型，我们通过 `ml_dtypes` 库提供了等效的标准低精度 NumPy 数据类型集合。

Warning

> **Tile IR** 布局目前限制为子字节类型必须是连续的。

| Tile IR Type | DLPack Type Code | DLPack Bits | DLPack Lanes | NumPy Type                 | PyTorch Type |
|--------------|------------------|-------------|--------------|----------------------------|--------------|
| i1           | kDLInt , kDLUInt | 8           | 1            | numpy.uint8 ( unpacked )   | N/A          |
| i8           | kDLInt , kDLUInt | 8           | 1            | numpy.uint8 , numpy.int8   | torch.bool   |
| i16          | kDLInt , kDLUInt | 16          | 1            | numpy.int16 , numpy.uint16 | torch.int16  |
| i32          | kDLInt , kDLUInt | 32          | 1            | numpy.int32 , numpy.uint32 | torch.int32  |
| i64          | kDLInt , kDLUInt | 64          | 1            | numpy.int64 , numpy.uint64 | torch.int64  |
| f16          | kDLFloat         | 16          | 1            | numpy.float16              | torch.float  |
| f32          | kDLFloat         | 32          | 1            | numpy.float32              | torch.float  |
| f64          | kDLFloat         | 64          | 1            | numpy.float64              | torch.float  |
| bf16         | kDLBfloat        | 16          | 1            | ml_dtypes.bfloat16         | torch.bfloat |
| fp8 (E3M4)   | kDLFloat8_e3m4   | 8           | 1            | ml_dtypes.float8_e4m3fn    | torch.float  |
| fp8 (E5M2)   | kDLFloat8_e5m2   | 8           | 1            | ml_dtypes.float8_e5m2      | torch.float  |

Note

> Allocations 是唯一在 **Tile IR** 中指定内存布局的值。

### 6.3.2 Tiles

一个瓦片是一个不可变的 N 维标量数组，其特征为：

- 它的秩（维度数量）
- 它的形状（每个维度的长度）
- 它的基本元素类型

这些属性既是瓦片类型的一部分，也是值的大小和形状的一部分。

一个瓦片可以具有任意非负秩，其中：

- Rank-0 tiles 表示标量值。
- Rank-1 tiles 表示向量。
- Rank-2 tiles 表示矩阵。
- 而 Rank-N 图块表示高阶 N 维数组。

对于给定的 **tile**<N<sub>x</sub>K<sub>x</sub>E> 类型图块，其形状为 (N, K)，元素类型为 E，结果值的大小为  $N \times K \times \text{size}(E)$ ，包含  $N * K$  个独立元素。

这种类型的瓦片值被抽象地表示为一个元组，该元组包含一个具有  $N * K$  个类型为 E 的元素的数组，以及一个不透明的布局，该布局提供了从元素索引空间到线性索引的映射。

Note

> 给定图块的物理布局和内存表示对程序不可见，且不由语言语义规定。

>

> 编译器将选择以对目标架构、特定程序和瓦片大小最高效的方式在内存中表示瓦片。

### 6.3.3 Views

**Tile IR** 提供了一组视图类型，如 Tensor View 中所述。

视图通过为指针添加额外数据，提供了对内存的结构化视图。

视图拥有自己的一组内存操作 `cuda_tile.load_view_tko` 和 `cuda_tile.store_view_tko`，这些操作的行为根据视图类型的不同而变化，并利用了额外的元数据。由于视图并非单一类型，而是一个类型家族，每个具体的视图类型都有其自身的值表示形式。

我们的一阶视图类型是 Tensor View。张量视图值在逻辑上是一个元组  $(ptr, \vec{shape}, \vec{strides}, \vec{dimgroups})$ ，加载和存储操作作用于整个张量视图。该视图的索引空间是零秩的（即索引空间中只有一个元素）。

我们的主要二阶（或“子视图”）视图类型是分区视图。

分区视图类型 分区视图 是一种子视图类型，它表示被分割成图块的张量视图。

分区视图值在逻辑上是一个元组  $(tensor\_view, \vec{tile\_size})$ ，其中加载和存储操作在给定大小的图块值上进行。此视图的索引空间为：

$$indexSpace[i] = tensorViewShape[dimGroups[i]]/tileSize[i]$$

例如，一个形状为 (1024, 1024) 的张量视图可以被划分为一个  $64 \times 32$  的瓦片网格，从而得到一个形状为 (16, 32) 的分区视图。有关不同类型视图的更多详细信息，请参阅类型系统。

Note

> 与瓦片一样，视图的物理布局和内存表示对程序不可见，且不由语言语义规定。

Warning

> 读取或写入任何分配的内存边界是未定义行为，如果需要进行边界检查，必须由程序员自行执行。

## 6.4 瓦片网格

在执行过程中，抽象机实例化了一个瓦片块网格。瓦片网格是由瓦片块按一维、二维或三维数组排列而成的网格。

网格的每个位置对应一个独立的瓦片内核实例。

抽象机存储了一系列由坐标网格索引的图块块，记为  $TB$ ，索引方式为  $\vec{g}$ 。

每个瓦片块根据网格大小及其在网格中的位置被分配一个唯一的瓦片块 id。

1 维、2 维或 3 维网格坐标与扁平化的瓦片块 ID 之间的双射映射计算如下：

$$tile\_grid(\vec{i}) = \begin{cases} i_0 & \text{where grid} = (x) \\ i_0 \cdot x + i_1 & \text{where grid} = (x, y) \\ i_0 \cdot x + i_1 \cdot y + i_2 & \text{where grid} = (x, y, z) \end{cases}$$

where  $\vec{i} = (i_0, \dots, i_n)$  and  $0 \leq i_k < grid_k$  for all  $k \leq 3$

## 6.5 Register File

寄存器文件  $R$  将命名寄存器映射到值。图块函数体中的每个赋值（即 SSA 变量）都被分配到一个唯一的寄存器，该寄存器最终保存操作结果的值。

寄存器是瓦片块（tile block）局部的，对其他瓦片块不可见。如前所述，寄存器中值的内存表示对程序不可见，且不由语言语义规定。

值仅通过内存操作从全局内存中获取或持久化到全局内存中（参见内存）。

寄存器文件通过瓦片块的坐标  $\vec{g}$  和寄存器的名称  $r$  进行索引，并产生一个值  $v$ 。

$$S.R(\vec{g}, r) = v$$

## 6.6 全局内存

全局内存， $M$ ，是一个从地址到标量值的映射。

全局内存用于存储瓦片块全局变量的值。

堆被抽象地建模为从地址到标量值的映射，而不是图块值。

这种区分对于将瓦片操作的内存效应描述为一系列独立的标量内存操作至关重要。细粒度模型既能支持对聚合操作进行精细推理，也能直接映射到现有的 PTX 内存模型中。

Note

> 全局内存是与 CUDA 程序使用的相同全局设备内存，并在 PTX 规范中有所描述。

在内存模型中详细阐述了 **Tile IR** 内存模型的复杂特性。

## 6.7 瓦片块

瓦片块是执行的一个单一线程，在瓦片网格中被分配了一个唯一的坐标。

抽象而言，其状态包含：

- 正在执行的瓦片内核。
- 一个寄存器文件， $R$ ，它将命名寄存器映射到值。
- 一个正在求值的语句，由整数索引表示，该索引指向瓦片函数体中 SSA 语句序列中的位置。

## 6.8 执行语义

**Tile IR** 程序的执行始于内核启动。内核启动 API 对所有 CUDA 内核都是统一的，并在 CUDA 运行时 API 中规定。

### 6.8.1 初始化

一次瓦片内核的启动会初始化抽象机，其内容为：

- 模块，表示完整的程序。
- 一个由瓦片块组成的网格，其中每个瓦片块使用相同的瓦片内核实例化，从语句 0 开始，分配一个唯一的网格坐标，并分配一个唯一的空寄存器文件。
- 对全局内存的引用，其状态为内核启动前全局内存的状态。
- 待处理内存操作集合初始化为空。

### 6.8.2 前向进展

执行过程以未指定的瓦片块调度进行。每个瓦片块将以某种顺序执行，这种顺序是非确定性的，且未由语言语义指定。我们保证执行的前向进展，即所有瓦片块最终都会被调度执行。所有瓦片块可能完全并行运行、完全串行运行，或介于两者之间的任何方式。

### 6.8.3 图块块执行

单个瓦片块的执行与其他瓦片块是隔离的。瓦片块只能通过全局内存观察其他块的效果，这可用于实现某种形式的协作或通信。

函数体是一系列静态单赋值（SSA）语句，每个语句将操作结果赋值给一个唯一的变量。每个变量映射到抽象机器寄存器文件中的一个寄存器。函数体按顺序依次执行这些语句。

编译器可以自由地重新排序语句，只要不影响程序的可见效果或违反程序语义。

有关更详细的示例程序及其执行说明，请参阅编程模型或附录。

**Tile IR** 的一个独特语义是将内存操作划分为程序顺序操作和令牌顺序操作。所有内存操作都会立即产生它们的结果值，但这些操作影响内存的顺序则更为微妙。

对于程序顺序操作，作用于同一地址的任意一对内存操作之间的顺序由操作在程序中的位置定义。直观地说，所有先前对同一地址的内存操作的效果对于所有后续对同一地址的内存操作都是可见的。

相反，任意一对令牌排序操作之间的顺序是未定义的，并且与程序顺序无关。任意两个令牌排序操作之间的顺序

（*A* 和 *B*）仅当在 *A* 的输出令牌与 *B* 的输入令牌之间建立了直接或传递关系时才被定义。

这一选择的重要性在于，它允许 **Tile IR** 的生成者通过插入适当的内存排序标记来引入不同的内存排序语义。

例如，从一个单独的新令牌开始，根据第一个操作，第一个操作的结果令牌被第二个操作所依赖，依此类推，按照程序顺序将令牌贯穿每个操作。

这样的令牌线程建立了一种内存操作的顺序，该顺序与同一程序一致，其中每个令牌有序操作被程序有序内存操作所替代。

关于内存模型和内存操作的详细讨论，请参见内存模型。

### 6.8.4 终止

当瓦片块的函数体到达最终语句时，瓦片块将终止。瓦片内核必须通过返回操作 `cuda_tile.return` 来终止，该操作标志着执行结束。



## 7 内存模型

内存模型定义了加载操作可以从内存中返回的合法值。

这并不像乍看之下那么简单，为了启用编译器和硬件优化，我们允许指令的明显重排序。

该内存模型源自 PTX 内存模型，且同步原语被刻意设计为相似。

### 7.1 内存模型操作

内存模型由瓦片操作中各个元素访问之间的关系以及这些关系循环的限制构成。

因此，一条 **Tile IR** 内存指令会生成一个或多个内存模型操作。

特别是，图块加载、存储和原子更新会为图块中的每个元素生成一个内存操作。

当将一个瓦片操作扩展为多个内存模型操作时，人们可能会问内存模型操作以什么顺序发生。

这是为了保持实现灵活性而故意未作规定的。

为了在 **Tile IR** 操作上泛化内存模型，我们在内存一致性模型的规范中始终使用内存模型操作这一术语。

我们在下表中列举了哪些 **Tile IR** 操作会扩展为哪些内存模型操作。

### 7.2 作用域

**Tile IR** 中的内存操作可能具有作用域。

没有作用域的操作被称为 **weak**。

所有内存操作都必须指定一个作用域或 **weak**。

除弱作用域外的任何作用域都需要设置内存排序。

| Scope             | Description                                                     |
|-------------------|-----------------------------------------------------------------|
| <b>tile_block</b> | Tile block scope, for communication within a single tile block. |
| <b>device</b>     | Device scope, for communication within the same GPU.            |
| <b>sys</b>        | System scope, for communication anywhere in the system.         |

弱操作不能用于在线程之间，或同一瓦片块中未被令牌顺序排序的片段之间通过内存进行通信。

编译器可以假定使用 **weak** 访问的图块不会被任何其他线程同时访问。

Note

> 在构建需要单个瓦片块内部进行通信的通信算法时，需要瓦片块作用域。

> 当通过内存存在不按令牌顺序排序的操作之间进行通信时，或在存储到具有内部别名的图块时，这是必要的。

### 7.3 内存排序

内存操作具有一个内存排序参数，该参数控制该操作如何用于同步。

通过内存进行同步是一个双方参与的过程，它要求释放者和获取者观察同一个位置。

当一对内存访问通过内存同步时，它建立了一种 *happens before* 关系。

下面是 happens before 的定义。

除 **weak** 外的任何排序都需要设置作用域。

| 内存排序           | 描述                                                               |
|----------------|------------------------------------------------------------------|
| <b>weak</b>    | 无并发访问源/目标位置。                                                     |
| <b>relaxed</b> | 可能存在对位置的并发访问，但此访问不建立 happens-before 关系。                          |
| <b>release</b> | 可能存在对位置的并发访问。若此 release 操作被 acquire 操作观察到，则建立 happens-before 关系。 |
| <b>acquire</b> | 可能存在对位置的并发访问。若此 acquire 操作观察到 release 操作，则建立 happens-before 关系。  |
| <b>acq_rel</b> | 可能存在对位置的并发访问。同时具有 release 和 acquire 操作的效果。                       |

### 7.4 Moral Strength

对同一位置的两次访问是 morally strong，如果操作在 *restricted program order* 中相关，或者每个操作指定的作用域包含执行另一个操作的瓦片块。

## 7.5 令牌与令牌顺序

在 **Tile IR** 中，我们明确标注了令牌有序操作的加载和存储之间的依赖关系。**Tile IR** 生成对整个数据块的宽加载和存储，从而并行高效地利用 GPU 的各种资源。

我们提供令牌有序操作，以明确告知 **Tile IR** 工具链两个操作可以并行发生，且不会相互干扰。

有一类称为令牌有序操作的内存操作，它们产生和消费令牌。

Tokens 是 **Tile IR** 语言中的抽象值，用于在同一瓦片块线程内的内存操作之间建立依赖关系。

它们在运行时没有具体表示，无法被比较、计算，也不能存储到内存中或从内存中加载。

程序依赖（即从控制流、数据依赖或地址依赖中显现的依赖）并不在两个内存操作之间提供顺序保证。

即使令牌排序看起来与程序依赖关系冗余，也必须使用令牌。

程序依赖可能被 **Tile IR** 工具链优化掉，而令牌依赖则不会。

## 7.6 基础关系

### Coherence order

存在一种部分传递顺序，它在运行时确定，用于关联重叠的写操作，称为一致性顺序。

两个重叠的写操作如果具有道德强序，或者它们在发生先于顺序中相关，则它们在一致性顺序中相关。

### Program order

内存操作  $A$  在程序顺序上先于内存操作  $B$ ，当且仅当在程序源代码中产生  $A$  的指令位于产生  $B$  的指令之前。

### Waits-for order

一个操作  $A$  *waits-for* 一个操作  $B$  如果：

- 指令  $I_A$  引发了  $A$ ， $I_A$  生成了一个 token  $t$ ，指令  $I_B$  引发了  $B$ ，且  $I_B$  依赖于 token  $t$ ；或
- 存在某个操作  $C$ ，使得  $A$  *waits-for*  $C$  且  $C$  *waits-for*  $B$ 。

### Reads from

一个读操作  $R$  从一个写操作  $W$  读取，当  $R$  和  $W$  访问相同的位置且  $R$  读取了由  $W$  写入的值。

### Read-modify-write order

读取-修改-写入原子操作会为图块内的每个元素位置生成一对内存操作。

每对内存操作通过读-改-写顺序相互关联。

### Atomicity

当一个原子操作  $A$  和一个写操作  $W$  重叠且 *morally strong* 的时，那么以下两种通信不可能同时存在于同一个执行中：

- $A$  reads from a write  $W'$  that precedes  $W$  in *coherence order*.
- $A$  follows  $W$  in *coherence order*.

## 7.7 No-thin-air

在不阻止有用的编译器优化的情况下，指定“无凭空创造”公理是不切实际的。

因此我们非正式地说，实现不会凭空提供值来满足程序执行。

## 7.8 数据竞争

两次访问被称为冲突，当它们访问同一位置且至少其中一次是写操作。

两个冲突的内存访问被称为处于数据竞争状态，如果它们在发生顺序上没有关联，并且在道德上也不强。

具有数据竞争的程序具有未定义行为。

## 7.9 PTX 互操作性

**Tile IR** 内存模型的公理和关系旨在成为 PTX 内存模型的严格弱化版本。

**Tile IR** 内存模型旨在实现与 PTX 线程的通信。

**Tile IR** 中的释放与获取模式将与 PTX 中的获取与释放模式相匹配，以构建 PTX 的因果性和 **Tile IR** 的先于发生关系。

对于数据竞争也是如此，**Tile IR** 程序中的访问与 PTX 之间的数据竞争也是如此。

程序将导致未定义行为，就像数据竞争全部发生在 **Tile IR** 中一样。

## 7.10 Tile IR 内存模型中的风险与直觉

### 7.10.1 从未来读取的令牌顺序操作

在不使用令牌约束的令牌有序操作中，存储缓冲和加载缓冲行为是可见的。

### 7.10.2 单个瓦片块线程内的竞争风险

数据竞争的定义依赖于两个操作处于 happens before 顺序中。

在许多编程语言中，这在单个线程内总是成立的，但在 **Tile IR** 中并非如此。

在 **Tile IR** 中，当对单个图块块存储中的同一位置进行两次访问时（由于目标图块内部重叠），或者在图块块线程内对同一位置进行两次未按令牌顺序排序的访问时，可能会引发数据竞争。

这促使了需要一个 `tile_block` 作用域，并且是该语言中需要注意的一个隐患。

### 7.10.3 关于如何使用令牌排序的一些直观理解

当你对彼此之间没有重叠的图块进行内存访问时，通常可以安全地不在令牌顺序中相互排序这些访问。

当你使用释放操作时，需要将所有必须保持在释放之前的内存事件按令牌顺序排列到释放本身。

类似地，当使用获取操作时，所有必须在获取操作之后保留的内存事件都需要在获取操作本身之后进行令牌排序。

重申：**Tile IR** 的用户不能依赖除令牌依赖之外的任何形式的程序依赖来强制 Tile Block Thread 内的顺序。

**Tile IR** 工具链能够通过任何可能的复杂推理移除依赖，打破那些在程序语法中看似存在的依赖关系。

## 8 运算操作

本节描述了所有 **Tile IR** 指令名称、签名和语义的完整分类列表。

### 8.1 元类型

操作具有参数，这些参数是带有 **Tile IR** 类型的 **Tile IR** 值，但许多操作具有立即或静态参数，这些参数对应于 MLIR 方言中的属性。这些 **元类型** 在 **Tile IR** 类型系统中无法表示，但用于构建 **Tile IR** 程序，并且仅在编译时存在。规范中的操作在 **Tile IR** IR 和字节码中独立于 MLIR 或字节码编码进行抽象描述。对于这些类型中的每一种，我们在下面提供了它们的定义，并从每个操作链接到它们。

Note

> 惯例是元类型使用大写，而 **Tile IR** 类型使用蛇形命名法。

惯例是元类型使用大写，而原生的 **Tile IR** 类型采用驼峰命名法或蛇形命名法。

#### 8.1.1 符号

Symbol 程序中的一个符号，以 @ 开头，用于唯一标识程序中的符号。

#### 8.1.2 标志

Flag 一个布尔值，可用于控制操作的行为。

#### 8.1.3 令牌

Token 表示一个内存排序令牌，可用于控制内存操作的顺序。

#### 8.1.4 可变参数

Variadic 表示一个可以接受静态大小但数量可变的参数的参数。

#### 8.1.5 任意

Any 表示任何有效的 **Tile IR** 类型的值。

#### 8.1.6 名称

Name 表示程序中的一个名称，以 # 开头，并在程序中唯一标识一个名称。

#### 8.1.7 类型

Type 表示一个 **Tile IR** 类型，并作为属性附加到定义 IR 条目的操作上。

#### 8.1.8 数组

Array 表示一个静态大小的值数组，可以传递给属性。

#### 8.1.9 字符串

String 表示可以传递给属性的字符串值。

#### 8.1.10 bool

bool 表示可以传递给属性的布尔值。

#### 8.1.11 DenseConstant

DenseConstant 表示一个可以传递给属性的密集常量值。

#### 8.1.12 view\_type

view\_type 代表一种实现了视图接口的类型，目前仅由 partition\_view 实现，但在未来的版本中会有新的实现者。

## 8.2 操作设计注意事项

**Tile IR** 的设计包含一系列适用于该方言中所有操作的设计考量，本节将介绍一些普遍适用于所有操作或通用操作类别的常见设计考量。

### 8.2.1 显式广播

在 **Tile IR** 方言中，操作不会执行隐式广播，所有需要相同形状操作数的操作都必须显式进行广播。

例如，要使用 `cuda_tile.offset` 操作向指针添加偏移瓦片，必须使用 `cuda_tile.reshape` 或 `cuda_tile.broadcast` 操作对指针和偏移进行重塑或广播，使其具有相同的形状。

### 8.2.2 独立的浮点与整数运算

数值运算根据舍入模式、NaN 处理和快速数学等标志的差异，被分为整数和浮点类型。

例如，`cuda_tile.addf` 操作支持舍入属性，但 `addi` 操作不支持。

### 8.2.3 显式溢出标注

一些操作，例如 `cuda_tile.addi` 支持显式的溢出注解，用以表达该操作预期的溢出行为。

这些属性作为实现可用于推理操作的假设。代码生成器的责任是确保操作在执行过程中动态地遵循这些假设。

我们建议 **Tile IR** 程序的生成器利用这些注解来帮助实现推理操作的溢出行为，从而开启额外的优化机会。

## 8.3 Core

### 8.3.1 `cuda_tile.broadcast`

*Broadcast tile to new shape*

```
cuda_tile.broadcast %source
```

#### 参数

- `source` ( `tile` ) - 要广播的 `tile`。 13.1
- `result` ( `tile` ) - 广播后的 `tile`。 13.1

`broadcast` 操作通过沿该维度复制数据来扩展输入瓦片中的每个一元（1）维度。

扩展仅针对大小为 1 的维度进行，这些维度会被拉伸或“复制”以匹配操作结果类型所隐含的维度大小。该操作不会改变源图块的秩。任何对源图块秩的更改必须在广播之前通过类似重塑的操作来完成。

#### 约束条件

- 该操作基于特定操作数和属性有条件地可推测。
- 该操作可能会被推测性地执行，且没有副作用。
- 该操作是纯粹的，不执行任何内存副作用。
- `source` 和 `result` 必须具有相同的元素类型（`tile`）。
- `source` and `result` must have the same rank.

### 8.3.2 `cuda_tile.cat`

*Concatenate tiles along specified dimension*

```
cuda_tile.cat %lhs %rhs %dim
```

## 参数

- **lhs** ( tile ) - 左侧操作数。 [13.1]
- **rhs** ( tile ) - 右侧操作数。 [13.1]
- **dim** ( i64 ) - 沿此维度进行拼接。 [13.1]
- **result** ( tile ) - 拼接后的结果图块。 [13.1]

**cat** 操作将两个输入张量进行拼接。输入张量在除拼接维度外的所有维度上必须具有相同的形状。拼接沿着由属性 **dim** 指定的维度进行，结果张量在该维度上的尺寸为两个输入张量拼接维度尺寸之和。

$$\text{cat}(x, y, \text{dim}_{\text{cat}})[\vec{i}] = \begin{cases} x[\dots, i_{\text{cat}}, \dots, i_n] & \text{if } i_{\text{cat}} < d_{\text{cat}} \\ y[\dots, i_{\text{cat}} - d_{\text{cat}}, \dots, i_n] & \text{if } i_{\text{cat}} \geq d_{\text{cat}} \end{cases}$$

## 约束条件

- 该操作是否可推测取决于特定的操作数和属性。
- 该操作可以无副作用地推测执行。
- 该操作是纯粹的，不会产生任何内存副作用。
- **lhs**、**rhs** 和 **result** 必须具有相同的秩。
- **lhs**、**rhs** 和 **result** 必须具有相同的元素类型 (tile)。

### 示例

```
// A valid invocation of cat.
%0 = cat %arg0, %arg1 dim = 1
: tile<2x4xf32>, tile<2x4xf32> -> tile<2x8xf32>
// >>> %arg0 = tile([[ A, B, C ],
//                  [ D, E, F ]])
// >>> %arg1 = tile([[ 1, 2, 3 ],
//                  [ 4, 5, 6 ]])
// >>> %0 = tile([[ A, B, C, 1, 2, 3 ],
//               [ D, E, F, 4, 5, 6 ]])
// A valid invocation of cat.
%1 = cat %arg0, %arg1 dim = 0
: tile<2x4xf32>, tile<2x4xf32> -> tile<4x4xf32>
// >>> %arg0 = tile([[ A, B, C ],
//                  [ D, E, F ]])
// >>> %arg1 = tile([[ 1, 2, 3 ],
//                  [ 4, 5, 6 ]])
// >>> %1 = tile([[ A, B, C ],
//               [ D, E, F ],
//               [ 1, 2, 3 ],
//               [ 4, 5, 6 ]])
```

完整示例列表请参见 `cuda_tile.cat_0`。

### 8.3.3 cuda\_tile.cmpf

*Element-wise floating-point comparison*

```
cuda_tile.cmpf %comparison_predicate %comparison_ordering %lhs %rhs
```

## 参数

- **comparison\_predicate** ( ComparisonPredicate ) - 比较谓词。 13.1
- **comparison\_ordering** ( ComparisonOrdering ) - 比较顺序。 13.1
- **lhs** (tile<f16|bf16|f32|f64>) - 左侧操作数。 13.1
- **rhs** ( tile<f16|bf16|f32|f64>) - 右侧操作数。 \boxed{13.1}
- **result** ( tile<i1>) - 比较的结果。 13.1

**cmpf** 操作是对浮点类类型的通用比较。操作数必须具有相同的形状和类型，且该类型必须是浮点类型。

如果比较为真，则结果为 1，否则为 0。比较是按元素进行的，结果的每个元素表示操作数中与结果索引相同的元素是否满足比较条件。

`cmpf(x,y,pred)i={1if xi pred yi0otherwise`  
`comparison_predicate` 属性指定要执行的比较类型。

- **equal** - Equal comparison.
- **not\_equal** - Not equal comparison.
- **less\_than** - Less than comparison.
- **less\_than\_or\_equal** - 小于或等于比较。
- **greater\_than** - Greater than comparison.
- **greater\_than\_or\_equal** - 大于或等于比较。

`comparison_ordering` 属性指定了在比较操作中要执行的排序类型。

- **unordered** - Unordered comparison.
- **ordered** - Ordered comparison.

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可能会被推测性地执行，且不产生副作用。
- 该操作是纯粹的，不会产生任何内存副作用。
- **lhs** 与 **rhs** 必须具有相同的形状和元素类型 (tile<f16|bf16|f32|f64 > )。
- 结果类型具有 **i1** 元素类型，且与操作数形状相同
- 操作的结果类型可以从其操作数和属性中推断出来。

### 示例

```
%lhs0 = constant <f16: 0.0> : tile<f16>
%rhs0 = constant <f16: 0.0> : tile<f16>
// Custom form of scalar "ordered equal" comparison.
%x0 = cmpf equal ordered %lhs0, %rhs0 : tile<f16> -> tile<i1>
%lhs1 = constant <f16: 0.0> : tile<2x2xf16>
%rhs1 = constant <f16: 0.0> : tile<2x2xf16>
// Custom form of scalar "unordered less than" comparison.
%x2 = cmpf less_than unordered %lhs1, %rhs1 : tile<2x2xf16> -> tile<2x2xi1>
%lhs2 = constant <f64: 0.0> : tile<2x2xf64>
%rhs2 = constant <f64: 0.0> : tile<2x2xf64>
```

请参阅 `cuda_tile.cmpf_0` 获取完整示例列表。

### 8.3.4 cuda\_tile.cmpi

*Element-wise integer comparison*

```
cuda_tile.cmpi %comparison_predicate %lhs %rhs %signedness
```

#### 参数

- **comparison\_predicate** ( ComparisonPredicate ) - 比较谓词。 13.1
- **lhs** ( tile<i1|i8|i16|i32|i64> ) - 左侧操作数。 13.1
- **rhs** ( tile<i1|i8|i16|i32|i64> ) - 右侧操作数。 13.1
- **有符号性** ( 有符号性 ) - 将整数解释为 有符号或 无符号 13.1
- **result** ( tile<i1> ) - 比较的结果。 13.1

**cmpi** 操作是对整数类类型的通用比较。操作数必须具有相同的形状和类型，且该类型必须是整数类型。结果类型具有 i1 元素类型，且形状与操作数相同。

如果比较为真，结果为 1，否则为 0。比较是按元素进行的，结果的每个元素表示操作数中与结果索引相同的元素是否满足比较条件。

$$\text{cmpi}(x,y,\text{pred})i = \begin{cases} 1 & \text{if } x_i \text{ pred } y_i \\ 0 & \text{otherwise} \end{cases}$$
  
**comparison\_predicate** 属性指定要执行的比较类型。

- **equal** - Equal comparison.
- **not\_equal** - Not equal comparison.
- **less\_than** - Less than comparison.
- **less\_than\_or\_equal** - 小于或等于比较。
- **greater\_than** - Greater than comparison.
- **greater\_than\_or\_equal** - 大于或等于比较。

**signedness** 属性指定操作数的有符号性。

- **unsigned** - 将操作数视为无符号整数。
- **signed** - 将操作数视为有符号整数。
- 该操作根据特定的操作数和属性有条件地可推测。
- 该操作可以在没有副作用的情况下被推测性地执行。
- 该操作是纯的，不执行任何内存副作用。
- **lhs** 和 **rhs** 必须具有相同的形状和元素类型 (tile<i1|i8|i16|i32|i64 >)。
- 结果类型具有 i1 元素类型，且形状与操作数相同
- 操作的结果类型可以从其操作数和属性中推断出来。



#### 示例

```
%lhs0 = constant <i16: 0> : tile<i16>
%rhs0 = constant <i16: 0> : tile<i16>
// Scalar "signed less than" comparison.
%x0 = cmpi less_than %lhs0, %rhs0, signed : tile<i16> -> tile<i1>
%lhs1 = constant <i64: 0> : tile<2x2xi64>
%rhs1 = constant <i64: 0> : tile<2x2xi64>
// Tile equality comparison.
// There is no difference between "signed" and "unsigned" when performing
// equality and inequality comparison.
%x1 = cmpi equal %lhs1, %rhs1, signed : tile<2x2xi64> -> tile<2x2xi1>
```

请参阅 `cuda_tile.cmpi_0` 获取完整的示例列表。

### 8.3.5 cuda\_tile.constant

*Construct a constant tile*

```
cuda_tile.constant %value
```

#### 参数

- **value** ( DenseConstant ) - 要创建的常量值。 13.1
- **result** ( tile<i1|i8|i16|i32|i64|f16|bf16|f32|f64|fp8e4m3fn|fp8e5m2|tf32> ) - 常量图块。 13.1

`constant` 操作创建一个由 `$value` 初始化的瓦片。

There are two main forms of using the operation:

- 一种情况是，该值是由 `<D: c>` 指定的单个常量，且图块的所有元素（元素类型为 `D`）都填充为相同的值。
- 其中值是由 `dense<D: [c0, c1, c2, ...]>` 指定的常量列表，且常量值的形状必须与瓦片的形状匹配，元素类型为 `D`。

图块的注释类型限制了它的秩、形状和元素类型。

#### 约束条件

- 该操作没有操作数，并且可以进行常量折叠。
- 该操作根据特定的操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯粹的，不会产生任何内存副作用。
- `value` 和 `result` 必须具有相同的形状和元素类型（`DenseConstant`）。
- 操作的结果类型可以从其操作数和属性中推断出来。

#### 示例

```
%c0 = constant <i32: 0> : tile<i32>
%c1 = constant <i64: 1> : tile<i64>
%c2 = constant <i32: [0, 1, 2, 3]> : tile<4xi32>
%c3 = constant <f32: 0.0> : tile<2x4xf32>
%c4 = constant <f64: [0.0, 1.0, 2.0, 3.0]> : tile<4xf64>
```

请参阅 `cuda_tile.constant_0` 获取完整的示例列表。

### 8.3.6 cuda\_tile.entry

Define a tile kernel

```
cuda_tile.entry %sym_name %function_type %arg_attrs %res_attrs %  
optimization_hints
```

#### 参数

- **sym\_name** ( Symbol ) - 函数名称。 [13.1](#)
- **function\_type** ( 类型 ) - 函数的类型。 [13.1](#)
- **arg\_attrs** ( 属性 ) - 函数的参数属性: TileIR 目前不支持其中任何属性。 [13.1](#)
- **res\_attrs** ( 属性 ) - 函数的结果属性: TileIR 目前不支持其中任何属性。 [13.1](#)
- **optimization\_hints** ( OptimizationHints ) - 编译器架构特定的优化提示 [13.1](#)

No results.

**描述** entry 操作定义了一个瓦片内核; 内核是一种可以作为程序入口点的函数。它在每个模块中具有唯一的名称。内核不能返回任何值。必须使用 cuLaunchKernel 或类似的 CUDA 运行时 API 函数从主机端启动。

瓦片内核要求用户在启动时指定三维网格维度, 这定义了将并行执行内核的瓦片块 (或内核实例) 数量。

有关瓦片内核的详细语义, 请参阅瓦片内核。

optimization\_hints 属性以嵌套字典的形式提供特定于体系结构的编译器提示。

提示是针对每个架构 (例如, sm\_100、sm\_120) 指定的, 并且对于每个架构, 用户可以为每个操作指定具体的提示。

- **num\_cta\_in\_cga** - 为 cuda\_tile.entry 建议 CGA 中的 CTA 数量 (必须是小于或等于 16 的 2 的幂次)。
- **allow\_tma** - 建议是否对 cuda\_tile.load\_view\_tko 和 cuda\_tile.store\_view\_tko 使用 TMA。
- **latency** - 为 cuda\_tile.load\_view\_tko 和 cuda\_tile.store\_view\_tko 提供的延迟提示。

For example they can be annotated as:

```
optimization_hints=<  
  sm_100 = {num_cta_in_cga = 8},  
  sm_120 = {num_cta_in_cga = 16}  
>
```

#### 约束条件

- 操作必须是全局符号表中的一个符号。
- 操作必须实现可调用的目标接口。
- 该操作必须实现类似函数的行为接口。
- 该区域不得捕获在操作上方定义的 SSA 值。
- 该操作必须提供自定义的解析和打印方法。
- 每个提供的区域必须恰好包含一个块。

### 8.3.7 cuda\_tile.extract

*Extract a subtile from a tile*

```
cuda_tile.extract %source %indices
```

#### 参数

- **source** ( tile ) - 从中提取数据的源 tile。 [13.1]
- **indices** ( 可变参数<瓦片<i32>> ) - 要提取的切片索引。 [13.1]
- **result** ( tile ) - 提取出的子瓦片。 [13.1]

**extract** 操作从给定的源图块中提取子图块。

结果瓦片的形状必须能整除源瓦片的形状，例如，从 `tile<8xf32>` 中提取 `tile<4xf32>` 是有效的，但提取 `tile<3xf32>` 则无效。

`$indices` 表示要提取的切片编号，但重要的是，它并非用于构建提取子图块的偏移量。提取的语义意味着只能提取完整大小的切片。

具有相同形状的源图块切片，根据定义，对于唯一索引而言是非重叠的。

`indices` 操作数被解释为无符号整数。

Warning

如果 `indices` 指定了一个不存在的（即越界的）切片，则该操作的行为是未定义的。

#### 约束条件

- 该操作基于特定操作数和属性有条件地可推测。
- 该操作可以推测性地执行，而不会产生副作用。
- 该操作是纯粹的，不会产生任何内存副作用。
- `source` and `result` must have the same rank.

#### 示例

```
// Extract a subtile from %t at dim_0 = [4;8) and dim_1 = [4;6).
%c1 = constant <i32: 1> : tile<i32>
%c2 = constant <i32: 2> : tile<i32>
%t = constant <f32: 0.0> : tile<32x8xf32>
// Valid indices are: [ {0, 1, 2, 3, 4, 5, 6, 7}, {0, 1, 2, 3} ]
%0 = extract %t[%c1, %c2]
: tile<32x8xf32> -> tile<4x2xf32>
```

请参阅 `cuda_tile.extract_0` 获取完整示例列表。

### 8.3.8 cuda\_tile.get\_global

*Get a pointer to a global variable*

```
cuda_tile.get_global %name
```

#### 参数

- **name** ( Symbol ) - 全局变量的名称。 [13.1]
- **result** ( tile<ptr> ) - `get_global` 操作的结果。 [13.1]

`get_global` 操作返回指向指定 `global` 变量的指针。全局变量是一种静态全局内存分配形式，可以使用 `cuda_tile.global` 操作来声明。

返回指针的元素类型将与声明的全局变量的元素类型相同。

有关全局变量的详细语义，请参见全局变量。

### 约束条件

- 该操作根据特定的操作数和属性有条件地可推测。
- 该操作可以无副作用地推测性执行。
- 该操作是纯函数，不执行任何内存副作用。

#### 示例

```
global @val <f32: [0.1, 0.2, 0.3, 0.4]> : tile<4xf32>
entry @example() {
    %ptr = get_global @val : tile<ptr<f32>>
    return
}
```

完整示例列表请参见 `cuda_tile.get_global_0`。

### 8.3.9 `cuda_tile.get_num_tile_blocks`

*Get total number of tile blocks*

```
cuda_tile.get_num_tile_blocks
```

**参数** No parameters.

### 结果

- `gridSize_x` ( `tile<i32>`) - 维度 `x` 中的瓦片块数量。 13.1
- `gridSize_y` ( `tile<i32>`) - 维度 `y` 中的瓦片块数量。 13.1
- `gridSize_z` ( `tile<i32>`) - 维度 `z` 中的瓦片块数量。 13.1

`get_num_tile_blocks` 操作以指定每个网格维度范围的 3 元组形式查询图块块的总数。

一个瓦片 `id` 是三维空间中的一个坐标，因此它也必须是一个包含每个维度范围的 3 元组: `x`、`y` 和 `z`。当启动一维或二维网格时，未指定的维度将具有基数 1。

例如，如果用于启动内核的网格是 (1024, 1024)，那么此操作的结果将是 (1024, 1024, 1)。

Note

**网格维度限制：**网格维度限制为  $2^{24}-1$  (16,777,215)

per axis. Larger dimensions may result in incorrect tile block ID calculations. Use multiple kernel launches for larger workloads.

### 约束条件

- 该操作是否可推测取决于特定的操作数和属性。
- 该操作可以无副作用地推测执行。
- 该操作是纯粹的，不执行任何内存副作用。
- 操作的结果类型可以从其操作数和属性中推断出来。

#### 示例

```
entry @example() {
    %x, %y, %z = get_num_tile_blocks : tile<i32>
    // print "x: %, y: %, z: %\n", %x, %y, %z : tile<i32>, tile<i32>, tile<i32>
}
```

请参阅 `cuda_tile.get_num_tile_blocks_0` 获取完整的示例列表。

### 8.3.10 `cuda_tile.get_tile_block_id`

获取当前正在执行的图块块坐标

```
cuda_tile.get_tile_block_id
```

**参数** No parameters.

#### 结果

- **blockId\_x** ( tile<i32> ) - 维度 x 的图块块 ID。 13.1
- **blockId\_y** ( tile<i32> ) - 维度 y 的瓦片块 ID。 13.1
- **blockId\_z** ( tile<i32> ) - 维度 z 的瓦片块 ID。 13.1

`get_tile_block_id` 返回当前正在执行的瓦片块的三维瓦片块坐标（或 ID）。

瓦片 ID 具有三个维度：x、y 和 z。此操作同时返回所有三个维度。该操作返回的每个维度的值介于 0（包含）和相应轴上 `get_num_tile_blocks` 返回的值（不包含）之间，由闭区间 `[0, get_num_tile_blocks(dim) - 1]` 表示。内核启动时未指定的网格维度（即一维或二维网格）在所有瓦片块中始终为 0。

Note

**网格维度限制：**网格维度限制为  $2^{24}-1$  (16,777,215)

每个轴。更大的维度可能导致瓦片块 ID 计算错误。对于更大的工作负载，请使用多个内核启动。

#### 约束条件

- 该操作根据特定的操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯粹的，不执行任何内存副作用。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 8.3.11 `cuda_tile.global`

*Allocate static global memory*

```
cuda_tile.global %sym_name %value %alignment
```

#### 参数

- **sym\_name** ( Symbol ) - 全局变量的名称。 13.1
- **value** ( DenseConstant ) - 用于初始化分配的值。 13.1
- **alignment** ( i64 ) - 缓冲区的对齐方式。 13.1

No results.

**描述** `global` 操作在全局内存中静态分配一个可变的一维位置，并使用 `value` 对其进行初始化。分配的初始化在 CUDA 模块加载时执行。分配的生存期与模块的生存期相同。

分配的内存可以通过以下方式读取或写入：首先使用 `cuda_tile.get_global` 获取指向该内存的指针，然后使用 `cuda_tile.load_ptr_tko` 读取，或使用 `cuda_tile.store_ptr_tko` 写入。

初始值按线性顺序存储在内存中，因此由 `cuda_tile.get_global` 返回的指针指向第一个元素，通过偏移 `x` 可以加载位置 `x` 处的元素。

`global` 操作必须直接嵌套在 **Tile IR** 模块内。它们不能在函数内部定义。

由于全局变量定义在模块作用域中，它们的名称是全局唯一的符号，不得与模块中的任何其他符号冲突。

有关全局变量的更详细语义，请参阅全局变量。

### 约束条件

- 该操作必须是全局符号表中的一个符号。

#### 示例

```
global @val alignment = 128 <f32: [0.1, 0.2, 0.3, 0.4]> : tile<4xf32>
entry @example() {}
```

完整示例列表请参见 `cuda_tile.global_0`。

### 8.3.12 `cuda_tile.iota`

*Generate a 1-d tile range from 0 to n-1*

```
cuda_tile.iota
```

**参数** No parameters.

### 结果

- result** (`tile<i1|i8|i16|i32|i64>`) - `iota` 操作的结果。 13.1

`iota` 操作生成一个包含整数序列的一维切片。起始值为 0，步长为 1。如果结果切片的形状为 `(n)`，则生成的值为 `[0, n - 1]`。

`iota(n)` `i=ifor i [0,n-1]`

结果值应解释为无符号整数。

Note

结果图块中的元素数量不得超过元素类型所能表示的最大值。

### 约束条件

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可能会被推测性地执行，且无副作用。
- 该操作是纯的，不会产生任何内存副作用。

### 8.3.13 `cuda_tile.mmaf`

*Floating-point matrix-multiply-accumulate*

```
cuda_tile.mmaf %lhs %rhs %acc
```

## 参数

- **lhs** ( tile<f16|bf16|f32|f64|tile<tf32 > > ) - 左侧矩阵操作数。 [13.1]
- **rhs** ( tile<f16|bf16|f32|f64|tile<tf32 > > ) - 右侧矩阵操作数。 [13.1]
- **acc** ( tile<f16|f32|f64> ) - 累加器矩阵操作数。 [13.1]
- **result** ( tile<f16|f32|f64> ) - 乘加运算后的结果矩阵。 [13.1]

**mmaf** 操作实现了针对浮点矩阵块的 MMA（矩阵乘积累加）运算。

它对浮点瓦片 **lhs** 和 **rhs** 执行矩阵乘法，然后将瓦片 **acc** 加到结果上。**lhs**、**rhs** 和 **acc** 必须是二维瓦片或三维瓦片。后一种情况表示批处理矩阵乘法。

$\text{mmaf}(A, B, C)_{ij} = \sum_{k=0}^{K-1} A_{ik} \times B_{kj} + C_{ij}$

所有操作数的类型必须是受支持的组合（参见 **mmaf** 支持的数据类型）。

形状必须是有效的矩阵乘法配置。未批处理的（2D）

MMA 期望操作数 **lhs**、**rhs** 和 **acc** 的形状分别为  $M \times K$ 、 $K \times N$  和  $M \times N$ 。批处理（3D）MMA 期望操作数的形状分别为  $B \times M \times K$ 、 $B \times K \times N$  和  $B \times M \times N$ 。

下表展示了每种可能的 **mmaf** 输入类型所支持的输出类型。输入操作数必须具有相同的元素类型。

| Input Type | Supported Output Types |
|------------|------------------------|
| f8E4M3FN   | f16 or f32             |
| f8E5M2     | f16 or f32             |
| f16        | f16 or f32             |
| bf16       | f32                    |
| tf32       | f32                    |
| f32        | f32                    |
| f64        | f64                    |

## 约束条件

- 该操作根据特定的操作数和属性有条件地可推测。
- 该操作可以推测性地执行，且无副作用。
- 该操作是纯粹的，不会产生任何内存副作用。
- **acc** 和 **result** 必须具有相同的形状和元素类型（tile<f16|f32|f64 >）。
- **lhs** 和 **rhs** 必须具有相同的元素类型（tile<f16|bf16|f32|f64|tile<tf32 > >）。
- **lhs**、**rhs** 和 **acc** 必须具有相同的秩。
- 运算的结果类型可以从其操作数和属性中推断出来。

### 示例

```
%lhs0 = constant <f16: 0.0> : tile<4x8xf16>
%rhs0 = constant <f16: 0.0> : tile<8x2xf16>
%acc0 = constant <f32: 0.0> : tile<4x2xf32>
%0 = mmaf %lhs0, %rhs0, %acc0
: tile<4x8xf16>, tile<8x2xf16>,
tile<4x2xf32>
%lhs1 = constant <f16: 0.0> : tile<2x4x8xf16>
%rhs1 = constant <f16: 0.0> : tile<2x8x2xf16>
%acc1 = constant <f32: 0.0> : tile<2x4x2xf32>
%1 = mmaf %lhs1, %rhs1, %acc1
: tile<2x4x8xf16>, tile<2x8x2xf16>,
tile<2x4x2xf32>
```

请参阅 `cuda_tile.mmaf_0` 获取完整示例列表。

### 8.3.14 cuda\_tile.mmai

*Integer matrix-multiply-accumulate*

```
cuda_tile.mmai %lhs %rhs %acc %signedness_lhs %signedness_rhs
```

#### 参数

- **lhs** ( tile<i8> ) - 左侧矩阵操作数。 13.1
- **rhs** ( tile<i8> ) - 右侧矩阵操作数。 13.1
- **acc** ( tile<i32> ) - 累加器矩阵操作数。 13.1
- **signedness\_lhs** ( Signedness ) - lhs 操作数的有符号性。 13.1
- **signedness\_rhs** ( Signedness ) - rhs 操作数的有符号性。 13.1
- **result** ( tile<i32> ) - 乘加运算后的结果矩阵。 13.1

**mmai** 操作实现了针对整数图块的 MMA（矩阵乘加）运算。

它对整数瓦片 **lhs** 和 **rhs** 执行矩阵乘法，然后将瓦片 **acc** 加到结果上。**lhs**、**rhs** 和 **acc** 必须是二维瓦片或三维瓦片。后一种情况表示批量矩阵乘法。

$$\text{mmai}(A, B, C)_{ij} = \sum_{k=0}^{K-1} A_{ik} \times B_{kj} + C_{ij}$$

输入瓦片 **lhs** 和 **rhs** 必须是整数类型 **i8**。**lhs** 和 **rhs** 的有符号性分别由属性 **signedness\_lhs** 和 **signedness\_rhs** 单独指定。累加器瓦片 **acc** 必须是 **i32** 类型，且始终解释为有符号数。输出瓦片 **result** 为 **i32** 类型，且始终解释为有符号数。

形状必须是有效的矩阵乘法配置。非批处理（2D）

MMA 期望操作数 **lhs**、**rhs** 和 **acc** 的形状分别为  $M \times K$ 、 $K \times N$  和  $M \times N$ 。批处理（3D）MMA 期望操作数的形状分别为  $B \times M \times K$ 、 $B \times K \times N$  和  $B \times M \times N$ 。

**signedness** 属性指定操作数的符号性。

- **unsigned** - 将操作数视为无符号整数。
- **signed** - 将操作数视为有符号整数。
- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯函数，不会产生任何内存副作用。
- **acc** 和 **result** 必须具有相同的形状和元素类型 (tile<i32>)。
- **lhs** 和 **rhs** 必须具有相同的元素类型 (tile<i8>)。
- **lhs**、**rhs** 和 **acc** 必须具有相同的秩。
- 操作的结果类型可以从其操作数和属性中推断出来。



#### 示例

```
%lhs0 = cuda_tile.constant <i8: 0> : tile<4x8xi8>
%rhs0 = cuda_tile.constant <i8: 0> : tile<8x2xi8>
%acc0 = cuda_tile.constant <i32: 0> : tile<4x2xi32>
%0 = mmai %lhs0, %rhs0, %acc0 signed signed
: tile<4x8xi8>, tile<8x2xi8>,
tile<4x2xi32>
%lhs1 = cuda_tile.constant <i8: 0> : tile<2x4x8xi8>
%rhs1 = cuda_tile.constant <i8: 0> : tile<2x8x2xi8>
%acc1 = cuda_tile.constant <i32: 0> : tile<2x4x2xi32>
%1 = mmai %lhs1, %rhs1, %acc1 unsigned unsigned
: tile<2x4x8xi8>, tile<2x8x2xi8>,
tile<2x4x2xi32>
```

请参阅 `cuda_tile.mmai_0` 获取完整的示例列表。

### 8.3.15 cuda\_tile.module

包含一系列已定义项的顶级模块。

```
cuda_tile.module %sym_name
```

#### 参数

- `sym_name` ( Symbol ) - 模块的名称。 13.1

No results.

**描述** `module` 操作表示一个单独的编译单元，并包含零个或多个项（全局变量、函数或内核）。

有关模块语义的详细说明以及每种项目类型的完整定义，请参阅模块。

`module` 操作是 **Tile IR** 模块中的顶层操作，必须仅包含 **Tile IR** 操作，而不能包含其他方言。

#### 约束条件

- 该区域不得捕获在操作上方定义的 SSA 值。
- 该操作必须提供自定义的解析和打印方法。
- All regions must have zero arguments.
- 每个提供的区域必须恰好包含一个区块。
- The operation must define a symbol scope.
- 该区域不得要求显式的终止操作。
- 操作必须指定区域是 SSACFG 类型还是 Graph 类型。
- 操作必须仅包含数据流图区域。

### 8.3.16 cuda\_tile.offset

*Offsets a tile of pointers*

```
cuda_tile.offset %ptr %offset
```

#### 参数

- `ptr` ( ptr ) - 要推进的基础指针图块。 13.1
- `offset` ( tile<i1|i8|i16|i32|i64> ) - 要加到指针上的偏移量图块。 13.1

- **result** ( ptr ) - 前进后得到的结果指针瓦片。13.1

**offset** 使指针块前进。它以 **ptr** 为基址，**offset** 为增量，并执行 **ptr** 与 **offset** 的逐元素加法：  

$$\text{offset}(\text{ptr}, \text{offset})i = \text{ptr}i + \text{offset}i \times \text{bitwidth}$$

```
result[i,j] = ptr[i,j] + offset[i,j] * bitwidth
```

**ptr** 被解释为无符号整数。**offset** 被解释为有符号整数。**bitwidth** 是指针指向类型的存储位宽。乘法在符号意义上不得溢出（回绕）。加法在无符号意义上不得溢出（回绕）。如果发生溢出，结果是未定义的。

### 约束条件

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可能会被推测性地执行，且不产生副作用。
- 该操作是纯的，不执行任何内存副作用。
- 该操作必须逐元素地应用于其操作数。
- **ptr**、**offset** 和 **result** 必须具有相同的形状。
- **result** 和 **ptr** 必须具有相同的形状和元素类型（**ptr**）。
- 操作的结果类型可以从其操作数和属性中推断出来。

#### 8.3.17 cuda\_tile.pack

*Pack a tile into a byte array*

```
cuda_tile.pack %source
```

### 参数

- **source** ( tile<i1|i8|i16|i32|i64|f16|bf16|f32|f64|fp8e4m3fn|fp8e5m2|tf32> ) - 输入图块。13.2
- **result** ( tile<i8> ) - 打包后的 tile。13.2

**pack** 操作接收一个秩为 1 的数值瓦片，并生成一个秩为 1 的 **tile<i8>**。

类似于 **bitcast**，底层的比特值不会改变。然而，**pack** 并非逐元素操作，而是将整个图块重新解释为字节数组。

输入和输出图块必须是一维的，以消除打包歧义。输出图块的大小必须与输入图块中的字节数相匹配。

### 约束条件

- 该操作基于特定操作数和属性有条件地可推测。
- 该操作可以在无副作用的情况下被推测执行。
- 该操作是纯粹的，不执行任何内存副作用。
- **source** and **result** must have the same rank.

#### 示例

```
%arg0 = constant <f16: 0.0> : tile<64xf16>
%0 = pack %arg0 : tile<64xf16> -> tile<128xi8>
```

请参阅 `cuda_tile.pack_0` 获取完整示例列表。

```
%arg0 = constant <f4E2M1FN: 0.0> : tile<64xf4E2M1FN>
%0 = pack %arg0 : tile<64xf4E2M1FN> -> tile<32xi8>
```

完整示例列表请参见 `cuda_tile.pack_1`。

### 8.3.18 `cuda_tile.permute`

*Permute tile dimensions*

```
cuda_tile.permute %source %permutation
```

#### 参数

- **source** ( tile ) - 输入瓦片。 [13.1](#)
- **permutation** ( Array<i32> ) - 维度的排列。 [13.1](#)
- **result** ( tile ) - 置换后的 tile。 [13.1](#)

根据 `permutation` 数组置换输入瓦片 `source` 的维度。

`permutation` 数组是一个整数列表，用于指定维度的新顺序。

例如，如果输入瓦片的形状为 [2, 4, 8]，且排列顺序为 [2, 0, 1]，则输出瓦片的形状将为 [8, 2, 4]。

这个操作在逻辑上是图块索引的更改。

#### 约束条件

- 该操作根据特定的操作数和属性有条件地可推测。
- 该操作可以在无副作用的情况下被推测性地执行。
- 该操作是纯粹的，不会产生任何内存副作用。
- `source` 和 `result` 必须具有相同的元素类型 (tile)。
- `source` and `result` must have the same rank.

#### 示例

```
%arg0 = constant <f16: 0.0> : tile<2x4x8xf16>
%0 = permute %arg0 [2, 0, 1] : tile<2x4x8xf16> -> tile<8x2x4xf16>
```

请参阅 `cuda_tile.permute_0` 获取完整示例列表。

### 8.3.19 `cuda_tile.reduce`

*Variadic tile reduction across dimensions*

```
cuda_tile.reduce %operands %dim %identities
```

## 参数

- **操作数** (可变参数<瓦片 >) - 要归约的瓦片集合。 13.1
- **dim** ( i32 ) - 执行归约操作的维度索引。 13.1
- **identities** ( Array ) - 每个操作数的归约恒等值。 13.1
- **results** ( 可变参数<tile>) - 归约后的 tile 集合。 13.1

**reduce** 操作沿着一个或多个输入瓦片的指定维度应用自定义归约函数，生成相同数量的输出瓦片。

归约函数必须是定义在 **reduce** 操作区域内的一种结合操作。单个归约操作可以并行归约任意数量的输入分片，并为每个分片生成一个归约后的输出分片。

所有输入图块必须具有相同的形状。输出图块将在除被缩减的维度外的每个维度上具有匹配的形状，该维度将被移除。

对于每个输入图块，必须提供一个恒等值，该值需与输入图块元素类型相匹配。**identities** 中的恒等值 **i** 对应 **operands** 中的输入图块 **i**。正确的恒等值是 **body** 中归约函数的属性。（例如，如果归约函数执行 **min**，则恒等值为 **+inf**；而如果归约函数执行 **sum**，则恒等值为 0。）

归约函数必须接收 **2N** 个参数，其中 **N** 是输入图块的数量。

每对归约参数 **2i** 和 **2i+1** 将对应第 **i** 个输入瓦片。

每对参数中的第一个是输入分块的一个元素；第二个是沿指定维度所有先前归约的累加器。这第二个值可能是输入元素、单位值或先前归约迭代的结果。归约函数应为每个输入分块生成新的累加器值。

Note

无法保证沿指定维度元素归约的顺序。

然而，在同一设备上运行相同内核的不同次执行中，结果是确定性的。

## 约束

- 该操作必须提供自定义的解析和打印方法。
- 该操作有且仅在该区域的操作有效时才有效。
- All operands must have identical shapes.
- 每个提供的区域必须恰好包含一个区块。

### 示例

```
%input = constant <f32: 0.0> : tile<8xf32>
%0 = reduce %input dim=0 identities=[0.000000e+0 : f32] : tile<8xf32> ->
    tile<f32>
(%input_arg: tile<2xf32>, %input_accum: tile<f32>) {
%add_result = addf %input_arg, %input_accum : tile<f32>
yield %add_result : tile<f32>
}
```

请参阅 `cuda_tile.reduce_0` 查看完整的示例代码清单。

```
%input = constant <f32: 0.0> : tile<8x64xf32>
%0 = reduce %input dim=0 identities=[0.000000e+0 : f32] : tile<8x64xf32> -> tile
    <8xf32>
(%input_arg: tile<f32>, %input_accum: tile<f32>) {
%add_result = addf %input_arg, %input_accum : tile<f32>
yield %add_result : tile<f32>
}
```

请参阅 `cuda_tile.reduce_1` 获取完整示例列表。

### 8.3.20 cuda\_tile.reshape

*Reshape tile dimensions*

```
cuda_tile.reshape %source
```

#### 参数

- **source** ( tile ) - 要重塑形状的源 tile。 13.1
- **result** ( tile ) - 重塑后的 tile。 13.1

**reshape** 操作会改变 **source** 操作数的形状。**reshape** 仅是对图块索引方式的更改。元素数量和元素类型必须保持不变。

0 维瓦片（即标量）恰好包含一个元素，因此是唯一例外，即 0 维瓦片可以重塑为 `size(shape) == 1` 的形状。

从概念上讲，重塑一个数据块等同于首先假设行优先布局，从源数据创建一个一维数据块，然后在行优先布局中将该一维数据块转换为新形状。

#### 约束条件

- 该操作基于特定的操作数和属性有条件地可推测。
- 该操作可以推测性地执行，且不产生副作用。
- 该操作是纯粹的，不执行任何内存副作用。
- **source** 和 **result** 必须具有相同的元素类型（tile）。

#### 示例

```
%cst = constant <i8: 0> : tile<i8>
%0 = reshape %cst
: tile<i8> -> tile<1x1x1xi8>
%t = constant <f32: 0.0> : tile<8x2xf32>
%1 = reshape %t
: tile<8x2xf32> -> tile<2x2x4x1xf32>
```

完整示例列表请参见 `cuda_tile.reshape_0`。

```
%cst = constant <i32: [[0, 1, 2, 3], [4, 5, 6, 7]]>
: tile<2x4xi32>
%r0 = reshape %cst
: tile<2x4xi32> -> tile<2x2x2xi32>
// Step 1: Turn source into 1D tile. Use row-major by convention.
// %tmp: [0, 1, 2, 3, 4, 5, 6, 7]
%tmp = reshape %cst
: tile<2x4xi32> -> tile<8xi32>
// Step 2: Turn 1D tile into result tile. Use row-major by convention.
// %r: [[0, 1], [2, 3]], [[4, 5], [6, 7]]
%r1 = reshape %tmp
: tile<8xi32> -> tile<2x2x2xi32>
```

完整示例列表请参见 `cuda_tile.reshape_1`。

### 8.3.21 cuda\_tile.scan

*A parallel prefix sum operation*

```
cuda_tile.scan %operands %dim %reverse %identities
```

## 参数

- **操作数** (可变数量<瓦片>) - 要扫描的一组瓦片。 [13.1]
- **dim** ( i32 ) - 沿其进行扫描的维度索引。 [13.1]
- **reverse** ( bool ) - 是否以相反顺序进行扫描。 [13.1]
- **identities** ( Array ) - 扫描操作的恒等值。 [13.1]
- **results** ( 可变参数<tile>) - 扫描操作产生的 tile 结果。 [13.1]

**scan** 操作使用二元关联函数和单位元，沿输入瓦片的给定维度计算包含性并行前缀。

**scan** 操作对给定类型的一组元素瓦片应用定义的扫描函数，利用结合性操作和单位元值。它在指定的 **dim** 维度上对 **operands** 和 **identities** 进行操作，生成新的 **results** 瓦片值。每个前缀内的具体求值顺序由实现定义，但在同一设备上运行相同内核的不同执行中，结果保持确定性。

`scan(X, dim, identity, f) i1, ..., id[j] = fold(f, identity, (X i1, ..., idim-1, 0, idim+1, ..., id, ..., X i1, ..., idim-1, j, idim+1, ..., id))`

扫描保留所有中间累加器值：

`result[0]=f(identity,X[...,0,...])`

`result[1]=f(result[0],X[...,1,...])`

`result[j]=f(result[j-1],X[...,j,...])`

当 **reverse** 为 **true** 时，前缀按递减的索引顺序获取。

Let N be the size of the scanned dimension; then:

`scanrev(X)[j] = fold(f, identity, (X[..., N-1, ...], ..., X[..., j, ...]))`

**identities** 属性是每个输入图块的标识元素列表；位置 **i** 处的标识与相同位置的操作数图块绑定。正确的标识是 **body** 中扫描函数的属性（例如，**sum** 使用 0，**prod** 使用 1，**min** 使用 **+inf**，**max** 使用 **-inf**）。

**body** 区域表示二元关联操作。该区域必须包含 **Tile IR** 操作，且这些操作使用 0 秩 (0-rank) 的 tile 类型。区域参数按操作数顺序绑定为 `[op_0_current_iter, op_0_prev_iter, op_1_current_iter, op_1_prev_iter, ...]`，其中 `op_i_current_iter` 是沿 **dim** 维度的当前元素，而 `op_i_prev_iter` 是操作数 **i** 的运行累加器。在第一步中，累加器是对应的单位元。

Note

二元运算的结合律允许编译器重新组织运算的应用，以便在 GPU 上实现高效的并行前缀扫描。

Warning

扫描操作仅限于支持单块瓦片输入。

## 约束条件

- 该操作必须提供自定义的解析和打印方法。
- 当且仅当该区域的操作生效时，此操作才会生效。
- All operands must have identical shapes.
- 每个提供的区域必须恰好包含一个块。

### 示例

```
%input = constant <f32: 0.0> : tile<8x16xf32>
%result = scan %input dim=1 reverse=false identities=[1.0 : f32] : tile<8
    x16xf32> -> tile<8x16xf32>
(%acc: tile<f32>, %elem: tile<f32>) {
%prod = mulf %acc, %elem rounding<nearest_even>: tile<f32>
yield %prod : tile<f32>
}
```

完整示例列表请参见 `cuda_tile.scan_0`。

### 8.3.22 cuda\_tile.select

Select values based on condition

```
cuda_tile.select %cond %val_if_true %val_if_false
```

#### 参数

- **cond** ( tile<i1> ) - 条件瓦片。 13.1
- **val\_if\_true** ( tile ) - 条件为真时的值 tile。 13.1
- **val\_if\_false** ( tile ) - 条件为假时的值 tile。 13.1
- **result** ( tile ) - 所选值的 tile。 13.1

**select** 操作根据作为 **cond** 操作数提供的二进制条件选择值。**val\_if\_true** 操作数包含条件为 1 时要使用的值。**val\_if\_false** 操作数包含条件为 0 时要使用的值。选择根据条件瓦片中的值逐元素进行。

`select(cond,x,y)i={xiif condi=1yiif condi=0`

所有图块必须具有相同的形状。图块 **val\_if\_true**、**val\_if\_false** 和结果必须具有相同的元素类型。**cond** 图块必须是 **i1** 值的图块。

#### 约束

- 该操作根据特定的操作数和属性有条件地可推测。
- 该操作可以在没有副作用的情况下被推测性地执行。
- 该操作是纯粹的，不会产生任何内存副作用。
- **val\_if\_true**、**val\_if\_false** 和 **result** 必须具有相同的形状和元素类型 (tile)。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 8.3.23 cuda\_tile.unpack

Unpack a byte array into a tile

```
cuda_tile.unpack %source
```

#### 参数

- **source** ( tile<i8> ) - 输入的 i8 tile。 13.2
- **result** ( tile<i1|i8|i16|i32|i64|f16|bf16|f32|f64|fp8e4m3fn|fp8e5m2|tf32> ) - 输出的解包后的 tile。 13.2

**unpack** 操作接收一个秩为 1 的 **tile<i8>** 并生成一个秩为 1 的数值 tile。

类似于 **bitcast**，底层的比特值不会改变。然而，**unpack** 并不按元素逐个操作，而是将整个图块重新解释为不同元素类型的数值图块。

输入和输出瓦片必须是一维的，以消除解包歧义。输入瓦片的大小必须与输出瓦片中的字节数匹配。

## 约束条件

- 该操作基于特定操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯的，不执行任何内存副作用。
- `source` and `result` must have the same rank.

### 示例

```
%arg0 = constant <i8: 0> : tile<64xi8>
%0 = unpack %arg0 : tile<64xi8> -> tile<32xf16>
```

完整示例列表请参见 `cuda_tile.unpack_0`。

```
%arg0 = constant <i8: 0> : tile<64xi8>
%0 = unpack %arg0 : tile<64xi8> -> tile<128xF4E2M1FN>
```

完整示例列表请参见 `cuda_tile.unpack_1`。

## 8.4 类型转换

**Tile IR** 中没有隐式类型转换，因此我们提供了一组显式转换操作，用于在具有兼容表示形式或转换规则的类型之间进行相互转换。

`cuda_tile.bitcast` 保留输入内容但允许更改元素类型，`cuda_tile.exti` 和 `cuda_tile.trunci` 改变整数图块的宽度，`cuda_tile.ftoi` 和 `cuda_tile.itof` 将浮点图块转换为整数图块，反之亦然，而 `cuda_tile.ftof` 在不同浮点类型之间进行转换。

有关转换及其规则的更多详细信息，请参阅各个操作的文档。

### 8.4.1 `cuda_tile.bitcast`

*Bitcast a tile from one element type to another*

```
cuda_tile.bitcast %source
```

### 参数

- **source** ( `tile<i1|i8|i16|i32|i64|f16|bf16|f32|f64|fp8e4m3fn|fp8e5m2|tf32>`) - 要转换的源 tile。  
13.1
- **result** ( `tile<i1|i8|i16|i32|i64|f16|bf16|f32|f64|fp8e4m3fn|fp8e5m2|tf32>`) - 转换后的 tile。13.1

`bitcast` 操作将输入图块从一种元素类型转换为另一种元素类型，而不修改底层位。

仅允许相同位宽的非指针类型（例如，从 `i32` 到 `f32`）。

指针类型必须使用 `cuda_tile.ptr_to_int` 或 `cuda_tile.int_to_ptr` 替代。

## 约束条件

- 该操作基于特定的操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯的，不执行任何内存副作用。



### 8.4.2 cuda\_tile.exti

*Extend the width of an integer tile*

```
cuda_tile.exti %from %signedness
```

#### 参数

- **from** ( tile<i1|i8|i16|i32|i64> ) - 要扩展的输入整数 tile。 13.1
- **有符号性** (Signedness) - 将整数解释为 有符号或 无符号 13.1
- **to** ( tile<i1|i8|i16|i32|i64> ) - 扩展整数瓦片。 13.1

**exti** 操作将给定宽度的整数瓦片转换为严格更大的宽度。对于 **unsigned** 整数使用零扩展, 对于 **signed** 整数使用符号扩展。

**signedness** 属性指定操作数的符号性。

- **unsigned** - 将操作数视为无符号整数。
- **signed** - 将操作数视为有符号整数。
- 该操作是否可推测执行取决于特定的操作数和属性。
- 该操作可以无副作用地推测执行。
- 该操作是纯粹的, 不执行任何内存副作用。

### 8.4.3 cuda\_tile.ftof

*Convert between floating-point types*

```
cuda_tile.ftof %from %rounding_mode
```

#### 参数

- **from** ( tile<f16|bf16|f32|f64|fp8e4m3fn|fp8e5m2|tf32> ) - 输入的浮点 tile。 13.1
- **rounding\_mode** ( RoundingMode ) - 该运算的舍入模式。 13.1
- **to** ( tile<f16|bf16|f32|f64|fp8e4m3fn|fp8e5m2|tf32> ) - 结果浮点 tile。 13.1

**ftof** 操作将给定浮点元素类型的图块转换为另一种浮点元素类型的图块（例如，从 **f32** 转换为 **f64**）。

源类型和结果类型必须不同。

**rounding\_mode** 属性指定了操作的舍入行为。

Only **NEAREST\_EVEN** rounding mode is supported.

**rounding** 属性指定了用于此操作的舍入模式。

- **nearest\_even** - 向最接近的值舍入（平局时取偶数）。
- **zero** - Round towards zero (truncate).
- **negative\_inf** - 向负无穷方向舍入。
- **positive\_inf** - 向正无穷方向舍入。
- **approx** - Approximate rounding mode.
- **full** - Full precision rounding mode.
- **nearest\_int\_to\_zero** - 向零方向舍入到最接近的整数。

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯粹的，不会产生任何内存副作用。

#### 8.4.4 **cuda\_tile.ftoi**

将瓦片从浮点数值转换为整数值

```
cuda_tile.ftoi %from %signedness %rounding_mode
```

#### 参数

- **from** ( **tile**<**f16**|**bf16**|**f32**|**f64**|**fp8e4m3fn**|**fp8e5m2**|**tf32**>) - 输入的浮点型 **tile**。 13.1
- **有符号性** (Signedness) - 将整数解释为 有符号或 无符号 13.1
- **rounding\_mode** ( **RoundingMode** ) - 该操作的舍入模式。 13.1
- **to** ( **tile**<**i1**|**i8**|**i16**|**i32**|**i64**>) - 结果整数 **tile**。 13.1

**ftoi** 操作将浮点数瓦片转换为整数瓦片。

与保留位数的**cuda\_tile.bitcast**不同，此操作会保留图块的数值，并向零舍入到所提供类型的最近整数。

**rounding\_mode** 属性指定了操作的舍入行为。

仅支持 **NEAREST\_INT\_TO\_ZERO** 舍入模式。

Warning

如果输入浮点值在舍入后超出

（有符号或无符号）目标整数类型的范围内，将使用最接近的可表示值。**NaN** 值会被转换为 0。

Input **Inf** values are undefined behavior.

**signedness** 属性指定操作数的有符号性。

- **unsigned** - 将操作数视为无符号整数。
- **signed** - 将操作数视为有符号整数。

**rounding** 属性指定了用于此操作的舍入模式。

- **nearest\_even** - 四舍五入到最接近的值（平局时取偶数）。
- **zero** - Round towards zero (truncate).
- **negative\_inf** - 向负无穷方向舍入。
- **positive\_inf** - 向正无穷方向舍入。
- **approx** - Approximate rounding mode.
- **full** - Full precision rounding mode.
- **nearest\_int\_to\_zero** - 向零方向舍入到最接近的整数。

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯粹的，不执行任何内存副作用。

#### 8.4.5 **cuda\_tile.itof**

*Convert integer to floating-point*

```
cuda_tile.itof %from %signedness %rounding_mode
```

#### 参数

- **from** ( tile<i1|i8|i16|i32|i64> ) - 输入的整数瓦片。 13.1
- **有符号性** (有符号性) - 将整数解释为 有符号或 无符号 13.1
- **rounding\_mode** ( RoundingMode ) - 该操作的舍入模式。 13.1
- **to** ( tile<f16|bf16|f32|f64|fp8e4m3fn|fp8e5m2|tf32> ) - 转换后的浮点型 tile。 13.1

**itof** 操作将整数图块转换为浮点图块。

与 **cuda\_tile.bitcast** 不同，此操作会保留图块的数值，并将其舍入到所提供类型的最接近的浮点数。

Warning

如果输入的整数值在四舍五入后超出了目标浮点类型的范围，对于支持该值的类型，它将被转换为 **Inf**，否则转换为 **NaN**。

**signedness** 属性指定操作数的有符号性。

- **unsigned** - 将操作数视为无符号整数。
- **signed** - 将操作数视为有符号整数。

**rounding** 属性指定了用于此操作的舍入模式。

- `nearest_even` - 四舍五入到最接近的值（平局时取偶数）。
  - `zero` - Round towards zero (truncate).
  - `negative_inf` - 向负无穷方向舍入。
  - `positive_inf` - 向正无穷方向舍入。
  - `approx` - Approximate rounding mode.
  - `full` - Full precision rounding mode.
  - `nearest_int_to_zero` - 向零方向舍入到最接近的整数。
- 该操作是否可推测取决于特定的操作数和属性。
  - 该操作可以无副作用地推测性执行。
  - 该操作是纯的，不会产生任何内存副作用。

#### 8.4.6 `cuda_tile.int_to_ptr`

将整数瓦片转换为指针瓦片

```
cuda_tile.int_to_ptr %source
```

##### 参数

- `source` ( `tile<i64>` ) - 输入的整数图块。 13.1
- `result` ( `ptr` ) - 输出的指针图块。 13.1

`int_to_ptr` 操作将整数图块转换为指针图块。  
`source` 操作数被解释为无符号整数。  
 此操作的逆运算是 `cuda_tile.ptr_to_int`。

##### 约束

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯粹的，不执行任何内存副作用。

#### 8.4.7 `cuda_tile.ptr_to_int`

将指针图块转换为整数图块

```
cuda_tile.ptr_to_int %source
```

##### 参数

- `source` ( `ptr` ) - 输入的指针图块。 13.1

- **result** ( tile<i64>) - 整数输出瓦片。 13.1

**ptr\_to\_int** 操作将一个指针类型元素的图块转换为一个 i64 类型元素的图块。  
 结果值应解释为无符号整数。  
 此操作的逆运算是 `cuda_tile.int_to_ptr`。

#### 约束条件

- 该操作根据特定的操作数和属性有条件地可推测。
- 该操作可能会被推测性地执行，且不产生副作用。
- 该操作是纯的，不执行任何内存副作用。

#### 8.4.8 `cuda_tile.ptr_to_ptr`

将一种指针类型的图块重新解释为另一种

```
cuda_tile.ptr_to_ptr %source
```

#### 参数

- **source** ( ptr ) - 具有源指针元素类型的 Tile。 13.1
- **result** ( ptr ) - 具有目标指针元素类型的 Tile。 13.1

**ptr\_to\_ptr** 操作将指针图块从一种元素类型的指针转换为另一种元素类型的指针。不允许在指针和非指针类型之间进行转换。

为了执行这些转换，请使用 `cuda_tile.ptr_to_int` 或 `cuda_tile.int_to_ptr`。  
 这些操作是独特的，以便未来编译器能够对指针来源进行推理。

#### 约束条件

- 该操作根据特定的操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯粹的，不会产生任何内存副作用。

#### 8.4.9 `cuda_tile.trunci`

*Truncates the width of an integer tile*

```
cuda_tile.trunci %from %overflow
```

#### 参数

- **from** ( tile<i1|i8|i16|i32|i64>) - 要截断的输入整数 tile。 13.1
- **overflow** ( IntegerOverflow ) - 操作的溢出行为。 13.1
- **to** ( tile<i1|i8|i16|i32|i64>) - 截断的整数瓦片。 13.1

**trunci** 操作将给定元素类型的整数图块转换为宽度严格更小的图块。

可选的溢出属性指定了在将操作数解释为有符号和/或无符号整数时是否可能发生溢出。对于“无符号溢出”的情况，所有被截断的位必须与被截断结果的最高有效位具有相同的值。对于“无符号溢出”的情况，被截断的位必须为零。

**overflow** 属性用于指示编译器如何推理特定操作的溢出行为。

这些属性作为编译器可用于推理操作的假设。代码生成器的责任是确保操作在执行过程中动态地遵守这些假设。

- **none** - 编译器不假设任何关于溢出的行为。
- **no\_signed\_wrap** - 编译器假定在将操作数解释为有符号整数时不会发生溢出（回绕）。
- **no\_unsigned\_wrap** - 编译器假定在将操作数解释为无符号整数时不会发生溢出（回绕）。
- **no\_wrap** - 编译器在将操作数解释为有符号或无符号整数时，假定不会发生溢出（回绕）。

如果运行时发生溢出，尽管溢出的值表明并非如此，其行为是未定义的。

## 约束条件

- 该操作根据特定的操作数和属性有条件地可推测。
- 该操作可以在没有副作用的情况下被推测性地执行。
- 该操作是纯粹的，不执行任何内存副作用。

## 8.5 控制流

**Tile IR** 包含一组标准的控制流操作，支持条件判断和循环。

这些操作的设计风格参考了 MLIR 控制流方言。

一个显著的区别在于我们允许控制流操作的嵌套，例如 `cuda_tile.if` 可以出现在 `cuda_tile.loop` 或 `cuda_tile.for` 内部。

The main control structures are:

- `cuda_tile.if` 用于实现条件分支。
- `cuda_tile.loop` 实现了具有任意退出条件的循环。
- `cuda_tile.for` 实现了一个基于范围的循环，具有固定的迭代次数。

这些操作及其支持操作将在下一节中描述。

### 8.5.1 `cuda_tile.assert`

如果条件为假值，则终止内核执行并显示错误消息

```
cuda_tile.assert %condition %message
```

#### 参数

- **condition** ( `tile<i1>` ) - 要检查的条件瓦片。 13.1
- **message** ( `String` ) - 断言失败时显示的错误信息。 13.1

No results.

**描述** `assert` 操作以 `i1` 值构成的瓦片作为 `condition`。对于每个值为 0 的元素，它会打印给定的错误消息，以及该值在瓦片内的索引。

如果至少有一个值为 0，则会向主机端发送错误信号。内核（包括断言失败的瓦片块）可能会继续运行。

断言用于调试目的。它们可能会影响性能，因此建议在生产代码中移除它们。

**约束条件** No constraints.

**示例**

```
assert %arg0, "assertion failed" : tile<i1>
```

请参阅 `cuda_tile.assert_0` 查看完整示例列表。

## 8.5.2 `cuda_tile.break`

*Break from loop*

```
cuda_tile.break %operands
```

**参数**

- **操作数** (可变参数<任意类型 >) - 在循环终止时返回给父循环的操作数。 13.1

No results.

**描述** `break` 操作是 `cuda_tile.loop` 的一个终止操作。

它可能在终止时向父循环产生任意数量的 `$operands`。所产生的值的数量以及它们如何产生的执行语义由父循环决定。

`break` 操作总是将控制权返回到最内层的封闭循环操作，即使它嵌套在其他控制结构（如 `if` 或额外的循环）中也是如此。

**约束条件**

- 该操作必须终止其父基本块。

**示例**

```
// Break from the body of a loop.
loop {
  break
}

// Break from an if nested within the loop.
loop {
  %condition = constant <i1: 1> : tile<i1>
  if %condition {
    break
  }
  // ...
}

%initValue0 = constant <f32: 0.0> : tile<f32>
// Break from an if nested within the loop, while yielding values.
%results = loop iter_values(%var0 = %initValue0): tile<f32> -> tile<f32> {
  %condition = constant <i1: 1> : tile<i1>
  if %condition {
    // ...
    yield
  } else {
    // %if.loopValue0 = ...
    %loopValue0 = constant <f32: 1.0> : tile<f32>
    break %loopValue0 : tile<f32>
  }
  %loopValue1 = constant <f32: 1.0> : tile<f32>
  continue %loopValue1 : tile<f32>
}
```

完整示例列表请参见 `cuda_tile.break_0`。

### 8.5.3 cuda\_tile.continue

*Continue to next loop iteration*

```
cuda_tile.continue %operands
```

#### 参数

- **操作数** (可变参数<任意类型 >) - 要传递给父循环的值。 13.1

No results.

**描述** `continue` 操作表示一个块终止符,它将控制权返回给循环操作,例如 `cuda_tile.for` 和 `cuda_tile.loop`。该操作在终止时可以向父循环传递任意数量的 `$operands`。

`continue` 操作的要求和语义由父循环操作定义,具体语义请参见循环操作的描述。

`continue` 操作总是将控制权返回给最内层的封闭循环操作,即使它嵌套在其他控制结构(如 `if` 或额外的循环)中也是如此。

#### 约束条件

- 该操作必须终止其父基本块。

#### 示例

```
%lowerBound = constant <i32: 0> : tile<i32>
%upperBound = constant <i32: 10> : tile<i32>
%step = constant <i32: 1> : tile<i32>
%condition = constant <i1: 1> : tile<i1>
// Continue from the body of a loop.
for %iv in (%lowerBound to %upperBound, step %step) : tile<i32> {
  continue
}
// Continue from an if nested within the loop.
for %iv in (%lowerBound to %upperBound, step %step) : tile<i32> {
  if %condition {
    continue
  }
  // ...
}
// Continue from an if nested within the loop, while yielding values.
%initVar0 = constant <f32: 0.0> : tile<f32>
%results = for %iv in (%lowerBound to %upperBound, step %step) : tile<i32>
iter_values(%var0 = %initVar0) -> (tile<f32>)
{
  if %condition {
    // ...
    yield
  } else {
    %loopValue0 = constant <f32: 1.0> : tile<f32>
    continue %loopValue0 : tile<f32>
  }
  %loopValue1 = constant <f32: 1.0> : tile<f32>
  continue %loopValue1 : tile<f32>
}
```

完整示例列表请参见 `cuda_tile.continue_0`。

### 8.5.4 cuda\_tile.for

*For loop over integer range*

```
cuda_tile.for %lowerBound %upperBound %step %initValues %unsignedCmp
```



## 参数

- **lowerBound** ( tile<any>) - 循环的下界。 [13.1]
- **upperBound** ( tile<any>) - 循环的上界。 [13.1]
- **step** ( tile<any>) - 循环的步长。 [13.1]
- **initValues** ( Variadic<Any>) - 循环传递值的初始值。 [13.1]
- **unsignedCmp** (Flag) - 如果存在，则使用无符号整数比较进行循环终止。 13.2
- **resultValues** ( 可变参数<任意类型>) - 循环结束后循环携带变量的值。 [13.1]

**for** 操作是一种基于范围的结构化顺序循环。

循环操作包括 (1) 由 **lowerBound**、**upperBound** 和 **step** 构成的范围，  
(2) 一组由 **initValues** 初始化并在循环的每次迭代中更新的循环携带值，以及  
(3) a region which represents the loop body.

迭代空间由区间 [lowerBound, upperBound) 定义，其中每个值以 **step** 为间隔分隔。

$\text{range}(\text{Lb}, \text{Ub}, \text{S}) = \{\text{Lb} + i \text{ S} \mid \text{Lb} + i \text{ S} < \text{Ub}\}$

**lowerBound**、**upperBound** 和 **step** 必须为相同类型。**lowerBound** 和 **upperBound** 指定一个半开（或排除）区间：该区间包含 **lowerBound**，但不包含 **upperBound**。**step** 必须为正数，但边界值可以为负数或零。

**lowerBound**、**upperBound** 和 **step** 操作数被解释为有符号整数。

循环的第一次迭代接收初始化为 **lowerBound** 值的归纳变量，以及初始化为 **initValues** 值的循环携带值。

循环体为范围内的每个值执行，接收一个整数归纳变量，该变量在每次迭代中增加 **step**，以及循环传递的值，这些值对应于前一次循环迭代产生的循环传递值。

循环在归纳变量大于或等于 **upperBound** 时终止。默认情况下，在 **upperBound** 和归纳变量之间使用有符号比较。若要改用无符号比较，请指定可选的 **unsigned** 单元属性。

循环体必须以一个 `cuda_tile.continue` 结束，该操作为每个循环携带的变量生成下一次迭代的值。

**for** 操作针对每个循环携带变量生成一个返回值。第 *i* 个返回值的类型与第 *i* 个循环携带变量的类型相同，其值为第 *i* 个循环携带变量的最终值。

Warning

- 循环携带的变量不能是 `tensor_view` 或视图类型。
- **for** 操作无法提前终止，必须以 `cuda_tile.continue` 结束。
- 当栈分配自动释放时，操作必须定义作用域。
- **lowerBound**、**upperBound** 和 **step** 必须具有相同的形状和元素类型 (tile<any>)。
- **initValues** 和 **resultValues** 必须具有相同的形状和元素类型 (Variadic<Any>)。
- 该操作必须提供自定义的解析和打印方法。
- 该操作有且仅在该区域的操作有效时才有效。
- 每个提供的区域必须恰好包含一个区块。

#### 示例

```
%lowerBound = constant <i32: 0> : tile<i32>
%upperBound = constant <i32: 10> : tile<i32>
%step = constant <i32: 1> : tile<i32>
// A simple loop iterating over an i32 range.
for %iv in (%lowerBound to %upperBound, step %step) : tile<i32> {
  continue
}
%initVal0 = constant <f32: 0.0> : tile<f32>
// A similar loop to the above, but with a loop carried value, val0.
%results = for %iv in (%lowerBound to %upperBound, step %step) : tile<i32>
  iter_values(%val00 = %initVal0) -> (tile<f32>) {
    %loopVal0 = constant <f32: 1.0> : tile<f32>
    continue %loopVal0 : tile<f32>
  }
```

完整示例列表请参见 `cuda_tile.for_0`。

### 8.5.5 cuda\_tile.if

*Conditional execution*

```
cuda_tile.if %condition
```

#### 参数

- **condition** ( `tile<i1>`) - if 操作的条件。 13.1
- **results** ( 可变参数<任意类型>) - if 操作的结果。 13.1

`if` 操作表示一个 if-then-else 结构。

`if` 操作包含 (1) 一个控制操作数，它是一个 `tile<i1>` 类型的值，(2) 一个真分支 `thenRegion`，以及 (3) 一个可选的假分支 `elseRegion`。

`if` 操作可以通过在每个分支中使用 `cuda_tile.yield` 产生值来生成结果。

如果产生值，则产生的值的类型必须匹配，并且 `if` 操作的结果类型将与产生的值的类型相同。

如果生成值，则需要 `else` 分支并且它也必须生成一个值。

返回的值将取决于选择哪个分支。

Warning

`if` 操作目前有一组额外的限制：

- `if` 的结果不能是 `tensor_view` 或视图类型。
- All regions must have zero arguments.
- 该操作必须提供自定义的解析和打印方法。
- 该操作仅在区域操作有效时才会有效。
- 每个提供的区域必须恰好包含一个区块。

#### 示例

```
%condition = constant <i1: 1> : tile<i1>
// A simple if operation that conditionally executes a region.
if %condition {
// ...
}
// An if operation with an "else" branch.
if %condition {
// ...
} else {
// ...
}
// An if operation that returns mixed types (f32,i32)
%x, %y = if %condition -> (tile<f32>, tile<i32>) {
%x_then = constant <f32: 1.0> : tile<f32>
%y_then = constant <i32: 2> : tile<i32>
yield %x_then, %y_then : tile<f32>, tile<i32>
} else {
%x_then = constant <f32: 1.0> : tile<f32>
%y_then = constant <i32: 42> : tile<i32>
yield %x_then, %y_then : tile<f32>, tile<i32>
}
```

请参阅cuda\_tile.if\_0获取完整示例列表。

### 8.5.6 cuda\_tile.loop

*Loop until a break operation*

```
cuda_tile.loop %initValues
```

#### 参数

- **initValues** ( Variadic<Any>) - 循环的初始值。 13.1
- **resultValues** ( 可变参数<任意类型>) - 循环的结果值。 13.1

**loop** 操作表示一个非结构化的无限循环，该循环会一直执行，直到遇到 `cuda_tile.break`。

循环包含 (1) 一组循环传递值，这些值由 **initValues** 初始化，并在每次循环迭代中更新。

(2) a region which represents the loop body.

循环将执行循环体，直到动态执行了`cuda_tile.break`。

循环的每个控制路径都必须以以下方式终止：

- 一个 `cuda_tile.continue`，它为每个循环携带的变量生成下一次迭代的值。
- 一个终止循环并产生最终循环携带值的 `cuda_tile.break`。

只要每个循环迭代由这些操作之一终止，它们就可以与其他控制流操作结合，以表达不同的控制流模式。

循环操作为每个循环携带变量产生一个返回值。第 *i* 个返回值的类型与第 *i* 个循环携带变量的类型相同，其值为第 *i* 个循环携带变量的最终值。

**Warning**

循环操作目前有一系列额外的限制：

- 不支持从循环内部提前返回，代码生成器若希望完全结束函数执行，必须先终止循环，然后返回。
- 循环携带的变量不能是`tensor_view`或视图类型。

- 当栈分配自动释放时，操作必须定义作用域。
- 该操作必须提供自定义的解析和打印方法。
- 该操作仅在且仅在该区域的操作有效时才有效。
- 每个提供的区域必须恰好包含一个区块。

#### 示例

```
// A simple "while-do" loop.
loop {
%cond = constant <i1: 1> : tile<i1>
if %cond {
continue
}
break
}
```

请参阅 `cuda_tile.loop_0` 获取完整示例列表。

```
// A simple "do-while" loop.
loop {
//... body of the loop.
%cond = constant <i1: 1> : tile<i1>
if %cond {
continue
}
break
}
```

请参阅 `cuda_tile.loop_1` 获取完整示例列表。

```
%initValue0 = constant <f32: 0.0> : tile<f32>
// A loop that yields carried-iteration values, returning the final values.
%results = loop iter_values(%value0 = %initValue0) : tile<f32> -> tile<f32> {
%cond = constant <i1: 1> : tile<i1>
if %cond {
%loopValue0 = constant <f32: 0.0> : tile<f32>
continue %loopValue0 : tile<f32>
}
break %value0 : tile<f32>
}
```

请参阅 `cuda_tile.loop_2` 获取完整示例列表。

```
%initValue0 = constant <i32: 0> : tile<i32>
// A loop that uses loop-carried values and returns a different type.
%results = loop iter_values(%value0 = %initValue0) : tile<i32> -> tile<f32> {
%cond = constant <i1: 1> : tile<i1>
if %cond {
%newLoopValue = constant <i32: 0> : tile<i32>
continue %newLoopValue : tile<i32>
}
%finalReturnValue = constant <f32: 0.0> : tile<f32>
break %finalReturnValue : tile<f32>
}
```

请参阅 `cuda_tile.loop_3` 查看完整示例列表。

### 8.5.7 `cuda_tile.return`

*Return value(s) from a function*

```
cuda_tile.return %operands
```

## 参数

- **操作数**（可变参数<任意类型>）- 要返回的值。 13.1

No results.

**描述** `return` 操作将控制权返回给函数的调用者。

Warning

目前 `return` 实现了受限的返回语义，具体包括：

- `cuda_tile.entry` 操作不产生返回值，因此可以通过不带操作数调用该操作来使用 `return` 终止内核的执行
- `return` 不能直接在循环体内部用于终止内核的执行
- 该操作必须终止其父基本块。

### 示例

```
experimental$func @foo() -> (tile<i32>, tile<f16>) {  
  %0 = constant <i32: 0> : tile<i32>  
  %1 = constant <f16: 0.0> : tile<f16>  
  // ...  
  return %0, %1 : tile<i32>, tile<f16>  
}
```

完整示例代码请参见 `cuda_tile.return_0`。

```
entry @foo() {  
  %0 = constant <i32: 0> : tile<i32>  
  %1 = constant <f16: 0.0> : tile<f16>  
  // ...  
  return  
}
```

请参阅 `cuda_tile.return_1` 获取完整的示例列表。

## 8.5.8 cuda\_tile.yield

*Yield a value from the block*

```
cuda_tile.yield %operands
```

## 参数

- **操作数**（可变数量<任意类型>）- 要传递给父操作的操作数。 13.1

No results.

**描述** `yield` 操作会终止一个必须将控制权交还给父操作（如 `if`、`scan`、`reduce`）的代码块。

该操作在终止时可以向父操作产生任意数量的 `$operands`。产生的值的数量以及它们如何产生的执行语义由父操作决定。

Note

与标准的 MLIR 控制流方言不同，`yield` 不用于循环控制流，有关循环控制流，请参阅 `cuda_tile.break` 和 `cuda_tile.continue`。

## 约束条件

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可以推测性地执行，且无副作用。
- 该操作是纯粹的，不执行任何内存副作用。
- 该操作必须终止其父基本块。

### 示例

```
%condition = constant <i1: true> : tile<i1>
// Yield from the body of an if conditional.
if %condition {
yield
}
// Yield values from within an if conditional.
%x, %y = if %condition -> (tile<f32>, tile<f32>) {
%x_then = constant <f32: 0.0> : tile<f32>
%y_then = constant <f32: 1.0> : tile<f32>
yield %x_then, %y_then : tile<f32>, tile<f32>
} else {
%x_else = constant <f32: 2.0> : tile<f32>
%y_else = constant <f32: 3.0> : tile<f32>
yield %x_else, %y_else : tile<f32>, tile<f32>
}
```

请参阅 `cuda_tile.yield_0` 获取完整示例列表。

## 8.6 内存

**Tile IR** 包含一组内存操作，支持加载、存储和操作内存。

**Tile IR** 中有几类内存操作：

- 基于指针的内存操作，例如 `cuda_tile.load_ptr_tko` 和 `cuda_tile.store_ptr_tko`，用于从全局内存加载图块以及将图块存储到全局内存。
- 基于视图的内存操作，例如 `cuda_tile.load_view_tko` 和 `cuda_tile.store_view_tko`，它们分别从视图加载图块以及将图块存储到视图。
- 原子内存操作，例如 `cuda_tile.atomic_rmw_tko` 和 `cuda_tile.atomic_cas_tko`，它们对全局内存执行原子操作。

目前所有内存操作都是令牌有序的；任意一对内存操作之间的顺序是未定义的，除非通过令牌连接。有关令牌有序操作的更多讨论，请参阅内存模型。

### Warning

读取或写入任何分配内存的边界都是未定义行为。越界访问的示例如下：

- 对包含分配范围外元素的图块进行指针内存操作，例如偏移超出分配的末尾。
- 将一个无效的布局与基指针关联，该布局描述了一个跨越分配范围并随后索引到视图中的步幅或形状。
- 使用越界的索引对视图进行索引。

### Note

当使用填充视图或掩码时，关于构成越界的规则会被修改，有关特定类型的更多信息，请参阅类型系统。

### 8.6.1 `cuda_tile.join_tokens`

根据输入令牌生成一个新的令牌

```
cuda_tile.join_tokens %tokens
```

## 参数

- **tokens** ( 可变参数<令牌>) - 要连接的输入令牌。 [13.1]
- **result** ( token ) - 合并后的令牌。 [13.1]

`join_tokens` 操作会生成一个新的令牌，该令牌依赖于所有输入令牌。  
消耗新令牌的令牌排序操作将相对于所有已加入的令牌进行排序。

## 约束条件

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯的，不执行任何内存副作用。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 8.6.2 `cuda_tile.load_ptr_tko`

使用指针图块从全局内存中加载和收集数据，不保证顺序

```
cuda_tile.load_ptr_tko %memory_ordering_semantics %memory_scope %source %mask %paddingValue %token %optimization_hints
```

## 参数

- **memory\_ordering\_semantics** ( MemoryOrderingSemantics ) - 加载操作的内存排序语义。 [13.1]
- **memory\_scope** ( MemoryScope ) - 原子操作的内存作用域。 [13.1]
- **source** ( ptr ) - 指针的源图块。 [13.1]
- **mask** ( tile<i1>) - 加载操作的掩码。 [13.1]
- **paddingValue** ( tile<i1|i8|i16|i32|i64|f16|bf16|f32|f64|fp8e4m3fn|fp8e5m2|tf32>) - 加载操作的填充值。 [13.1]
- **token** ( token ) - 用于加载操作的令牌。 [13.1]
- **optimization\_hints** ( OptimizationHints ) - 操作 [13.1] 的优化提示
- **result** ( tile ) - 加载操作的结果。 [13.1]
- **result\_token** ( token ) - 加载操作的结果令牌。 [13.1]

这个 `load` 操作通过从全局内存加载一个数据块到结果块中，基于 `source` 操作数提供的指针块执行收集操作。

`source` 操作数是一个指针图块，它指定了从中收集数据的内存位置。该操作会加载这些数据，并将其作为 `result` 图块返回。当加载 `i1` 类型的值时，每个值都是从内存中的一个完整字节加载的。任何非零字节会被规范化为 `0x01`，而零字节则变为 `0x00`。

可选地，可以提供一个 `mask` 操作数以控制元素的收集。如果存在，则仅加载由 `mask` 指定的元素。

`mask` 的形状必须与 `result` 的形状匹配。

当存在 `mask` 时，也可以选择性地提供一个 `paddingValue`。

`paddingValue` 必须与 `source` 图块具有相同的形状。如果未提供，被遮罩元素的值将是未定义的。

令牌排序操作不受程序顺序的约束。

编译器可能会重新排序它们（即在程序顺序中提前或延后放置），除非受到标记的进一步约束。

`memory_ordering_semantics` 属性指定了不同线程间内存访问的并发假设，这控制了所需的同步。

例如，`weak` 排序允许编译器假设对任何被访问的位置都没有并发访问。

更多信息，请参阅规范中的内存模型章节。

- `weak` - 对源/目标位置没有并发访问。
- `relaxed` - 可能存在对该位置的同时访问，但这种访问不会建立 `happens-before` 关系。
- `acquire` - 可能存在对该位置的同时访问。如果此获取操作观察到一个释放操作，那么先发生于关系即被建立。

注意：此操作不支持以下变体：`release`、`acq_rel`。

`memory_scope` 属性指定了内存操作的通信范围。

当与系统中的其他并发线程通信时，范围必须足够广泛，以涵盖参与通信的所有其他线程，否则可能发生数据竞争。

- `tl_blk` - 同一瓦片块内可能存在并发访问。
- `device` - 同一设备（例如 GPU）内部可能存在并发访问。
- `sys` - 系统中任何位置（即所有设备）都可能存在并发访问。

`optimization_hints` 属性以嵌套字典的形式提供特定于架构的编译器提示。

提示是针对每个架构（例如，`sm_100`、`sm_120`）指定的，并且对于每个架构，用户可以为每个操作指定特定的提示。

- `num_cta_in_cga` - 为 `cuda_tile.entry` 建议 CGA 中的 CTA 数量（必须是小于或等于 16 的 2 的幂）。
- `allow_tma` - 建议是否为 `cuda_tile.load_view_tko` 和 `cuda_tile.store_view_tko` 使用 TMA。
- `latency` - 为 `cuda_tile.load_view_tko` 和 `cuda_tile.store_view_tko` 提供的延迟提示。

For example they can be annotated as:

```
optimization_hints=<
sm_100 = {num_cta_in_cga = 8},
sm_120 = {num_cta_in_cga = 16}
>
```

## 约束条件

- 操作必须将可变参数操作数段大小编码在属性中。
- 源类型应为结果类型的指针类型
- ‘`mask`’ 的形状必须与 ‘`source`’ 的形状匹配
- ‘`paddingValue`’ 的类型必须与 ‘`result`’ 的类型匹配



#### 示例

```
%mask = constant <i1: 1> : tile<i1>
%padding = constant <f32: 0.0> : tile<f32>
// Load without token.
%result0, %res_token0 = load_ptr_tko weak %ptr, %mask, %padding
: tile<ptr<f32>>, tile<i1>, tile<f32> -> tile<f32>, token
// Load with token.
%token0 = make_token : token
%result1, %res_token1 = load_ptr_tko weak %ptr, %mask, %padding token=%
    token0
: tile<ptr<f32>>, tile<i1>, tile<f32> -> tile<f32>, token
return
```

完整示例列表请参见 `cuda_tile.load_ptr_tko_0`。

### 8.6.3 `cuda_tile.make_token`

*Create a fresh token with no prior dependencies*

```
cuda_tile.make_token
```

**参数** No parameters.

#### 结果

- **result** ( token ) - 一个全新的令牌，没有任何先前的依赖关系。 13.1

`make_token` 操作创建一个没有先前依赖的新令牌。

#### 约束

- 该操作基于特定的操作数和属性有条件地可推测。
- 该操作可能会被推测性地执行，且不会产生副作用。
- 该操作是纯粹的，不执行任何内存副作用。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 8.6.4 `cuda_tile.store_ptr_tko`

从瓦片指针存储和分散数据到全局内存，无顺序保证

```
cuda_tile.store_ptr_tko %memory_ordering_semantics %memory_scope %destination %
    value %mask %token %optimization_hints
```

#### 参数

- **memory\_ordering\_semantics** ( MemoryOrderingSemantics ) - 内存排序语义。 13.1
- **memory\_scope** ( MemoryScope ) - 可选的内存作用域。 13.1
- **destination** ( ptr ) - 目标指针图块。 13.1
- **value** ( tile ) - 要存储的值 tile。 13.1
- **mask** ( tile<i1> ) - 用于选择性存储的可选掩码。 13.1
- **token** ( token ) - 用于操作排序的可选令牌。 13.1
- **optimization\_hints** ( OptimizationHints ) - 操作 13.1 的优化提示

- `result_token ( token )` - 用于同步的结果令牌。 13.1

`store` 操作通过将瓦片中的数据块存储到全局内存中执行分散操作。

`destination` 操作数是一个指针图块，指示来自 `value` 图块的数据将存储到的全局内存位置。当存储 `il` 值时，每个值在内存中占据一个完整的字节。任何非零字节被规范化为 `0x01`，零字节变为 `0x00`。

此外，该操作支持一个可选的 `mask` 操作数，它允许选择性分散元素。如果提供了 `mask`，则仅存储由 `mask` 指定的元素。`mask` 的形状必须与 `value` 图块的形状对齐。

`memory_ordering_semantics` 属性指定了不同线程间内存访问的并发假设，这控制了所需的同步。

例如，`weak` 排序允许编译器假设对任何被访问的位置都没有并发访问。

更多信息，请参阅规范中的内存模型章节。

- `weak` - 对源/目标位置没有并发访问。
- `relaxed` - 可能存在对该位置的同时访问，但这种访问不会建立 `happens-before` 关系。
- `release` - 可能同时存在对该位置的并发访问。如果此释放操作与获取操作被共同观测到，则建立 *happens before* 关系。

注意：以下变体不受此操作支持：`acquire`、`acq_rel`。

`memory_scope` 属性为内存操作指定了通信范围。

在与系统中的其他并发线程通信时，作用域必须足够广泛，以涵盖所有参与通信的其他线程，否则可能发生数据竞争。

- `tl_blk` - 同一瓦片块内可能存在并发访问。
- `device` - 同一设备（例如 GPU）内部可能存在并发访问。
- `sys` - 系统中任何位置（即所有设备）都可能存在并发访问。

`optimization_hints` 属性以嵌套字典的形式提供特定于架构的编译器提示。

提示是针对每个架构（例如，`sm_100`、`sm_120`）指定的，并且对于每个架构，用户可以为每个操作指定特定的提示。

- `num_cta_in_cga` - 为 `cuda_tile.entry` 建议 CGA 中的 CTA 数量（必须是小于或等于 16 的 2 的幂）。
- `allow_tma` - 建议是否对 `cuda_tile.load_view_tko` 和 `cuda_tile.store_view_tko` 使用 TMA。
- `latency` - 为 `cuda_tile.load_view_tko` 和 `cuda_tile.store_view_tko` 提供的延迟提示。

For example they can be annotated as:

```
optimization_hints=<
sm_100 = {num_cta_in_cga = 8},
sm_120 = {num_cta_in_cga = 16}
>
```

## 约束条件

- 操作必须将可变操作数段大小编码在属性中。
- 目标类型应为值类型的指针类型
- ‘`destination`’ 的形状必须与 ‘`mask`’ 的形状相匹配
- 操作的结果类型可以从其操作数和属性中推断出来。

## 8.7 浮点数

**Tile IR** 包含一组类型化的算术运算，这些运算实现了浮点类型上常见的算术操作，关于整数运算请参见 整数运算。

所有操作均以针对目标架构和设备系列高效的方式实现。在大多数常见情况下，这意味着利用底层硬件的原生浮点运算。由于 **Tile IR** 的稳定性保证和更高级的编程模型，某些硬件上的某些类型可能被模拟，有关稳定性保证的更多信息以及各设备行为的信息，请参阅稳定性。

### 8.7.1 浮点运算

### 8.7.2 浮点运算

**Tile IR** 包含一组标准数学库操作，这些操作实现了在支持 16 位、32 位和 64 位浮点数据类型的张量上常见的数学函数。

Note

32 位和 64 位运算通常利用高效的硬件特定指令。一些 16 位运算通过更宽的中间计算进行模拟，可能无法提供相同的性能。

Warning

基于数据类型支持存在一些限制，这些限制在类型系统章节中有详细说明。

### 8.7.3 `cuda_tile.absf`

*Element-wise floating-point absolute value*

```
cuda_tile.absf %source
```

#### 参数

- **source** ( `tile<f16|bf16|f32|f64>` ) - 输入的浮点类型 `tile`。 13.1
- **result** ( `tile<f16|bf16|f32|f64>` ) - 输入 `tile` 的绝对值。 13.1

`absf` 操作计算输入浮点数图块的逐元素绝对值。

$\text{absf}(x)_i = |x|_i$

逐元素浮点算术运算由目标架构的原生浮点指令执行。若指定了 `rounding` 修饰符，则特定的舍入模式将应用于结果的每个元素。更多详情请参阅浮点运算。

#### 约束条件

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可能会被推测性地执行，且无副作用。
- 该操作是纯的，不执行任何内存副作用。
- `source` 和 `result` 必须具有相同的形状。
- `source` 和 `result` 必须具有相同的形状和元素类型 ( `tile<f16|bf16|f32|f64>` )。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 8.7.4 `cuda_tile.addf`

*Element-wise floating-point addition*

```
cuda_tile.addf %lhs %rhs %rounding_mode %flush_to_zero
```

## 参数

- **lhs** ( tile<f16|bf16|f32|f64> ) - 左侧操作数。 [13.1]
- **rhs** ( tile<f16|bf16|f32|f64> ) - 右侧操作数。 [13.1]
- **rounding\_mode** ( RoundingMode ) - 该操作的舍入模式。 [13.1]
- **flush\_to\_zero** ( 标志 ) - 如果设置，将次正规输入和结果刷新为保留符号的零。 [13.1]
- **result** ( tile<f16|bf16|f32|f64> ) - 输入 tile 的总和。 [13.1]

**addf** 操作计算两个具有浮点元素类型的图块的逐元素加法。

`addf(x,y)i=xi+yi`

除非另有说明，否则单个元素的加法由目标架构针对给定元素类型的原生浮点加法执行。

逐元素浮点算术运算由目标架构的原生浮点指令执行。若指定了 **rounding** 修饰符，则特定的舍入模式将应用于结果的每个元素。更多详情请参阅浮点运算。

**rounding** 属性指定了用于此操作的舍入模式。

- **nearest\_even** - 四舍五入到最接近的值（平局时取偶数）。
- **zero** - Round towards zero (truncate).
- **negative\_inf** - 向负无穷方向舍入。
- **positive\_inf** - 向正无穷方向舍入。
- **approx** - Approximate rounding mode.
- **full** - Full precision rounding mode.
- **nearest\_int\_to\_zero** - 向零方向四舍五入到最近的整数。

下表展示了每种数据类型支持的修饰符和舍入模式。带有 ‘\*’ 的条目在 f32 中为模拟实现。

| Modifier               | Float32 | Float64 | BFloat16 | Float16 |
|------------------------|---------|---------|----------|---------|
| —                      | —       | —       | —        | —       |
| flush_to_zero          | yes     | no      | no       | no      |
| rounding<nearest_even> | yes     | yes     | yes      | yes     |
| rounding<zero>         | yes     | yes     | yes      | yes     |
| rounding<negative_inf> | yes     | yes     | yes      | yes     |
| rounding<positive_inf> | yes     | yes     | yes      | yes     |

## 约束条件

- 该操作基于特定的操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯的，不执行任何内存副作用。
- **lhs**、**rhs** 和 **result** 必须具有相同的形状和元素类型 (tile<f16|bf16|f32|f64 >)。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 8.7.5 cuda\_tile.atan2

*Element-wise atan2*

```
cuda_tile.atan2 %x %y
```

## 参数

- **x** ( tile<f16|bf16|f32|f64>) - 输入 x 浮点 tile。13.2
- **y** ( tile<f16|bf16|f32|f64>) - 输入的 y 浮点数 tile。13.2
- **result** ( tile<f16|bf16|f32|f64>) - 逐元素结果图块。13.2

**atan2** 运算计算第一个和第二个输入参数比值  $x / y$  的反正切主值。结果的象限由输入  $x$  和  $y$  的符号决定。

$(\text{atan2}(x,y))_i = \text{atan2}(x_i, y_i)$

此操作在半精度输入 (:code:f16' 和 :code:bf16) 上执行时, 会在 :code:f32' 中进行模拟。更多详情请参阅浮点运算。

## 约束条件

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可能会被推测性地执行, 且不产生副作用。
- 该操作是纯粹的, 不会执行任何内存副作用。
- $x$ 、 $y$  和 **result** 必须具有相同的形状和元素类型 (tile<f16|bf16|f32|f64 >)。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 示例

```
%x = constant <f32: [1.0, -1.0, 0.0, 2.0]> : tile<4xf32>
%y = constant <f32: [1.0, 1.0, 1.0, 0.0]> : tile<4xf32>
%res = atan2 %x, %y : tile<4xf32>
```

完整示例列表请参见 `cuda_tile.atan2_0`。

## 8.7.6 cuda\_tile.ceil

*Element-wise ceiling*

```
cuda_tile.ceil %source
```

## 参数

- **source** ( tile<f16|bf16|f32|f64>) - 输入的浮点型 tile。13.1
- **result** ( tile<f16|bf16|f32|f64>) - 输入 tile 的上限值。13.1

**ceil** 操作对输入的浮点数张量进行逐元素向上取整运算。向上取整操作将每个元素向上舍入到大于或等于输入值的最大整数值。

$\text{ceil}(x)_i = \min\{n \in \mathbb{Z} \mid n \geq x_i\}$

## 约束条件

- 该操作基于特定操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯粹的，不执行任何内存副作用。
- `source` 与 `result` 必须具有相同的形状和元素类型 (`tile<f16|bf16|f32|f64>`)。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 示例

```
%result = ceil %source : tile<f32>
```

请参阅 `cuda_tile.ceil_0` 获取完整示例列表。

## 8.7.7 `cuda_tile.cosh`

*Element-wise hyperbolic cosine*

```
cuda_tile.cosh %source
```

## 参数

- `source` (`tile<f16|bf16|f32|f64>`) - 输入的浮点瓦片。 13.1
- `result` (`tile<f16|bf16|f32|f64>`) - 输入 `tile` 的双曲余弦值。 13.1

`cosh` 操作计算输入张量中每个元素的浮点双曲余弦值。

`cosh(x)i=coshxi`

此操作在半精度输入 (`:code:f16` 和 `:code:bf16`) 上执行时，会在 `:code:f32` 中进行模拟。更多详情请参阅浮点运算。

## 约束条件

- 该操作根据特定的操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯的，不执行任何内存副作用。
- `source` 和 `result` 必须具有相同的形状和元素类型 (`tile<f16|bf16|f32|f64>`)。
- 操作的结果类型可以从其操作数和属性中推断出来。

## 8.7.8 `cuda_tile.cos`

*Element-wise cosine*

```
cuda_tile.cos %source
```

## 参数

- `source` (`tile<f16|bf16|f32|f64>`) - 输入的浮点型 `tile`。 13.1

- **result** ( tile<f16|bf16|f32|f64> ) - 输入 tile 的余弦值。 13.1

**cos** 操作计算输入浮点数张量的逐元素余弦值。

`cos(x)i=cos(xi)`

此操作在半精度输入 ( :code:f16‘ 和 :code:bf16 ) 上执行时, 会在 :code:f32‘ 中进行模拟。更多详情请参阅浮点运算。

### 约束条件

- 该操作根据特定的操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯粹的, 不执行任何内存副作用。
- **source** 与 **result** 必须具有相同的形状和元素类型 (tile<f16|bf16|f32|f64 > )。
- 操作的结果类型可以从其操作数和属性中推断出来。

#### 示例

```
%in = constant <f32: [0.0, 1.0, 2.0, 3.0]> : tile<4xf32>

%res = cos %in : tile<4xf32>
```

请参阅 `cuda_tile.cos_0` 查看完整的示例列表。

### 8.7.9 cuda\_tile.divf

*Element-wise floating-point division*

```
cuda_tile.divf %lhs %rhs %rounding_mode %flush_to_zero
```

#### 参数

- **lhs** ( tile<f16|bf16|f32|f64> ) - 作为被除数的输入浮点 tile。 13.1
- **rhs** (tile<f16|bf16|f32|f64 > ) - 作为除数的输入浮点 tile。 13.1
- **rounding\_mode** ( RoundingMode ) - 该操作的舍入模式。 13.1
- **flush\_to\_zero** ( 标志 ) - 若设置, 将次正规输入和结果刷新为保留符号的零。 13.1
- **result** ( tile<f16|bf16|f32|f64> ) - **divf** 操作的结果。 13.1

**divf** 操作计算两个具有浮点元素类型的输入图块的逐元素除法。

**approx** 舍入模式实现了除法的快速近似, 通过乘以倒数来计算。对于归一化范围  $[2^{-(126)}, 2^{(126)}]$  内的  $|rhs|$ , 最大 ULP (最后一位单位) 误差为 2。

对于  $2^{(126)} < |rhs| < 2^{(128)}$ , 如果 **lhs** 为无穷大, 则操作返回 NaN, 否则返回 0。

**full** 舍入模式实现了一种相对快速、全范围的近似方法, 它通过缩放操作数来提高精度, 但并非完全

符合 IEEE 754 标准。在全部输入范围内, 最大 ulp 误差为 2。

`div(lhs, rhs)i=lhsi/rhsi`

逐元素浮点算术运算由目标架构的原生浮点指令执行。若指定了 **rounding** 修饰符, 则特定的舍入模式将应用于结果的每个元素。更多详情请参阅浮点运算。

**rounding** 属性指定了用于此操作的舍入模式。

- `nearest_even` - 四舍五入到最接近的值（平局时取偶数）。
- `zero` - Round towards zero (truncate).
- `negative_inf` - 向负无穷方向舍入。
- `positive_inf` - 向正无穷方向舍入。
- `approx` - Approximate rounding mode.
- `full` - Full precision rounding mode.
- `nearest_int_to_zero` - 向零方向舍入到最接近的整数。

下表显示了每种数据类型支持的修饰符和舍入模式。带有“\*”的条目在 `f32` 中为模拟实现。

| Modifier                                  | Float32 | Float64 | BFloat16 | Float16 |
|-------------------------------------------|---------|---------|----------|---------|
| <code>flush_to_zero</code>                | yes     | no      | no       | no      |
| <code>approx</code>                       | yes     | no      | no       | no      |
| <code>full</code>                         | yes     | no      | no       | no      |
| <code>rounding&lt;nearest_even&gt;</code> | yes     | yes     | yes      | yes     |
| <code>rounding&lt;zero&gt;</code>         | yes     | yes     | yes      | yes     |
| <code>rounding&lt;negative_inf&gt;</code> | yes     | yes     | yes      | yes     |
| <code>rounding&lt;positive_inf&gt;</code> | yes     | yes     | yes      | yes     |

## 约束条件

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯的，不执行任何内存副作用。
- `lhs`、`rhs` 和 `result` 必须具有相同的形状和元素类型 (`tile<f16|bf16|f32|f64>`)。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 8.7.10 `cuda_tile.exp2`

*Element-wise power of two*

```
cuda_tile.exp2 %source %flush_to_zero
```

## 参数

- `source` (`tile<f16|bf16|f32|f64>`) - 输入的浮点型 `tile`。 [13.1](#)
- `flush_to_zero` (标志) - 若设置，则将次正规输入和结果刷新为保留符号的零。 [13.1](#)
- `result` (`tile<f16|bf16|f32|f64>`) - 将 2 提升到输入 `tile` 的幂次方所得的结果。 [13.1](#)

`exp2` 操作计算输入浮点图块中每个元素的二的幂。

`exp2(x)i = 2xi`

此操作在半精度输入 (`:code:f16` 和 `:code:bf16`) 上执行时，会在 `:code:f32` 中进行模拟。更多详情请参阅浮点运算。

下表显示了每种数据类型支持的修饰符。

| Modifier                   | Float32 | Float64 | BFloat16 | Float16 |
|----------------------------|---------|---------|----------|---------|
| <code>flush_to_zero</code> | yes     | no      | no       | no      |



## 约束条件

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯的，不执行任何内存副作用。
- `source` 和 `result` 必须具有相同的形状和元素类型 (`tile<f16|bf16|f32|f64 >`)。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 示例

```
%in = constant <f32: [0.0, 1.0, 2.0, 3.0]> : tile<4xf32>
%res = exp2 %in : tile<4xf32>
```

完整示例列表请参见 `cuda_tile.exp2_0`。

## 8.7.11 cuda\_tile.exp

*Element-wise exponential*

```
cuda_tile.exp %source
```

## 参数

- `source` (`tile<f16|bf16|f32|f64>`) - 输入的浮点型 `tile`。 13.1
- `result` (`tile<f16|bf16|f32|f64>`) - 输入 `tile` 的指数。 13.1

`exp` 操作计算输入浮点数图块的逐元素指数。

`exp(x)i=exi`

此操作在半精度输入 (`:code:f16` 和 `:code:bf16`) 上执行时，会在 `:code:f32` 中进行模拟。更多详情请参阅浮点运算。

## 约束条件

- 该操作根据特定的操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯粹的，不执行任何内存副作用。
- `source` 和 `result` 必须具有相同的形状和元素类型 (`tile<f16|bf16|f32|f64 >`)。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 示例

```
%in = constant <f32: [0.0, 1.0, 2.0, 3.0]> : tile<4xf32>
%res = exp %in : tile<4xf32>
```

完整示例列表请参见 `cuda_tile.exp_0`。

### 8.7.12 cuda\_tile.floor

*Element-wise floor rounding*

```
cuda_tile.floor %source
```

#### 参数

- **source** ( tile<f16|bf16|f32|f64> ) - 向下取整操作的输入瓦片。 13.1
- **result** ( tile<f16|bf16|f32|f64> ) - 向下取整操作的结果。 13.1

**floor** 操作对输入的浮点型数据块进行逐元素向下取整，将每个元素向下舍入至小于或等于该元素的最大整数。

$$\text{floor}(x_i) = \max\{n \in \mathbb{Z} \mid n \leq x_i\}$$

逐元素浮点算术运算由目标架构的原生浮点指令执行。若指定了 **rounding** 修饰符，则特定的舍入模式将应用于结果的每个元素。更多详情请参阅浮点运算。

#### 约束条件

- 该操作根据特定的操作数和属性有条件地可推测。
- 该操作可能会被推测性地执行，且不产生副作用。
- 该操作是纯粹的，不会产生任何内存副作用。
- **source** 与 **result** 必须具有相同的形状和元素类型 (tile<f16|bf16|f32|f64> )。
- 操作的结果类型可以从其操作数和属性中推断出来。

#### 示例

```
%source = constant <f32: 1.5> : tile<f32>
%result = floor %source : tile<f32>
```

请参阅cuda\_tile.floor\_0查看完整示例列表。

### 8.7.13 cuda\_tile.fma

*Floating point fused multiply-add*

```
cuda_tile.fma %lhs %rhs %acc %rounding_mode %flush_to_zero
```

#### 参数

- **lhs** ( tile<f16|bf16|f32|f64> ) - 左侧操作数。 13.1
- **rhs** ( tile<f16|bf16|f32|f64> ) - 右侧操作数。 13.1
- **acc** ( tile<f16|bf16|f32|f64> ) - 累加器操作数。 13.1
- **rounding\_mode** ( RoundingMode ) - 操作的舍入模式。 13.1
- **flush\_to\_zero** ( 标志 ) - 若设置，将次正规输入和结果刷新为保留符号的零。 13.1

- **result** ( tile<f16|bf16|f32|f64>) - 输入 tile 的融合乘加运算结果。 13.1

接受三个操作数 lhs、rhs 和 acc，返回 `result = lhs * rhs + acc`。

`fma(x,y,z)i=xi×yi+zi`

`rounding` 属性指定了用于此操作的舍入模式。

- `nearest_even` - 向最接近的偶数舍入（平局时取偶）。
- `zero` - Round towards zero (truncate).
- `negative_inf` - 向负无穷方向舍入。
- `positive_inf` - 向正无穷方向舍入。
- `approx` - Approximate rounding mode.
- `full` - Full precision rounding mode.
- `nearest_int_to_zero` - 向零方向舍入到最接近的整数。

下表显示了每种数据类型支持的修饰符和舍入模式。带有 ‘\*’ 的条目在 f32 中模拟。

| Modifier                                  | Float32 | Float64 | BFloat16 | Float16 |
|-------------------------------------------|---------|---------|----------|---------|
| —                                         | —       | —       | —        | —       |
| <code>flush_to_zero</code>                | yes     | no      | no       | no      |
| <code>rounding&lt;nearest_even&gt;</code> | yes     | yes     | yes      | yes     |
| <code>rounding&lt;zero&gt;</code>         | yes     | yes     | yes      | yes     |
| <code>rounding&lt;negative_inf&gt;</code> | yes     | yes     | yes      | yes     |
| <code>rounding&lt;positive_inf&gt;</code> | yes     | yes     | yes      | yes     |

## 约束条件

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可以无副作用地推测性执行。
- 该操作是纯的，不执行任何内存副作用。
- `lhs`、`rhs`、`acc` 和 `result` 必须具有相同的形状和元素类型 (tile<f16|bf16|f32|f64 >)。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 8.7.14 cuda\_tile.log2

*Element-wise base-2 logarithm*

```
cuda_tile.log2 %source
```

## 参数

- **source** ( tile<f16|bf16|f32|f64>) - 输入的浮点瓦片。 13.1
- **result** ( tile<f16|bf16|f32|f64>) - log2 运算的结果。 13.1

`log2` 操作计算浮点数瓦片的逐元素以 2 为底的对数。

$\log_2(x)_i = \log_2(x_i)$

此操作在半精度输入（`:code:f16'` 和 `:code:bf16`）上执行时，会在 `:code:f32'` 中进行模拟。更多详情请参阅浮点运算。

### 约束条件

- 该操作基于特定操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯粹的，不会产生任何内存副作用。
- `source` 和 `result` 必须具有相同的形状和元素类型（`tile<f16|bf16|f32|f64 >`）。
- 操作的结果类型可以从其操作数和属性中推断出来。

#### 示例

```
%in = constant <f32: [0.0, 1.0, 2.0, 3.0]> : tile<4xf32>
%res = log2 %in : tile<4xf32>
```

完整示例列表请参见 `cuda_tile.log2_0`。

### 8.7.15 `cuda_tile.log`

*Element-wise natural logarithm*

```
cuda_tile.log %source
```

### 参数

- `source` ( `tile<f16|bf16|f32|f64>`) - 输入的浮点类型 `tile`。 13.1
- `result` ( `tile<f16|bf16|f32|f64>`) - 对数运算的结果。 13.1

`log` 操作计算浮点数张量的逐元素自然对数。

$\log(x)_i = \ln(x_i)$

此操作在半精度输入（`:code:f16'` 和 `:code:bf16`）上执行时，会在 `:code:f32'` 中进行模拟。更多详情请参阅浮点运算。

### 约束条件

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可能会被推测性地执行，且不会产生副作用。
- 该操作是纯粹的，不执行任何内存副作用。
- `source` 和 `result` 必须具有相同的形状和元素类型（`tile<f16|bf16|f32|f64 >`）。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 8.7.16 cuda\_tile.maxf

*Element-wise floating-point maximum*

```
cuda_tile.maxf %lhs %rhs %propagate_nan %flush_to_zero
```

#### 参数

- **lhs** ( tile<f16|bf16|f32|f64> ) - 左侧操作数。 13.1
- **rhs** ( tile<f16|bf16|f32|f64> ) - 右侧操作数。 13.1
- **propagate\_nan** (标志) — 当设置此标志时, 如果两个被比较的元素中任意一个是 NaN, 则 **maxf** (或 **minf**) 会返回 NaN。 13.1
- **flush\_to\_zero** (标志) - 如果设置, 将次正规输入和结果刷新为保留符号的零。 13.1
- **result** ( tile<f16|bf16|f32|f64> ) - **maxf** 操作的结果。 13.1

**maxf** 操作计算两个具有浮点元素类型的输入图块的逐元素最大值。

**propagate\_nan** 控制 **maxf** 如何解释 NaN。如果设置了 **propagate\_nan** 修饰符, 则当被比较的元素中任意一个是 NaN 时, **maxf** 返回一个规范的 NaN (IEEE 754-2019 的最大值)。而如果未设置 **propagate\_nan** 修饰符, 则 **maxf** 仅在两个元素都是 NaN 时才返回一个规范的 NaN; 否则, 它返回非 NaN 元素 (IEEE 754-2019's maximumNumber)。

如果两个元素都不是 NaN, **maxf** 将返回输入值中较大的那个。+0.0 被认为大于 -0.0。

如果指定了 **flush\_to\_zero** 修饰符, 非正规数将被刷新为保留符号的零。**flush\_to\_zero** 修饰符仅适用于 f32 数据类型。

$\max_i(x,y) = \begin{cases} x & \text{if } x \geq y \\ y & \text{if } x < y \end{cases}$

#### 约束条件

- 该操作根据特定的操作数和属性有条件地可推测。
- 该操作可以无副作用地推测性执行。
- 该操作是纯的, 不执行任何内存副作用。
- **lhs**、**rhs** 和 **result** 必须具有相同的形状和元素类型 (tile<f16|bf16|f32|f64 >)。
- 操作的结果类型可以从其操作数和属性中推断出来。

#### 示例

```
// Create tensor view from a pointer to global memory
%0 = make_tensor_view %arg0, shape = [2, 4], strides = [4, 1] : tensor_view
    <2x4xf32, strides=[4,1]>

%1 = make_tensor_view %arg1, shape = [2, 4], strides = [4, 1] : tensor_view
    <2x4xf32, strides=[4,1]>

// Convert tensor views to partition views and load tiles from partition
views.

%p0 = make_partition_view %0 : partition_view<tile=(2x4), tensor_view<2
    x4xf32, strides=[4,1]>>
```

```

%p1 = make_partition_view %1 : partition_view<tile=(2x4), tensor_view<2
      x4xf32, strides=[4,1]>>

%c0 = constant <i32: 0> : tile<i32>

%2, %token0 = load_view_tko weak %p0[%c0, %c0] : partition_view<tile=(2x4),
      tensor_view<2x4xf32, strides=[4,1]>>, tile<i32> -> tile<2x4xf32>, token

%3, %token1 = load_view_tko weak %p1[%c0, %c0] : partition_view<tile=(2x4),
      tensor_view<2x4xf32, strides=[4,1]>>, tile<i32> -> tile<2x4xf32>, token

// IEEE 754-2019's maximum

%4 = maxf %2, %3 propagate_nan : tile<2x4xf32>

// IEEE 754-2019's maximumNumber

%5 = maxf %2, %3 : tile<2x4xf32>

// flush denormal to positive zero

%6 = maxf %2, %3 flush_to_zero : tile<2x4xf32>

```

请参阅 `cuda_tile.maxf_0` 查看完整示例列表。

### 8.7.17 `cuda_tile.minf`

*Element-wise floating-point minimum*

```
cuda_tile.minf %lhs %rhs %propagate_nan %flush_to_zero
```

#### 参数

- **lhs** ( `tile<f16|bf16|f32|f64>`) - 左侧操作数。 13.1
- **rhs** ( `tile<f16|bf16|f32|f64>`) - 右侧操作数。 13.1
- **propagate\_nan** (标志) - 当设置此标志时, 如果两个被比较元素中任意一个是 NaN, 则 `maxf` (或 `minf`) 会返回一个 NaN。 13.1
- **flush\_to\_zero** (标志) - 如果设置, 会将次正规输入和结果刷新为保留符号的零。 13.1
- **result** ( `tile<f16|bf16|f32|f64>`) - 输入图块的最小值。 13.1

`minf` 操作计算两个具有浮点元素类型的输入图块的逐元素最小值。

`propagate_nan` 控制着 `minf` 如何解释 NaN。如果设置了 `propagate_nan` 修饰符, 当被比较的元素中任意一个是 NaN 时, `minf` 会返回一个规范的 NaN (遵循 IEEE 754-2019 的最小值规则)。而如果未设置 `propagate_nan` 修饰符, `minf` 仅在两个元素都是 NaN 时才返回一个规范的 NaN; 否则, 它会返回那个非 NaN 的元素 (遵循 IEEE 754-2019's minimumNumber)。

如果两个元素都不是 NaN, `minf` 将返回输入中的最小值。-0.0 被视为小于 +0.0。

如果指定了 `flush_to_zero` 修饰符, 非规范数将被刷新为保留符号的零。`flush_to_zero` 修饰符仅适用于 f32 数据类型。

$\text{minf}(x,y)_i = \begin{cases} x_i & \text{if } x_i \leq y_i \\ y_i & \text{if } x_i > y_i \end{cases}$

## 约束条件

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯的，不执行任何内存副作用。
- lhs、rhs 和 result 必须具有相同的形状和元素类型 (tile<f16|bf16|f32|f64 >)。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 示例

```
// Create tensor view from a pointer to global memory

%0 = make_tensor_view %arg0, shape = [2, 4], strides = [4, 1] : tensor_view
    <2x4xf32, strides=[4,1]>

%1 = make_tensor_view %arg1, shape = [2, 4], strides = [4, 1] : tensor_view
    <2x4xf32, strides=[4,1]>

// Convert tensor views to partition views and load tiles from partition
views.

%p0 = make_partition_view %0 : partition_view<tile=(2x4), tensor_view<2
    x4xf32, strides=[4,1]>>

%p1 = make_partition_view %1 : partition_view<tile=(2x4), tensor_view<2
    x4xf32, strides=[4,1]>>

%c0 = constant <i32: 0> : tile<i32>

%2, %token0 = load_view_tko weak %p0[%c0, %c0] : partition_view<tile=(2x4),
    tensor_view<2x4xf32, strides=[4,1]>>, tile<i32> -> tile<2x4xf32>, token

%3, %token1 = load_view_tko weak %p1[%c0, %c0] : partition_view<tile=(2x4),
    tensor_view<2x4xf32, strides=[4,1]>>, tile<i32> -> tile<2x4xf32>, token

// IEEE 754-2019's minimum

%4 = minf %2, %3 propagate_nan : tile<2x4xf32>

// IEEE 754-2019's minimumNumber

%5 = minf %2, %3 : tile<2x4xf32>

// flush denormal to positive zero

%6 = minf %2, %3 flush_to_zero : tile<2x4xf32>
```

完整示例列表请参见 `cuda_tile.minf_0`。

## 8.7.18 cuda\_tile.mulf

*Element-wise floating-point multiplication*

```
cuda_tile.mulf %lhs %rhs %rounding_mode %flush_to_zero
```

## 参数

- **lhs** ( tile<f16|bf16|f32|f64> ) - 左侧操作数。 [13.1]
- **rhs** ( tile<f16|bf16|f32|f64> ) - 右侧操作数。 [13.1]
- **rounding\_mode** ( RoundingMode ) - 该操作的舍入模式。 [13.1]
- **flush\_to\_zero** ( 标志 ) - 如果设置，将次正规输入和结果刷新为保留符号的零。 [13.1]
- **result** ( tile<f16|bf16|f32|f64> ) - 输入 tile 的乘积。 [13.1]

**mulf** 操作计算两个输入瓦片（元素类型为浮点数）之间的逐元素乘积。

如果指定了 **flush\_to\_zero** 修饰符，非正规数将被刷新为正零。

如果指定了 **rounding** 修饰符，则会对结果的每个元素应用特定的舍入模式。

$\text{mulf}(x,y)_i = x_i \times y_i$

逐元素的浮点算术运算由目标架构的原生浮点指令执行。如果指定了 **rounding** 修饰符，则特定的舍入模式将应用于结果的每个元素。更多详情请参阅浮点运算。

**rounding** 属性指定了操作中使用的舍入模式。

- **nearest\_even** - 四舍五入到最接近的值（平局时取偶数）。
- **zero** - Round towards zero (truncate).
- **negative\_inf** - 向负无穷方向舍入。
- **positive\_inf** - 向正无穷方向舍入。
- **approx** - Approximate rounding mode.
- **full** - Full precision rounding mode.
- **nearest\_int\_to\_zero** - 向零方向四舍五入到最近的整数。

下表展示了每种数据类型支持的修饰符和舍入模式。带有“\*”的条目在 f32 中为模拟实现。

| Modifier                            | Float32 | Float64 | BFloat16 | Float16 |
|-------------------------------------|---------|---------|----------|---------|
|                                     | —       | —       | —        | —       |
| <b>flush_to_zero</b>                | yes     | no      | no       | no      |
| <b>rounding&lt;nearest_even&gt;</b> | yes     | yes     | yes      | yes     |
| <b>rounding&lt;zero&gt;</b>         | yes     | yes     | yes      | yes     |
| <b>rounding&lt;negative_inf&gt;</b> | yes     | yes     | yes      | yes     |
| <b>rounding&lt;positive_inf&gt;</b> | yes     | yes     | yes      | yes     |

## 约束条件

- 该操作根据特定的操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯粹的，不会产生任何内存副作用。
- **lhs**、**rhs** 和 **result** 必须具有相同的形状和元素类型 (tile<f16|bf16|f32|f64>)。
- 操作的结果类型可以从其操作数和属性中推断出来。



### 8.7.19 cuda\_tile.negf

*Element-wise floating-point negation*

```
cuda_tile.negf %source
```

#### 参数

- **source** ( tile<f16|bf16|f32|f64>) - 输入图块。 13.1
- **result** ( tile<f16|bf16|f32|f64>) - 取反后的浮点类型 tile。 13.1

**negf** 是一个逐元素操作，用于对 **source** 的符号取反。

$\text{negf}(x) = -x$

逐元素浮点算术运算由目标架构的原生浮点指令执行。若指定了 **rounding** 修饰符，则特定的舍入模式将应用于结果的每个元素。更多详情请参阅浮点运算。

#### 约束条件

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可能会被推测性地执行，且没有副作用。
- 该操作是纯粹的，不会产生任何内存副作用。
- **source** 和 **result** 必须具有相同的形状和元素类型 (tile<f16|bf16|f32|f64>)。
- 操作的结果类型可以从其操作数和属性中推断出来。

#### 示例

```
%source = constant <f32: 0.0> : tile<4xf32>
%result = negf %source : tile<4xf32>
```

请参阅 `cuda_tile.negf_0` 获取完整的示例列表。

### 8.7.20 cuda\_tile.pow

*Element-wise floating-point exponentiation*

```
cuda_tile.pow %source %exponent
```

#### 参数

- **source** ( tile<f16|bf16|f32|f64>) - 基础 tile。 13.1
- **exponent** ( tile<f16|bf16|f32|f64>) - 指数 tile。 13.1
- **result** ( tile<f16|bf16|f32|f64>) - pow 操作的结果。 13.1

**pow** 操作计算源浮点图块逐元素取幂，以指数浮点图块为幂。

$\text{pow}(x,y)_i = x_i y_i$

逐元素的浮点算术运算由目标架构的原生浮点指令执行。如果指定了 **rounding** 修饰符，则特定的舍入模式将应用于结果的每个元素。更多详情请参阅浮点运算。

### 约束条件

- 该操作根据特定的操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯粹的，不会产生任何内存副作用。
- **result**、**source** 和 **exponent** 必须具有相同的形状和元素类型 (tile<f16|bf16|f32|f64 >)。
- **source**、**exponent** 和 **result** 必须具有相同的秩。
- 操作的结果类型可以从其操作数和属性中推断出来。

#### 示例

```
%source = constant <f32: 0.0> : tile<4xf32>
%exponent = constant <f32: 2.0> : tile<4xf32>
%result = pow %source, %exponent : tile<4xf32>
```

完整示例列表请参见 `cuda_tile.pow_0`。

### 8.7.21 cuda\_tile.remf

*Element-wise floating-point remainder*

```
cuda_tile.remf %lhs %rhs
```

#### 参数

- **lhs** ( tile<f16|bf16|f32|f64>) - 左侧操作数。 13.1
- **rhs** ( tile<f16|bf16|f32|f64>) - 右侧操作数。 13.1
- **result** ( tile<f16|bf16|f32|f64>) - 除法后的余数。 13.1

**remf** 操作使用截断除法（向零取整）计算逐元素的浮点余数。

$\text{remf}(x,y)_i = x_i - \text{trunc}(x_i/y_i) \times y_i$

结果与被除数 (**lhs**) 具有相同的符号，且其绝对值小于除数 (**rhs**) 的绝对值。

**Special cases:**

- If **y** is zero, returns **NaN**
- 如果 **x** 是无穷大且 **y** 是有限值，则返回 **NaN**
- 如果 **x** 是有限值且 **y** 是无限值，返回 **x**
- If either argument is **NaN** , returns **NaN**

逐元素的浮点算术运算由目标架构的原生浮点指令执行。如果指定了 **rounding** 修饰符，特定的舍入模式将应用于结果的每个元素。更多详情请参阅浮点运算。

## 约束条件

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可以推测性地执行，且不产生副作用。
- 该操作是纯粹的，不执行任何内存副作用。
- `lhs`、`rhs` 和 `result` 必须具有相同的形状和元素类型 (`tile<f16|bf16|f32|f64>`)。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 8.7.22 `cuda_tile.rsqrt`

*Element-wise reciprocal square root*

```
cuda_tile.rsqrt %source %flush_to_zero
```

## 参数

- `source` (`tile<f16|bf16|f32|f64>`) - 用于计算其倒数平方根的输入瓦片。 13.1
- `flush_to_zero` (标志) - 如果设置，将次正规输入和结果刷新为保留符号的零。 13.1
- `result` (`tile<f16|bf16|f32|f64>`) - 输入 `tile` 的平方根倒数。 13.1

`rsqrt` 操作计算输入浮点数分量的逐元素倒数平方根。

此操作支持: `flush_to_zero`: 若由用户设置，将把次正规输入和结果刷新为保留符号的零。

`rsqrt(x)`<sub>*i=1xi*</sub>

此操作在半精度输入 (`:code:f16` 和 `:code:bf16`) 上执行时，会在 `:code:f32` 中进行模拟。更多详情请参阅浮点运算。

下表显示了每种数据类型支持的修饰符。

| Modifier                   | Float32 | Float64 |
|----------------------------|---------|---------|
| —                          | —       | —       |
| <code>flush_to_zero</code> | yes     | no      |

## 约束条件

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可以无副作用地推测性执行。
- 该操作是纯粹的，不会产生任何内存副作用。
- `source` 与 `result` 必须具有相同的形状和元素类型 (`tile<f16|bf16|f32|f64>`)。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 示例

```
%in = constant <f32: [0.0, 1.0, 2.0, 3.0]> : tile<4xf32>

%res = rsqrt %in : tile<4xf32>

// Rsqrt op with flush to zero modifier

%ftz_res = rsqrt %in flush_to_zero : tile<4xf32>
```

请参阅 `cuda_tile.rsqrt_0` 获取完整的示例列表。

### 8.7.23 cuda\_tile.sinh

*Element-wise hyperbolic sine*

```
cuda_tile.sinh %source
```

#### 参数

- **source** ( tile<f16|bf16|f32|f64> ) - 输入的浮点型 tile。 13.1
- **result** ( tile<f16|bf16|f32|f64> ) - 输入 tile 的双曲正弦值。 13.1

**sinh** 操作计算输入浮点数张量的逐元素双曲正弦值。

$\text{sinh}(x)_i = \text{sinh}(x_i)$

此操作在输入为半精度 ( :code:f16‘ 和 :code:bf16 ) 时, 会在 :code:f32‘ 中进行模拟执行。更多详情请参阅浮点运算。

#### 约束条件

- 该操作根据特定的操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯粹的, 不会产生任何内存副作用。
- **source** 和 **result** 必须具有相同的形状和元素类型 ( tile<f16|bf16|f32|f64 > )。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 8.7.24 cuda\_tile.sin

*Element-wise sine*

```
cuda_tile.sin %source
```

#### 参数

- **source** ( tile<f16|bf16|f32|f64> ) - 输入的浮点类型 tile。 13.1
- **result** ( tile<f16|bf16|f32|f64> ) - 输入 tile 的正弦值。 13.1

**sin** 操作计算输入浮点数图块的逐元素正弦值。

$\text{sin}(x)_i = \text{sin}(x_i)$

此操作在半精度输入 ( :code:f16‘ 和 :code:bf16 ) 上执行时, 会在 :code:f32‘ 中进行模拟。更多详情请参阅浮点运算。

## 约束条件

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯粹的，不执行任何内存副作用。
- `source` 与 `result` 必须具有相同的形状和元素类型 (`tile<f16|bf16|f32|f64>`)。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 示例

```
%in = constant <f32: [0.0, 1.0, 2.0, 3.0]> : tile<4xf32>
%res = sin %in : tile<4xf32>
```

完整示例列表请参见 `cuda_tile.sin_0`。

## 8.7.25 `cuda_tile.sqrt`

*Element-wise square root*

```
cuda_tile.sqrt %source %rounding_mode %flush_to_zero
```

### 参数

- `source` (`tile<f16|bf16|f32|f64>`) - 要计算其平方根的输入 `tile`。 13.1
- `rounding_mode` (`RoundingMode`) - 该运算的舍入模式。 13.1
- `flush_to_zero` (标志) - 如果设置，将次正规输入和结果刷新为保留符号的零。 13.1
- `result` (`tile<f16|bf16|f32|f64>`) - 输入 `tile` 的平方根。 13.1

`sqrt` 操作计算浮点数张量的逐元素平方根。

`sqrt(x)i=xi`

`rounding` 属性指定了用于此操作的舍入模式。

- `nearest_even` - 四舍五入到最接近的值（平局时取偶数）。
- `zero` - Round towards zero (truncate).
- `negative_inf` - 向负无穷方向舍入。
- `positive_inf` - 向正无穷方向舍入。
- `approx` - Approximate rounding mode.
- `full` - Full precision rounding mode.
- `nearest_int_to_zero` - 向零方向舍入到最接近的整数。

下表显示了每种数据类型支持的修饰符和舍入模式。带有 ‘\*’ 的条目在 f32 中模拟。

| Modifier               | Float32 | Float64 | BFloat16 | Float16 |
|------------------------|---------|---------|----------|---------|
| —                      | —       | —       | —        | —       |
| flush_to_zero          | yes     | no      | no       | no      |
| approx                 | yes     | no      | no       | no      |
| rounding<nearest_even> | yes     | yes     | yes      | yes     |
| rounding<zero>         | yes     | yes     | yes      | yes     |
| rounding<negative_inf> | yes     | yes     | yes      | yes     |
| rounding<positive_inf> | yes     | yes     | yes      | yes     |

## 约束条件

- 该操作根据特定的操作数和属性有条件地可推测。
- 该操作可以在没有副作用的情况下被推测性执行。
- 该操作是纯粹的，不会产生任何内存副作用。
- `source` 和 `result` 必须具有相同的形状和元素类型 (`tile<f16|bf16|f32|f64>`)。
- 运算的结果类型可以从其操作数和属性中推断出来。

### 8.7.26 cuda\_tile.subf

*Element-wise floating-point subtraction*

```
cuda_tile.subf %lhs %rhs %rounding_mode %flush_to_zero
```

## 参数

- `lhs` (`tile<f16|bf16|f32|f64>`) - 左侧操作数。 [13.1](#)
- `rhs` (`tile<f16|bf16|f32|f64>`) - 右侧操作数。 [13.1](#)
- `rounding_mode` (`RoundingMode`) - 该操作的舍入模式。 [13.1](#)
- `flush_to_zero` (标志) - 如果设置，将次正规输入和结果刷新为保留符号的零。 [13.1](#)
- `result` (`tile<f16|bf16|f32|f64>`) - 减法运算的结果。 [13.1](#)

`subf` 操作计算输入浮点图块的逐元素减法。

`subf(x,y)i=xi-yi`

逐元素的浮点算术运算由目标架构的原生浮点指令执行。如果指定了 `rounding` 修饰符，特定的舍入模式将应用于结果的每个元素。更多详情请参阅浮点运算。

`rounding` 属性指定了用于此操作的舍入模式。

- `nearest_even` - 四舍五入到最接近的值（平局时取偶数）。
- `zero` - Round towards zero (truncate).
- `negative_inf` - 向负无穷方向舍入。
- `positive_inf` - 向正无穷方向舍入。
- `approx` - Approximate rounding mode.
- `full` - Full precision rounding mode.

- `nearest_int_to_zero` - 向零方向舍入到最接近的整数。

下表显示了每种数据类型支持的修饰符和舍入模式。带有 ‘\*’ 的条目在 `f32` 中为模拟实现。

| Modifier                                  | Float32 | Float64 | BFloat16 | Float16 |
|-------------------------------------------|---------|---------|----------|---------|
| <code>flush_to_zero</code>                | yes     | no      | no       | no      |
| <code>rounding&lt;nearest_even&gt;</code> | yes     | yes     | yes      | yes     |
| <code>rounding&lt;zero&gt;</code>         | yes     | yes     | yes      | yes     |
| <code>rounding&lt;negative_inf&gt;</code> | yes     | yes     | yes      | yes     |
| <code>rounding&lt;positive_inf&gt;</code> | yes     | yes     | yes      | yes     |

## 约束条件

- 该操作基于特定操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯粹的，不会产生任何内存副作用。
- `lhs`、`rhs` 和 `result` 必须具有相同的形状和元素类型 (`tile<f16|bf16|f32|f64>`)。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 8.7.27 `cuda_tile.tanh`

*Element-wise hyperbolic tangent*

```
cuda_tile.tanh %source %rounding_mode
```

## 参数

- `source` (`tile<f16|bf16|f32|f64>`) - 输入的浮点类型 `tile`。13.1
- `rounding_mode` (`RoundingMode`) - 该操作的舍入模式。13.2
- `result` (`tile<f16|bf16|f32|f64>`) - 输入浮点 `tile` 的双曲正切值。13.1

`tanh` 运算计算输入浮点张量的逐元素双曲正切值。默认舍入模式为完全舍入。

`approx` 舍入模式实现了双曲正切函数的快速近似。

此快速近似的次正规结果不会被刷新为零。

`full` 舍入模式实现了一种相对快速的全范围近似。

最大 `ulp` 误差在 `FP32` 的整个输入范围内为 2，在 `FP64` 中为 1。

$\tanh(x)_i = \tanh(x_i)$

此操作在半精度输入 (`:code:f16` 和 `:code:bf16`) 上执行时，会在 `:code:f32` 中进行模拟。更多详情请参阅浮点运算。

`rounding` 属性指定了用于此操作的舍入模式。

- `nearest_even` - 四舍五入到最接近的值（平局时取偶数）。
- `zero` - Round towards zero (truncate).
- `negative_inf` - 向负无穷方向舍入。
- `positive_inf` - 向正无穷方向舍入。
- `approx` - Approximate rounding mode.

- **full** - Full precision rounding mode.
- **nearest\_int\_to\_zero** - 向零方向舍入到最接近的整数。

下表显示了每种数据类型支持的修饰符。带有“\*”的条目在 f32 中进行了模拟。

| Modifier      | Float32 | Float64 | BFloat16 | Float16 |
|---------------|---------|---------|----------|---------|
| —             | —       | —       | —        | —       |
| <b>approx</b> | yes     | no      | no       | no      |
| <b>full</b>   | yes     | yes     | yes      | yes     |

## 约束条件

- 该操作根据特定的操作数和属性有条件地可推测。
- 该操作可以在没有副作用的情况下被推测性地执行。
- 该操作是纯粹的，不执行任何内存副作用。
- **source** 和 **result** 必须具有相同的形状和元素类型 (tile<f16|bf16|f32|f64 >)。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 示例

```
%in = constant <f32: [0.0, 1.0, 2.0, 3.0]> : tile<4xf32>

%res0 = tanh %in : tile<4xf32>

// tanh with approx modifier

%res1 = tanh %in rounding<approx> : tile<4xf32>
```

请参阅 `cuda_tile.tanh_0` 查看完整的示例列表。

## 8.7.28 cuda\_tile.tan

*Element-wise tangent*

```
cuda_tile.tan %source
```

## 参数

- **source** ( tile<f16|bf16|f32|f64>) - 输入的浮点型 tile。 13.1
- **result** ( tile<f16|bf16|f32|f64>) - 输入浮点 tile 的正切值。 13.1

**tan** 操作计算输入浮点数分量的逐元素正切值。

$\tan(x)_i = \tan(x_i)$

此操作在半精度输入 (:code:f16' 和 :code:bf16) 上执行时，会在 :code:f32' 中进行模拟。更多详情请参阅浮点运算。



## 约束条件

- 该操作基于特定的操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯粹的，不执行任何内存副作用。
- `source` 和 `result` 必须具有相同的形状和元素类型 (`tile<f16|bf16|f32|f64>`)。
- 操作的结果类型可以从其操作数和属性中推断出来。

## 8.8 整数

**Tile IR** 包含一组类型化的算术运算，这些运算实现了对整数瓦片的常见算术操作，有关浮点运算请参阅浮点运算。

所有操作均以针对目标架构和设备系列高效的方式实现。在大多数常见情况下，这意味着利用底层硬件的原生浮点运算。由于 **Tile IR** 的稳定性保证和更高级的编程模型，某些硬件上的某些类型可能被模拟，有关稳定性保证的更多信息以及各设备行为的信息，请参阅稳定性。

### 8.8.1 整数运算

**Tile IR** 中的整数类型是无符号的，但重要的是，这与无符号整数并不相同。我们以二进制补码形式存储所有整数，并根据需要支持带符号或无符号标志的运算操作。这种设计使我们在 IR 层面无需区分有符号和无符号整数类型，并将符号信息保持在运算操作的局部范围内。

对于 `i1` 类型，无符号操作将值视为 0/1，而有符号操作将值视为 0/-1，所有 `i1` 值都被规范化为 0x00（假）或 0x01（真）

for consistent LSB-only semantics.

### 8.8.2 `cuda__tile.absi`

*Element-wise integer absolute value*

```
cuda__tile.absi %source
```

## 参数

- `source` (`tile<i1|i8|i16|i32|i64>`) - 输入的整数 `tile`。 13.1
- `result` (`tile<i1|i8|i16|i32|i64>`) - 输入 `tile` 的绝对值。 13.1

`absi` 操作计算输入整数图块的绝对值。

输入瓦片始终被解释为有符号整数。

输出图块始终被解释为无符号整数。

$\text{absi}(x) = |x|$

逐元素整数算术运算由目标架构的原生整数指令执行。默认语义在溢出或下溢时采用环绕语义。更多详情请参阅整数运算。

## 约束条件

- 该操作根据特定的操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯粹的，不执行任何内存副作用。
- `source` 和 `result` 必须具有相同的形状。
- `source` 和 `result` 必须具有相同的形状和元素类型 (`tile<i1|i8|i16|i32|i64>`)。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 8.8.3 cuda\_tile.addi

*Element-wise integer addition*

```
cuda_tile.addi %lhs %rhs %overflow
```

#### 参数

- **lhs** ( tile<i1|i8|i16|i32|i64> ) - 左侧操作数。 13.1
- **rhs** ( tile<i1|i8|i16|i32|i64> ) - 右侧操作数。 13.1
- **overflow** ( IntegerOverflow ) - 操作的溢出行为。 13.1
- **result** ( tile<i1|i8|i16|i32|i64> ) - 输入图块的总和。 13.1

**addi** 操作计算两个具有整数元素类型的图块的逐元素加法。

$\text{addi}(x,y) \text{ i} = x_i + y_i$

逐元素整数算术运算由目标架构的原生整数指令执行。默认语义为溢出或下溢时的环绕语义。更多详情请参阅整数运算。

**overflow** 属性用于指示编译器如何推理特定操作的溢出行为。

这些属性作为编译器可能用来推理操作的假设。代码生成器的责任是确保在执行过程中动态地遵守这些假设。

- **none** - 编译器对溢出行为不做任何假设。
- **no\_signed\_wrap** - 编译器假定在将有符号整数作为操作数解释时不会发生溢出（环绕）。
- **no\_unsigned\_wrap** - 编译器假定在将操作数解释为无符号整数时不会发生溢出（环绕）。
- **no\_wrap** - 编译器假定在将操作数解释为有符号或无符号整数时不会发生溢出（环绕）。

如果在运行时发生溢出，尽管溢出的值表明不会发生，其行为是未定义的。

#### 约束条件

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可能会被推测性地执行，且不会产生副作用。
- 该操作是纯粹的，不执行任何内存副作用。
- **lhs**、**rhs** 和 **result** 必须具有相同的形状和元素类型 (tile<i1|i8|i16|i32|i64 >)。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 8.8.4 cuda\_tile.divi

*Element-wise integer division*

```
cuda_tile.divi %lhs %rhs %signedness %rounding
```

## 参数

- **lhs** ( tile<i1|i8|i16|i32|i64> ) - 左侧操作数。 13.1
- **rhs** ( tile<i1|i8|i16|i32|i64> ) - 右侧操作数。 13.1
- **有符号性** (Signedness) - 将整数解释为 有符号或 无符号 13.1
- **rounding** ( RoundingMode ) - 设置舍入方向（实现向下取整除法/向上取整除法）。 13.1
- **result** ( tile<i1|i8|i16|i32|i64> ) - 除法的结果。 13.1

**divi** 操作计算两个具有整数元素类型的瓦片值的逐元素除法。

默认的舍入方式是向零舍入。舍入模式可以设置为 `positive_inf`（“向上取整除法”）或 `negative_inf`（“向下取整除法”），其他值是非法的。

使用负无穷舍入标志与无符号数结合不是一个有效的组合。

如果提供了无符号标志，操作数将被视为无符号整数，否则它们将被视为有符号整数。

如果右侧为零，则行为未定义。有符号除法溢出（最小值除以-1）是未定义行为。

`div(lhs, rhs)i=lhsi/rhsi`

逐元素整数算术运算由目标架构的原生整数指令执行。默认语义是在溢出或下溢时采用环绕语义。更多详情请参阅整数运算。

**signedness** 属性指定操作数的有符号性。

- **unsigned** - 将操作数视为无符号整数。
- **signed** - 将操作数视为有符号整数。

**rounding** 属性指定了用于此操作的舍入模式。

- **nearest\_even** - 四舍五入到最接近的值（平局时取偶数）。
- **zero** - Round towards zero (truncate).
- **negative\_inf** - 向负无穷方向舍入。
- **positive\_inf** - 向正无穷方向舍入。
- **approx** - Approximate rounding mode.
- **full** - Full precision rounding mode.
- **nearest\_int\_to\_zero** - 向零方向四舍五入到最接近的整数。
- 该操作是纯函数，不会产生任何内存副作用。
- **lhs**、**rhs** 和 **result** 必须具有相同的形状和元素类型 (tile<i1|i8|i16|i32|i64 >)。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 8.8.5 cuda\_tile.maxi

*Element-wise integer maximum*

```
cuda_tile.maxi %lhs %rhs %signedness
```

## 参数

- **lhs** ( tile<i1|i8|i16|i32|i64>) - 左侧操作数。 13.1
- **rhs** ( tile<i1|i8|i16|i32|i64>) - 右侧操作数。 13.1
- **有符号性** (Signedness) - 将整数解释为 有符号或 无符号 13.1
- **result** ( tile<i1|i8|i16|i32|i64>) - maxi 操作的结果。 13.1

maxi 操作计算两个输入整数 tile 之间的逐元素最大值。

$\text{maxi}(x,y) = \begin{cases} x & \text{if } x \geq y \\ y & \text{if } x < y \end{cases}$

逐元素整数算术运算由目标架构的原生整数指令执行。默认语义是在溢出或下溢时采用环绕语义。更多详情请参阅整数。

**signedness** 属性指定操作数的有符号性。

- **unsigned** - 将操作数视为无符号整数。
- **signed** - 将操作数视为有符号整数。
- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可以无副作用地推测性执行。
- 该操作是纯粹的，不会产生任何内存副作用。
- **lhs**、**rhs** 和 **result** 必须具有相同的形状和元素类型 (tile<i1|i8|i16|i32|i64 >)。
- 操作的结果类型可以从其操作数和属性中推断出来。

## 示例

```
// Create tensor view from a pointer to global memory

%0 = make_tensor_view %arg0, shape = [2, 4], strides = [4, 1] : tensor_view
    <2x4xi32, strides=[4,1]>

%1 = make_tensor_view %arg1, shape = [2, 4], strides = [4, 1] : tensor_view
    <2x4xi32, strides=[4,1]>

// Convert tensor views to partition views and load tiles from them.

%p0 = make_partition_view %0 : partition_view<tile=(2x4), tensor_view<2
    x4xi32, strides=[4,1]>>

%p1 = make_partition_view %1 : partition_view<tile=(2x4), tensor_view<2
    x4xi32, strides=[4,1]>>

%c0 = constant <i32: 0> : tile<i32>

%2, %token0 = load_view_tko weak %p0[%c0, %c0] : partition_view<tile=(2x4),
    tensor_view<2x4xi32, strides=[4,1]>>, tile<i32> -> tile<2x4xi32>, token

%3, %token1 = load_view_tko weak %p1[%c0, %c0] : partition_view<tile=(2x4),
    tensor_view<2x4xi32, strides=[4,1]>>, tile<i32> -> tile<2x4xi32>, token

// Signless i32 treated as unsigned
```

```
%4 = maxi %2, %3 unsigned : tile<2x4xi32>

// Signless i32 treated as signed

%5 = maxi %2, %3 signed : tile<2x4xi32>
```

完整示例列表请参见 `cuda_tile.maxi_0`。

### 8.8.6 `cuda_tile.mini`

*Element-wise integer minimum*

```
cuda_tile.mini %lhs %rhs %signedness
```

#### 参数

- **lhs** ( `tile<i1|i8|i16|i32|i64>`) - 左侧操作数。 13.1
- **rhs** ( `tile<i1|i8|i16|i32|i64>`) - 右侧操作数。 13.1
- **有符号性** (Signedness) - 将整数解释为 有符号或 无符号 13.1
- **result** ( `tile<i1|i8|i16|i32|i64>`) - 输入图块的最小值。 13.1

**mini** 操作计算两个输入图块之间逐元素的最小值，输入图块具有整数元素类型。

$\text{mini}(x,y)_i = \begin{cases} x_i & \text{if } x_i \leq y_i \\ y_i & \text{if } x_i > y_i \end{cases}$

逐元素整数算术运算由目标架构的原生整数指令执行。默认语义是溢出或下溢时的环绕语义。更多详情请参阅整数。

**signedness** 属性指定操作数的有符号性。

- **unsigned** - 将操作数视为无符号整数。
- **signed** - 将操作数视为有符号整数。
- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯的，不会产生任何内存副作用。
- **lhs**、**rhs** 和 **result** 必须具有相同的形状和元素类型 (`tile<i1|i8|i16|i32|i64>`)。
- 操作的结果类型可以从其操作数和属性中推断出来。

## 示例

```
// Create tensor view from a pointer to global memory

%0 = make_tensor_view %arg0, shape = [2, 4], strides = [4, 1] : tensor_view
    <2x4xi32, strides=[4,1]>

%1 = make_tensor_view %arg1, shape = [2, 4], strides = [4, 1] : tensor_view
    <2x4xi32, strides=[4,1]>

// Convert tensor views to partition views and load tiles from partition
views.

%p0 = make_partition_view %0 : partition_view<tile=(2x4), tensor_view<2
    x4xi32, strides=[4,1]>>

%p1 = make_partition_view %1 : partition_view<tile=(2x4), tensor_view<2
    x4xi32, strides=[4,1]>>

%c0 = constant <i32: 0> : tile<i32>

%2, %token0 = load_view_tko weak %p0[%c0, %c0] : partition_view<tile=(2x4),
    tensor_view<2x4xi32, strides=[4,1]>>, tile<i32> -> tile<2x4xi32>, token

%3, %token1 = load_view_tko weak %p1[%c0, %c0] : partition_view<tile=(2x4),
    tensor_view<2x4xi32, strides=[4,1]>>, tile<i32> -> tile<2x4xi32>, token

// Signless i32 treated as unsigned

%4 = mini %2, %3 unsigned : tile<2x4xi32>

// Signless i32 treated as signed

%5 = mini %2, %3 signed : tile<2x4xi32>
```

完整示例列表请参见 `cuda_tile.mini_0`。

### 8.8.7 `cuda_tile.muli`

*Element-wise integer multiplication*

```
cuda_tile.muli %lhs %rhs %overflow
```

#### 参数

- **lhs** ( `tile<i1|i8|i16|i32|i64>` ) - 左侧输入的整数 `tile`。 13.1
- **rhs** ( `tile<i1|i8|i16|i32|i64>` ) - 右侧输入的整数图块。 13.1
- **overflow** ( `IntegerOverflow` ) - 操作的溢出行为。 13.1
- **result** ( `tile<i1|i8|i16|i32|i64>` ) - 输入 `tile` 的乘积。 13.1

`muli` 操作计算两个具有整数元素类型的输入图块之间的逐元素乘积。

`muli(x,y)i=xi×yi`

逐元素整数算术运算由目标架构的原生整数指令执行。默认语义是在上溢或下溢时采用环绕语义。更多详情请参阅整数运算。

`overflow` 属性用于指示编译器如何推理特定操作的溢出行为。

这些属性作为编译器可能用来推理操作的假设。代码生成器的责任是确保操作在执行过程中动态地遵守这些假设。

- `none` - 编译器对溢出行为不做任何假设。
- `no_signed_wrap` - 编译器假设在将操作数解释为有符号整数时不会发生溢出（回绕）。
- `no_unsigned_wrap` - 编译器假设在将操作数解释为无符号整数时不会发生溢出（回绕）。
- `no_wrap` - 编译器在将操作数解释为有符号或无符号整数时，假定不会发生溢出（环绕）。

如果在运行时发生溢出，尽管溢出的值表明不会发生，其行为是未定义的。

## 约束条件

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可能会被推测性地执行，且不会产生副作用。
- 该操作是纯粹的，不会产生任何内存副作用。
- `lhs`、`rhs` 和 `result` 必须具有相同的形状和元素类型（`tile<i1|i8|i16|i32|i64>`）。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 8.8.8 `cuda_tile.mulhii`

整数乘法的高位逐元素运算

```
cuda_tile.mulhii %x %y
```

## 参数

- `x` ( `tile<i1|i8|i16|i32|i64>` ) - 左侧输入的整数 `tile`。 13.1
- `y` ( `tile<i1|i8|i16|i32|i64>` ) - 右侧输入的整数 `tile`。 13.1
- `result` ( `tile<i1|i8|i16|i32|i64>` ) - 输入 `tile` 乘积的最高有效位。 13.1

`mulhii` 操作生成两个 `N` 位整数瓦片乘积（`2N` 位）的最高有效 `N` 位。对于 `i64`，这是最高有效的 64 位。完整 128 位乘积的位；对于 `i8`，它是最高有效的 8

bits of the full 16-bit product; etc.

这与 `muli` 形成对比，后者生成 `2N` 位乘积的低 `N` 位。

`mulhii` 操作仅针对无符号整数定义。

`mulhii(xi,yi)=xi×yi»bitwidth(type(xi))`

逐元素整数算术运算由目标架构的原生整数指令执行。默认语义在溢出或下溢时采用环绕语义。更多详情请参阅整数运算。

## 约束条件

- 该操作根据特定的操作数和属性有条件地可推测。
- 该操作可以在没有副作用的情况下被推测性执行。
- 该操作是纯粹的，不执行任何内存副作用。
- `x`、`y` 和 `result` 必须具有相同的形状和元素类型（`tile<i1|i8|i16|i32|i64>`）。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 示例

```
// 2^31 * 2 = 2^32, or 0x100000000.

// The most significant 32 bits of the product are 0x00000001.

// The lower 32 bits of the product are 0x00000000.

%a = constant <i32: 2147483648> : tile<i32>    // %a = 2^31

%b = constant <i32: 2> : tile<i32>            // %b = 2

%res_hi = mulhii %a, %b : tile<i32>           // %res_hi = 1

%res_lo = muli %a, %b : tile<i32>             // %res_lo = 0
```

完整示例列表请参见 `cuda_tile.mulhii_0`。

### 8.8.9 cuda\_tile.negi

*Element-wise integer negation*

```
cuda_tile.negi %source %overflow
```

#### 参数

- **source** ( tile<i1|i8|i16|i32|i64> ) - 输入的整数瓦片。 13.1
- **overflow** ( IntegerOverflow ) - 操作的溢出行为。 13.2
- **result** ( tile<i1|i8|i16|i32|i64> ) - 取反后的整数 tile。 13.1

**negi** 操作计算输入整数图块的逐元素取反。

输入和输出瓦片始终被解释为有符号整数。

`negi(xi)=-xi`

逐元素整数算术运算由目标架构的原生整数指令执行。默认语义为溢出或下溢时的环绕语义。更多详情请参阅整数。

**overflow** 属性用于指示编译器如何推理特定操作的溢出行为。

这些属性作为编译器可用于推理操作的假设。代码生成器的责任是确保操作在执行过程中动态地遵守这些假设。

- **none** - 编译器对溢出行为不做任何假设。
- **no\_signed\_wrap** - 编译器假定在将操作数解释为有符号整数时不会发生溢出（环绕）。
- **no\_unsigned\_wrap** - 编译器假定在将操作数解释为无符号整数时不会发生溢出（环绕）。
- **no\_wrap** - 编译器在将操作数解释为有符号或无符号整数时，假定不会发生溢出（环绕）。

如果在运行时发生溢出，尽管溢出的值表明不会发生，其行为是未定义的。



## 约束条件

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可以在没有副作用的情况下被推测性地执行。
- 该操作是纯粹的，不执行任何内存副作用。
- `source` 和 `result` 必须具有相同的形状和元素类型 (`tile<i1|i8|i16|i32|i64>`)。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 示例

```
%source = constant <i16: [0, 1, 2, 3]> : tile<4xi16>

%result = negi %source : tile<4xi16>

// %result = [0, -1, -2, -3]
```

请参阅 `cuda_tile.negi_0` 查看完整示例列表。

## 8.8.10 `cuda_tile.ori`

*Element-wise bitwise OR*

```
cuda_tile.ori %lhs %rhs
```

## 参数

- `lhs` (`tile<i1|i8|i16|i32|i64>`) - 左侧操作数。 [13.1](#)
- `rhs` (`tile<i1|i8|i16|i32|i64>`) - 右侧操作数。 [13.1](#)
- `result` (`tile<i1|i8|i16|i32|i64>`) - 输入 `tile` 的按位或运算结果。 [13.1](#)

`ori` 操作计算两个具有整数元素类型的图块之间的逐元素按位或。

`ori(x,y)i=xi|yi`

逐元素整数算术运算由目标架构的原生整数指令执行。默认语义为溢出或下溢时的环绕语义。更多详情请参阅整数。

## 约束条件

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯粹的，不执行任何内存副作用。
- `lhs`、`rhs` 和 `result` 必须具有相同的形状和元素类型 (`tile<i1|i8|i16|i32|i64>`)。
- 操作的结果类型可以从其操作数和属性中推断出来。

## 8.8.11 `cuda_tile.remi`

*Element-wise integer remainder*

```
cuda_tile.remi %lhs %rhs %signedness
```

## 参数

- **lhs** ( tile<i1|i8|i16|i32|i64>) - 左侧操作数。 [13.1]
- **rhs** ( tile<i1|i8|i16|i32|i64>) - 右侧操作数。 [13.1]
- **有符号性** (Signedness) - 将整数解释为 有符号或 无符号 [13.1]
- **result** ( tile<i1|i8|i16|i32|i64>) - 除法后的余数。 [13.1]

**remi** 操作使用截断除法（向零取整）计算具有整数元素类型的输入图块的逐元素余数。

Division by zero is undefined behavior.

$\text{remi}(x,y)_i = x_i - \text{trunc}(x_i/y_i) \times y_i$

如果操作是有符号的，结果的符号与被除数（**lhs**）的符号相匹配。例如：

- `remi(7, 3) = 1`
- `remi(7, -3) = 1`
- `remi(-7, 3) = -1`
- `remi(-7, -3) = -1`

逐元素整数算术运算由目标架构的原生整数指令执行。默认语义为溢出或下溢时的环绕语义。更多详情请参阅整数。

**signedness** 属性指定操作数的有符号性。

- **unsigned** - 将操作数视为无符号整数。
- **signed** - 将操作数视为有符号整数。
- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯粹的，不会产生任何内存副作用。
- **result**、**lhs** 和 **rhs** 必须具有相同的形状和元素类型 (tile<i1|i8|i16|i32|i64>)。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 8.8.12 cuda\_tile.shli

*Element-wise shift-left*

```
cuda_tile.shli %lhs %rhs %overflow
```

## 参数

- **lhs** ( tile<i1|i8|i16|i32|i64>) - 左侧操作数。 [13.1]
- **rhs** ( tile<i1|i8|i16|i32|i64>) - 右侧操作数（移位量）。 [13.1]
- **overflow** ( IntegerOverflow ) - 操作的溢出行为。 [13.1]

- **result** ( tile<i1|i8|i16|i32|i64>) - 左移操作的结果。 13.1

**shli** 操作计算 **lhs** 整数操作数按 **rhs** 操作数逐元素左移的结果。右侧的低位用零填充。

**shli**(x,y)i=xi yi

**rhs** 操作数被解释为无符号整数。

逐元素整数算术运算由目标架构的原生整数指令执行。默认语义是溢出或下溢时的环绕语义。更多详情请参阅整数。

**overflow** 属性用于指示编译器如何推理特定操作的溢出行为。

这些属性作为编译器可能用于推理操作的假设。代码生成器的责任是确保在执行过程中动态地遵守这些假设。

- **none** - 编译器对溢出行为不做任何假设。
- **no\_signed\_wrap** - 编译器假定在将操作数解释为有符号整数时不会发生溢出（环绕）。
- **no\_unsigned\_wrap** - 编译器假定在将操作数解释为无符号整数时不会发生溢出（环绕）。
- **no\_wrap** - 编译器在将操作数解释为有符号或无符号整数时，假定不会发生溢出（环绕）。

如果运行时发生溢出，尽管溢出的值表明不会发生，其行为是未定义的。

### 约束条件

- 该操作基于特定的操作数和属性有条件地可推测。
- 该操作可能会被推测性地执行，且不产生副作用。
- 该操作是纯粹的，不执行任何内存副作用。
- **lhs**、**rhs** 和 **result** 必须具有相同的形状和元素类型 (tile<i1|i8|i16|i32|i64 >)。
- 操作的结果类型可以从其操作数和属性中推断出来。

#### 8.8.13 cuda\_tile.shri

*Element-wise shift-right*

```
cuda_tile.shri %lhs %rhs %signedness
```

### 参数

- **lhs** ( tile<i1|i8|i16|i32|i64>) - 左侧操作数。 13.1
- **rhs** ( tile<i1|i8|i16|i32|i64>) - 右侧操作数（移位置量）。 13.1
- **有符号性**（有符号性） - 将整数解释为 有符号或 无符号 13.1
- **result** ( tile<i1|i8|i16|i32|i64>) - 右移操作的结果。 13.1

**shri** 操作对具有整数元素类型的瓦片，计算 lhs 整数操作数按 rhs 操作数值逐元素右移的结果。

shri(x,y)i=xi yi

当 **unsigned** 时，高位补零；当 **signed** 时，高位填充符号位。

rhs 操作数始终被解释为无符号整数。

逐元素整数算术运算由目标架构的原生整数指令执行。默认语义为溢出或下溢时的环绕语义。更多详情请参阅整数运算。

**signedness** 属性指定操作数的有符号性。

- **unsigned** - 将操作数视为无符号整数。
- **signed** - 将操作数视为有符号整数。
- 该操作根据特定的操作数和属性有条件地可推测。
- 该操作可能会被推测性地执行，且不会产生副作用。
- 该操作是纯粹的，不执行任何内存副作用。
- lhs、rhs 和 result 必须具有相同的形状和元素类型 (tile<i1|i8|i16|i32|i64>)。
- 操作的结果类型可以从其操作数和属性中推断出来。

#### 8.8.14 cuda\_tile.subi

*Element-wise integer subtraction*

```
cuda_tile.subi %lhs %rhs %overflow
```

##### 参数

- **lhs** ( tile<i1|i8|i16|i32|i64>) - 左侧操作数。 13.1
- **rhs** ( tile<i1|i8|i16|i32|i64>) - 右侧操作数。 13.1
- **overflow** ( IntegerOverflow ) - 操作的溢出行为。 13.1
- **result** ( tile<i1|i8|i16|i32|i64>) - 减法运算的结果。 13.1

**subi** 操作计算两个输入整数图块的逐元素减法。

subi(x,y)i=xi-yi

逐元素整数算术运算由目标架构的原生整数指令执行。默认语义为溢出或下溢时的环绕语义。更多详情请参阅整数运算。

**overflow** 属性用于指示编译器如何推理特定操作的溢出行为。

这些属性作为编译器可能用于推理操作的假设。代码生成器的责任是确保操作在执行过程中动态地遵守这些假设。

- **none** - 编译器对溢出行为不做任何假设。
- **no\_signed\_wrap** - 编译器假定在将操作数解释为有符号整数时不会发生溢出（回绕）。
- **no\_unsigned\_wrap** - 编译器假定在将操作数解释为无符号整数时不会发生溢出（回绕）。
- **no\_wrap** - 编译器假设在将操作数解释为有符号或无符号整数时不会发生溢出（回绕）。

如果在运行时发生溢出，尽管溢出的值表明不会发生，其行为是未定义的。

## 约束条件

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯粹的，不会产生任何内存副作用。
- `lhs`、`rhs` 和 `result` 必须具有相同的形状和元素类型（`tile<i1|i8|i16|i32|i64>`）。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 8.8.15 `cuda_tile.xori`

*Element-wise bitwise XOR*

```
cuda_tile.xori %lhs %rhs
```

## 参数

- `lhs` ( `tile<i1|i8|i16|i32|i64>`) - 左侧操作数。 13.1
- `rhs` ( `tile<i1|i8|i16|i32|i64>`) - 右侧操作数。 13.1
- `result` ( `tile<i1|i8|i16|i32|i64>`) - 输入 `tile` 的按位异或结果。 13.1

`xori` 操作计算逐元素的按位异或（XOR）

of two tile values with integer element types.

`xori(x,y)i=xi yi`

逐元素整数算术运算由目标架构的原生整数指令执行。默认语义是溢出或下溢时的环绕语义。更多详情请参阅整数运算。

## 约束条件

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可以在没有副作用的情况下被推测性地执行。
- 该操作是纯粹的，不执行任何内存副作用。
- `lhs`、`rhs` 和 `result` 必须具有相同的形状和元素类型（`tile<i1|i8|i16|i32|i64>`）。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 示例

```
%lhs = constant <i32: [0, 1, 2, 3]> : tile<4xi32>

%rhs = constant <i32: [4, 5, 6, 7]> : tile<4xi32>

// This computes the bitwise XOR of each element in `lhs` and `rhs`,
// which
// are tiles of shape `4xi32`, and returns the result as `result`.

%result = xori %lhs, %rhs : tile<4xi32>
```

完整示例列表请参见 `cuda_tile.xori_0`。

## 8.9 位运算

### 8.9.1 cuda\_tile.andi

*Element-wise bitwise logical AND*

```
cuda_tile.andi %lhs %rhs
```

#### 参数

- **lhs** (tile<i1|i8|i16|i32|i64 >) - 左侧操作数。 13.1
- **rhs** (tile<i1|i8|i16|i32|i64>) - 右侧操作数。 13.1
- **result** (tile<i1|i8|i16|i32|i64>) - 输入 tile 的按位与运算结果。 13.1

**andi** 操作生成的值是两个整数元素类型图块按元素进行逐位“与”运算的结果。

$\text{andi}(x, y)_i = x_i \wedge y_i$

逐元素整数算术运算由目标架构的原生整数指令执行。默认语义在溢出或下溢时采用环绕语义。更多详情请参阅整数。

#### 约束条件

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可以无副作用地推测执行。
- 该操作是纯粹的，不执行任何内存副作用。
- **lhs**、**rhs** 和 **result** 必须具有相同的形状和元素类型 (tile<i1|i8|i16|i32|i64 >)。
- 操作的结果类型可以从其操作数和属性中推断出来。

## 8.10 原子操作

Warning

**Tile IR** 中的原子操作在早期访问版本中功能有限，将根据即将推出的内存模型和内存操作更新在后续版本中进行改进。

### 8.10.1 cuda\_tile.atomic\_cas\_tko

*Atomic compare-and-swap on global memory*

```
cuda_tile.atomic_cas_tko %memory_ordering_semantics %memory_scope %pointers %cmp  
%val %mask %token
```

#### 参数

- **memory\_ordering\_semantics** (MemoryOrderingSemantics) - 原子操作的内存排序语义。 13.1
- **memory\_scope** (MemoryScope) - 原子操作的内存作用域。 13.1
- **指针** (ptr) - 指向要执行原子比较与交换操作的内存位置的指针。 13.1
- **cmp** (tile) - 要进行比较的值。 13.1

- **val** ( tile ) - 要交换进去的值。 [13.1]
- **mask** ( tile<i>i1</i> ) - 原子操作的掩码。 [13.1]
- **token** ( token ) - 原子操作的令牌。 [13.1]
- **result** ( tile ) - 原子操作的结果。 [13.1]
- **result\_token** ( token ) - 原子操作的结果令牌。 [13.1]

**atomic\_cas** 操作在指定的全局内存 **pointers** 处执行逐元素的原子比较与交换。内存中的数据会与 **cmp** 进行比较，写入内存的数据由 **val** 指定。该操作返回执行原子操作前存储在内存中的原始值。

**pointers**、**cmp**、**val** 和 **result** 的形状（及元素类型）必须匹配。**atomic\_cas** 操作会在一个原子事务中为每个 (**pointer**, **cmp**, **val**) 元组执行以下步骤。（每个元组对应一个原子事务。）

```
atomic() {
    x = *pointer
    if x == cmp {
        *pointer = val
    }
    return x
}
```

一个可选参数 **mask** 允许指定哪些元素参与原子操作。位置 *i* 处的 **false** 值会屏蔽 **pointers** 中的相应元素，将其排除在操作之外。对于位置 *i* 处被屏蔽的元素，其返回值为 **cmp[i]**。如果未提供 **mask**，则默认所有元素都参与计算。**mask** 的形状必须与 **pointers**、**cmp** 和 **val** 相匹配。

令牌排序的原子比较并交换不受程序顺序的约束。除非受到令牌的限制，否则编译器可能会重新排序它（即在程序顺序中将其提前或延后）。

Supported data types:

- i32, i64: integer values
- f32, f64: floating-point values

对于浮点类型，比较使用按位相等而非

IEEE-754 语义。这意味着不同的 NaN 位模式被视为不同的值，并且如果 +0.0 和 -0.0 的位表示不同，则它们被视为不同的值。

**memory\_ordering\_semantics** 属性指定了不同线程间内存访问的并发假设，这控制了所需的同步。

例如，**weak** 排序允许编译器假设没有对任何被访问位置的并发访问。

更多信息，请参阅规范中的内存模型章节。

- **relaxed** - 可能同时访问该位置，但这种访问不会建立 **happens-before** 关系。
- **acquire** - 可能存在对该位置的并发访问。如果此获取操作观察到释放操作，则建立 *happens before* 关系。
- **release** - 可能同时存在对该位置的并发访问。如果此释放操作与获取操作一同被观察到，则建立了 *happens before* 关系。
- **acq\_rel** - 可能存在对该位置的同时访问。这同时具有释放和获取操作的效果。

注意：此操作不支持以下变体：`weak`。

`memory_scope` 属性指定了内存操作的通信范围。

在与系统中的其他并发线程通信时，作用域必须足够广泛，以涵盖参与通信的所有其他线程，否则可能发生数据竞争。

- `tl_blk` - 同一瓦片块内可能存在并发访问。
- `device` - 同一设备内部（例如 GPU）可能存在并发访问。
- `sys` - 系统中任何位置（即所有设备）都可能存在并发访问。

- `cmp`、`val` 和 `result` 必须具有相同的形状和元素类型（tile）。
- 操作必须在属性中对可变参数操作数段的大小进行编码。
- 操作的结果类型可以从其操作数和属性中推断出来。

#### 示例

```
%ptr_1x = reshape %ptr : tile<ptr<i32>> -> tile<1xptr<i32>>
%ptr_vec = broadcast %ptr_1x : tile<1xptr<i32>> -> tile<8xptr<i32>>
%offsets = iota : tile<8xi32>
%ptrs = offset %ptr_vec, %offsets : tile<8xptr<i32>>, tile<8xi32> -> tile<8xptr<i32>>

%cmp = constant <i32: [0, 1, 2, 3, 4, 5, 6, 7]> : tile<8xi32>
%val = constant <i32: [7, 6, 5, 4, 3, 2, 1, 0]> : tile<8xi32>
%mask = constant <i1: [0, 1, 0, 1, 0, 1, 0, 1]> : tile<8xi1>

// Atomic CAS without input token.
%0, %token = atomic_cas_tko relaxed device %ptrs, %cmp, %val :
    tile<8xptr<i32>>, tile<8xi32> -> tile<8xi32>, token

// Atomic CAS without input token.
%1, %token1 = atomic_cas_tko relaxed device %ptrs, %cmp, %val, %mask :
    tile<8xptr<i32>>, tile<8xi32>, tile<8xi1> -> tile<8xi32>, token

// Atomic CAS with input token.
%token2 = make_token : token
%2, %token3 = atomic_cas_tko relaxed device %ptrs, %cmp, %val token=%token2
:
    tile<8xptr<i32>>, tile<8xi32> -> tile<8xi32>, token

return
```

请参阅 `cuda_tile.atomic_cas_tko_0` 获取完整示例列表。



### 8.10.2 cuda\_tile.atomic\_rmw\_tko

*Atomic read-modify-write on global memory*

```
cuda_tile.atomic_rmw_tko %memory_ordering_semantics %memory_scope %pointers %  
mode %arg %mask %token
```

#### 参数

- **memory\_ordering\_semantics** ( MemoryOrderingSemantics ) - 加载操作的内存排序语义。  
13.1
- **memory\_scope** ( MemoryScope ) - 原子操作的内存作用域。  
13.1
- **指针** ( ptr ) - 执行原子操作的指针瓦片。  
13.1
- **mode** ( AtomicRMWMode ) - 原子操作模式 (例如: add、max、min 等)。  
13.1
- **arg** ( tile ) - 用于原子操作的值块。  
13.1
- **mask** ( tile<i>i1</i> ) - 加载操作的掩码。  
13.1
- **token** ( token ) - 原子操作的令牌。  
13.1
- **result** ( tile ) - 原子操作的结果。  
13.1
- **result\_token** ( token ) - 加载操作的结果令牌。  
13.1

`atomic_rmw_tko` 操作在 `pointers` 指定的全局内存位置上执行逐元素的原子读-修改-写操作。写入内存的值由 `mode` 和 `arg` 决定。该操作返回每个位置在原子更新之前存储的原始值。

`pointers`、`arg` 和 `result` 的形状必须匹配。指针类型的元素类型必须与 `arg` 和 `result` 的元素类型匹配。每个 (`pointer`、`arg`) 对在单个原子事务中处理。

```
atomic {  
  
    x = *pointer  
  
    y = mode(x, arg)  
  
    *pointer = y  
  
    return x  
  
}
```

一个可选参数 `mask` 指定哪些元素参与原子操作。在位置 `i` 处的 `False` 值会排除 `pointers` 中对应的元素参与操作。

被屏蔽元素的返回值由实现定义。

`mask` 的形状必须与 `pointers` 的形状匹配。

`atomic_addf` 操作被定义为向最接近的偶数舍入。

.. note::

当前编译器的实现会将非规格化数清零。此行为将在未来的编译器版本中修复，用户不应依赖此行为。

令牌排序的原子读-改-写操作不受程序顺序约束。除非受到令牌的进一步限制，编译器可以重新排序这些操作 (即在程序中将其提前或延后)。

Supported data types by mode :

>

```

>
> - ADD, AND, MAX, MIN, OR, UMAX, UMIN, XOR: i32, i64
>
> - ADDF: f16, f32, f64
>
> - XCHF: i32, i64, f32, f64
>
>
>

```

UMAX 和 UMIN 中的 U 前缀通过将比较解释为无符号，以区别于它们的有符号对应项 (MAX 和 MIN)。memory\_ordering\_semantics 属性指定了不同线程间内存访问的并发假设，这控制了所需的同步。例如，weak 排序允许编译器假设对任何被访问的位置都没有并发访问。更多信息，请参阅规范中的内存模型章节。

- relaxed - 可能同时访问该位置，但此访问不建立 happens-before 关系。
- acquire - 可能存在对该位置的并发访问。如果此获取操作观察到一个释放操作，则建立 happens before 关系。
- release - 可能同时存在对该位置的并发访问。如果此释放操作与获取操作一同被观察到，则建立了 happens before 关系。
- acq\_rel - 可能存在对该位置的同时访问。这同时具有释放和获取操作的效果。

注意：此操作不支持以下变体：weak。

memory\_scope 属性为内存操作指定通信范围。

当与系统中的其他并发线程通信时，范围必须足够广泛，以涵盖参与通信的所有其他线程，否则可能发生数据竞争。

- tl\_blk - 同一瓦片块内可能存在并发访问。
- device - 同一设备（例如 GPU）内部可能存在并发访问。
- sys - 系统内任何位置（即所有设备）都可能存在并发访问。

mode 属性指定了原子读-修改-写操作的模式。

- and - 执行按位与作为修改操作。
- or - 作为修改操作执行按位或运算。
- xor - 执行按位异或作为修改操作。
- add - 执行整数加法作为修改操作。
- addf - 将浮点数加法作为修改操作执行。
- max - 执行最大值作为修改操作。
- min - 执行最小值作为修改操作。
- umax - 执行无符号最大值作为修改操作。
- umin - 执行无符号最小值作为修改操作。
- xchg - 以交换作为修改操作。

mode 属性的默认值为 add。

## 约束条件

- `arg` 和 `result` 必须具有相同的形状和元素类型 (tile)。
- 操作必须在属性中对可变操作数段的大小进行编码。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 示例

```
// Reshape the input pointer tile to have a 1d shape
%ptr_1x = reshape %ptr : tile<ptr<f32>> -> tile<1xptr<f32>>

// Broadcast the reshaped tile to a tile with 8 rows, effectively
// replicating the pointer 8 times
%ptr_vec = broadcast %ptr_1x : tile<1xptr<f32>> -> tile<8xptr<f32>>

// Create a tile of offsets [0, 1, 2, ..., 7] to index into memory
%offsets = iota : tile<8xi32>

// Add the offsets to each pointer in the vector to create 8 unique
// pointers
%ptrs = offset %ptr_vec, %offsets : tile<8xptr<f32>>, tile<8xi32> -> tile<8
xptr<f32>>

%vals = constant <f32: [7.0, 6.0, 5.0, 4.0, 3.0, 2.0, 1.0, 0.0]> : tile<8
xf32>

// Perform atomic addf operations on the memory locations pointed by %ptrs
// without requiring an input token. Returns the original values and a
// result token
%0, %res_token0 = atomic_rmw_tko relaxed device %ptrs, addf, %vals :
    tile<8xptr<f32>>, tile<8xf32> -> tile<8xf32>, token

// Perform atomic add operations again, this time using the explicit input
// token
%token = make_token : token

%1, %res_token1 = atomic_rmw_tko relaxed device %ptrs, addf, %vals, token =
    %token :
    tile<8xptr<f32>>, tile<8xf32> -> tile<8xf32>, token
```

请参阅 `cuda_tile.atomic_rmw_tko_0` 获取完整示例列表。

## 8.11 视图

视图是一种与内存中的张量进行交互的结构化方式。相关内容在类型章节 张量视图 和语义章节 视图 中均有描述。

视图是 **Tile IR** 中与全局内存交互的主要方式。常见的模式是使用 `cuda_tile.make_tensor_view` 从指针构建一个张量视图，然后使用 `cuda_tile.load_view_tko` 和 `cuda_tile.store_view_tko` 操作对其进行读写。对于较大的张量，加载整个张量效率不高，因此我们提供了子视图分区视图，允许用户对 `tensor_view` 进行分块处理。

### 8.11.1 cuda\_tile.get\_index\_space\_shape

*Query the index space dimension size*

```
cuda_tile.get_index_space_shape %src
```

#### 参数

- **src** ( 视图类型 ) - 源视图类型。 13.1
- **result** ( Variadic<tile<any > > ) - 索引空间的形状，每个值表示对应维度的大小。 13.1

get\_index\_space\_shape 操作返回 src 的索引空间的形状。

结果图块具有与视图索引空间相同的秩，其元素表示相应维度的大小。

结果值应解释为无符号整数。

Warning

如果单个索引空间维度不适合结果图块的元素类型，则行为未定义。

#### 约束条件

- 该操作是纯粹的，不执行任何内存副作用。

#### 示例

```
%tensor_view = make_tensor_view %base,
    shape = [2, 2, 4], strides = [2, 2, 1]
    : tensor_view<2x2x4xf32, strides=[2,2,1]>

%partition_view = make_partition_view %tensor_view :
    partition_view<tile=(2x2x4), tensor_view<2x2x4xf32, strides=[2,2,1]>>

%dim0, %dim1, %dim2 = get_index_space_shape %partition_view :
    partition_view<tile=(2x2x4), tensor_view<2x2x4xf32, strides=[2,2,1]>> ->
    tile<i64>
```

完整示例列表请参见 cuda\_tile.get\_index\_space\_shape\_0。

### 8.11.2 cuda\_tile.get\_tensor\_shape

*Query the shape of a tensor view*

```
cuda_tile.get_tensor_shape %src
```

#### 参数

- **src** ( tensor\_view ) - 源张量视图。 13.1
- **result** ( Variadic<tile<any > > ) - 张量的形状，每个值代表对应维度的大小。 13.1

`get_tensor_shape` 操作返回支持所提供张量视图的张量的形状。  
 结果值应解释为无符号整数。  
 Warning  
 如果张量维度与结果图块的元素类型不匹配，则行为未定义。

## 约束条件

- 该操作是纯粹的，不执行任何内存副作用。

### 示例

```
%dim0, %dim1 = get_tensor_shape %tensor_view : tensor_view<32x32xf32,
    strides=[32,1]> -> tile<i64>
```

完整示例列表请参见 `cuda_tile.get_tensor_shape_0`。

### 8.11.3 `cuda_tile.load_view_tko`

*Load a tile from a tile view*

```
cuda_tile.load_view_tko %memory_ordering_semantics %memory_scope %view %index %
    token %optimization_hints
```

## 参数

- **memory\_ordering\_semantics** (`MemoryOrderingSemantics`) - 加载操作的内存排序语义。 [13.1]
- **memory\_scope** (`MemoryScope`) - 原子操作的内存作用域。 [13.1]
- **view** (`view_type`) - 加载图块所用的视图。 [13.1]
- **index** (`Variadic<tile<any>>`) - 要从视图加载的所需元素的 `n` 维索引。 [13.1]
- **token** (`token`) - 用于加载操作的可选令牌。 [13.1]
- **optimization\_hints** (`OptimizationHints`) - 操作 [13.1] 的优化提示
- **tile** (`tile`) - 加载的图块。 [13.1]
- **result\_token** (`token`) - 结果令牌。 [13.1]

`load_view_tko` 操作从图块视图中加载一个图块。

视图是从视图空间索引到视图中特定元素的映射，每种视图类型都定义了从视图空间索引到由视图元素生成的图块的映射。

例如，分区视图将一个张量视图划分为一个由相同大小图块组成的网格。该视图索引网格中一个被分区的图块。

对于给定的视图，索引的秩必须与视图索引空间的秩相匹配。有效索引的空间取决于传递给操作的视图。

例如，分区视图的索引空间等于分区图块的秩。

`index` 操作数被解释为无符号整数。

越界访问根据分区视图的语义处理。

`memory_ordering_semantics` 属性指定了不同线程间内存访问的并发假设，这控制了所需的同步。

例如，`weak` 排序允许编译器假设对任何访问位置都没有并发访问。

更多信息，请参阅规范中的内存模型部分。

- **weak** - 对源/目标位置无并发访问。
- **relaxed** - 可能同时访问该位置，但此访问不建立 happens-before 关系。
- **acquire** - 该位置可能存在并发访问。如果此获取操作观察到一个释放操作，则建立 *happens before* 关系。

注意：此操作不支持以下变体：release、acq\_rel。

memory\_scope 属性为内存操作指定通信范围。

当与系统中的其他并发线程通信时，作用域必须足够广泛，以涵盖参与通信的所有其他线程，否则可能发生数据竞争。

- **tl\_blk** - 同一瓦片块内可能存在并发访问。
- **device** - 同一设备（例如 GPU）内部可能存在并发访问。
- **sys** - 系统中任何位置都可能存在并发访问（即所有设备）。

optimization\_hints 属性以嵌套字典的形式提供特定于架构的编译器提示。

提示是针对每个架构（例如，sm\_100、sm\_120）指定的，并且对于每个架构，用户可以为每个操作指定具体的提示。

- **num\_cta\_in\_cga** - 为 cuda\_tile.entry 建议 CGA 中的 CTA 数量（必须是小于或等于 16 的 2 的幂次）。
- **allow\_tma** - 建议是否对 cuda\_tile.load\_view\_tko 和 cuda\_tile.store\_view\_tko 使用 TMA。
- **latency** - 为 cuda\_tile.load\_view\_tko 和 cuda\_tile.store\_view\_tko 提供的延迟提示。

For example they can be annotated as:

```
optimization_hints=<
  sm_100 = {num_cta_in_cga = 8},
  sm_120 = {num_cta_in_cga = 16}
>
```

## 约束条件

- 操作必须在属性中对可变操作数段的大小进行编码。

### 示例

```
%tensor_view = make_tensor_view %ptr, shape=[8192, 128], strides=[128, 1]
: tensor_view<8192x128xf32, strides=[128,1]>

// This example uses the PartitionView on a 8192x128xf32 tensor_view,
// dividing the tensor_view in tiles of 64x64.

%view = make_partition_view %tensor_view : partition_view<tile=(64x64),
  tensor_view<8192x128xf32, strides=[128,1]>>

%c0 = constant <i32: 0> : tile<i32>

%c1 = constant <i32: 1> : tile<i32>
```

```

// Load a tile at index (0, 0) in the view's index space.

// For this PartitionView, this is the rectangular tile such that
// X=[0,64) and Y=[0,64), in the coordinates of tiles.

%tile0, %res_token0 = load_view_tko weak %view[%c0, %c0]

    : partition_view<tile=(64x64), tensor_view<8192x128xf32, strides
      =[128,1]>>, tile<i32> -> tile<64x64xf32>, token

// Load a tile at index (0, 1) in the view's index space.

// For this PartitionView, this is the rectangular tile such that
// X=[0,64) and Y=[64,128), in the coordinates of tiles.

%tile1, %res_token1 = load_view_tko weak %view[%c0, %c1]

    : partition_view<tile=(64x64), tensor_view<8192x128xf32, strides
      =[128,1]>>, tile<i32> -> tile<64x64xf32>, token

// Same example as above but with memory token as input.

%token = make_token : token

%tile2, %res_token2 = load_view_tko weak %view[%c0, %c1] token = %token

    : partition_view<tile=(64x64), tensor_view<8192x128xf32, strides
      =[128,1]>>, tile<i32> -> tile<64x64xf32>, token

// Loads a tile at the dynamic index (%index, %index) in the view's index
space.

%tile3, %res_token3 = load_view_tko weak %view[%index, %index]

    : partition_view<tile=(64x64), tensor_view<8192x128xf32, strides
      =[128,1]>>, tile<i32> -> tile<64x64xf32>, token

```

完整示例列表请参见 `cuda_tile.load_view_tko_0`。

#### 8.11.4 `cuda_tile.make_partition_view`

*Create a partition view from a tensor view*

```
cuda_tile.make_partition_view %tensor_view
```

##### 参数

- **tensor\_\_view** ( `tensor_view` ) - 用于创建分区视图的源张量视图。 13.1
- **result** ( `partition_view` ) - 创建的划分视图。 13.1

`make_partition_view` 操作从 `tensor_view` 创建一个 `partition_view`。有关分区视图的更多详细信息，请参阅 `Partition View`。

该操作利用输入张量视图的类型约束和标注的返回类型来执行分区。张量视图的类型以形状和步幅的形式包含其物理布局，而分区视图则包含单个图块的逻辑大小。

生成的分区视图可以使用 `cuda_tile.load_view_tko` 加载，并使用 `cuda_tile.store_view_tko` 存储。

视图内存选项作用于分区视图的计算索引空间，详见Tensor View和Partition View获取详细语义。

### 约束条件

- 该操作根据特定操作数和属性有条件地可推测。
- 该操作可以在没有副作用的情况下被推测性地执行。
- 该操作是纯粹的，不会产生任何内存副作用。

#### 示例

```
%tensor_view0 = make_tensor_view %ptr, shape=[8192, 8192, 64], strides
               =[524288,64,1]

               : tensor_view<8192x8192x64xf32, strides=[524288,64,1]>

// Creates a partition with 32-bit-indexed tiles of size (1024x1x32) over
// the provided tensor_view.
make_partition_view %tensor_view0 :

    partition_view<

        tile=(1024x1x32),

        tensor_view<8192x8192x64xf32, strides=[524288,64,1]>

    >

%s0 = constant <i32: 8192> : tile<i32>
%str0 = constant <i32: 524288> : tile<i32>

// These seems very wrong.

%tensor_view1 = make_tensor_view %ptr, shape=[%s0, 8192, 64], strides=[%
               str0, 64, 1]

               : tile<i32> -> tensor_view<?x8192x64xf32, strides=[?,64,1]>

// Creates a partition with 32-bit-indexed tiles of size (1024x1x32) over
// the provided tensor_view, with masking. The provided tensor_view has a
// dynamically-sized dimension.
make_partition_view %tensor_view1 :

    partition_view<tile=(1024x1x32), tensor_view<?x8192x64xf32, strides
               =[,64,1]>>
```

请参阅 `cuda_tile.make_partition_view_0` 查看完整的示例列表。

### 8.11.5 `cuda_tile.make_tensor_view`

从指向全局内存的指针创建:code: `__PLACEHOLDER_0__`

```
cuda_tile.make_tensor_view %base %dynamicShape %dynamicStrides
```



## 参数

- **base** ( tile<ptr>) - 指向全局内存某一部分的标量基指针。 13.1
- **dynamicShape** (可变参数<图块<任意类型 > >) - 表示视图形状的数值数组，可以是完全动态的。 13.1
- **dynamicStrides** (可变参数<瓦片<任意类型 > >) - 表示视图步长的值数组，可以是完全动态的。 13.1
- **result** (tensor\_view) - 构造的 tensor\_view。 13.1

**make\_tensor\_view** 操作从一个全局内存指针、动态形状和动态步幅构建一个 **tensor\_view**。更多详情请参阅 Tensor View。

构造函数支持将形状和步长的动态数组作为参数，从而允许工作负载处理具有动态形状和步长的全局内存张量。如果这些参数是静态的，它们将静态地反映在生成的 **tensor\_view** 类型中；如果它们是动态的，则会在类型中显示为 ?。具体示例请参见下文。

**dynamicShape** 和 **dynamicStrides** 操作数被解释为无符号整数。

## 约束条件

- 操作必须在属性中对可变操作数段大小进行编码。
- 该操作是纯粹的，不会产生任何内存副作用。

### 示例

```
// tensor_view to a scalar tile of f32
%a0 = make_tensor_view %base,
    shape = [], strides = [] : tensor_view<f32>

// tensor_view to a tile of static shape and strides
%a1 = make_tensor_view %base,
    shape = [32, 32], strides = [32, 1]
    : tensor_view<32x32xf32, strides=[32,1]>

%sh0 = constant <i32: 32> : tile<i32>
%sh1 = constant <i32: 32> : tile<i32>
%st0 = constant <i32: 32> : tile<i32>
%st1 = constant <i32: 1> : tile<i32>

// tensor_view to a tile with partially dynamic shape and strides
// all dynamic values must be of the same type, here tile<i32>
%a2 = make_tensor_view %base,
    shape = [%sh0, %sh1], strides = [%st0, %st1]
    : tile<i32> -> tensor_view<?x?xf32, strides=[?,?]>
```

完整示例列表请参见 `cuda_tile.make_tensor_view_0`。

### 8.11.6 cuda\_tile.store\_view\_tko

*Stores a tile into a tile view*

```
cuda_tile.store_view_tko %memory_ordering_semantics %memory_scope %tile %view %  
index %token %optimization_hints
```

#### 参数

- **memory\_ordering\_semantics** ( MemoryOrderingSemantics ) - 存储操作的内存排序语义。  
13.1
- **memory\_scope** ( MemoryScope ) - 存储操作的内存作用域。 13.1
- **tile** ( tile ) - 要存储的瓦片。 13.1
- **视图** ( 视图类型 ) - 用于存储图块的视图。 13.1
- **index** ( 可变参数<瓦片<任意 > > ) - 视图中目标瓦片的索引。 13.1
- **token** ( token ) - 用于操作顺序的可选令牌。 13.1
- **optimization\_hints** ( OptimizationHints ) - 操作 13.1 的优化提示
- **result\_token** ( token ) - 用于同步的结果令牌。 13.1

`store_view_tko` 操作将图块存储到图块视图的索引视图中。

视图是从视图空间索引到视图中特定元素的映射，每种视图类型都定义了从视图空间索引到由视图元素生成的图块的映射。

例如，分区视图将张量视图划分为一个由等尺寸图块组成的网格。该视图用于索引网格中一个被分区的图块。

对于给定的视图，索引的秩必须与视图索引空间的秩匹配。有效索引的空间取决于传递给操作的视图。

例如，分区视图的索引空间等于分区图块的秩。

视图的索引空间是根据请求的图块大小和视图的形状计算得出的函数。

`index` 操作数被解释为无符号整数。

越界访问根据分区视图的语义进行处理。

`memory_ordering_semantics` 属性指定了不同线程间内存访问的并发假设，这控制了所需的同步。

例如，`weak` 排序允许编译器假设对任何被访问的位置都没有并发访问。

更多信息，请参阅规范中的内存模型章节。

- **weak** - 对源/目标位置没有并发访问。
- **relaxed** - 可能同时访问该位置，但此访问不会建立 `happens-before` 关系。
- **release** - 可能同时存在对该位置的并发访问。如果此释放操作与获取操作被共同观测到，则建立 `happens before` 关系。

注意：以下变体不受此操作支持：`acquire`、`acq_rel`。

`memory_scope` 属性为内存操作指定了通信范围。

当与系统中的其他并发线程通信时，范围必须足够广泛，以涵盖参与通信的所有其他线程，否则可能发生数据竞争。

- **tl\_blk** - 同一瓦片块内可能存在并发访问。
- **device** - 同一设备（例如 GPU）内部可能存在并发访问。

- `sys` - 系统中任何位置（即所有设备）都可能存在并发访问。

`optimization_hints` 属性以嵌套字典的形式提供特定于架构的编译器提示。

提示是针对每个架构（例如，`sm_100`、`sm_120`）指定的，并且对于每个架构，用户可以为每个操作指定特定的提示。

- `num_cta_in_cga` - 为 `cuda_tile.entry` 建议 CGA 中的 CTA 数量（必须是小于或等于 16 的 2 的幂次）。
- `allow_tma` - 建议是否对 `cuda_tile.load_view_tko` 和 `cuda_tile.store_view_tko` 使用 TMA。
- `latency` - 用于 `cuda_tile.load_view_tko` 和 `cuda_tile.store_view_tko` 的延迟提示。

For example they can be annotated as:

```
optimization_hints=<
    sm_100 = {num_cta_in_cga = 8},
    sm_120 = {num_cta_in_cga = 16}
>
```

## 约束条件

- 操作必须在属性中对可变操作数段的大小进行编码。
- 操作的结果类型可以从其操作数和属性中推断出来。

### 示例

```
%tensor_view = make_tensor_view %ptr, shape=[8192, 128], strides=[128,1] :
    tensor_view<8192x128xf32, strides=[128,1]>

// This example uses the PartitionView on a 8192x128xf32 tensor_view,
// dividing the tensor_view in tiles of 64x64.

%view = make_partition_view %tensor_view :
    partition_view<tile=(64x64), tensor_view<8192x128xf32, strides=[128,1]>>

%c0 = constant <i32: 0> : tile<i32>
%c1 = constant <i32: 1> : tile<i32>

%tile = constant <f32: 0.0> : tile<64x64xf32>

// Store a tile at index (0, 0) in the view's index space.
// For this TilePartitionView, this is the rectangular tile such that
// X=[0,64) and Y=[0,64), in the coordinates of tiles.

%res_token0 = store_view_tko weak %tile, %view[%c0, %c0]

: tile<64x64xf32>, partition_view<tile=(64x64), tensor_view<8192x128xf32,
    strides=[128,1]>>, tile<i32> -> token

// Store a tile at index (0, 1) in the view's index space.
```

```

// For this PartitionView, this is the rectangular tile such that
// X=[0,64) and Y=[64,128), in the coordinates of tiles.

%res_token1 = store_view_tko weak %tile, %view[%c0, %c1]

: tile<64x64xf32>, partition_view<tile=(64x64), tensor_view<8192x128xf32,
  strides=[128,1]>>, tile<i32> -> token

// Same example as above but with input token.

%token = make_token : token

%res_token2 = store_view_tko weak %tile, %view[%c0, %c1] token = %token

: tile<64x64xf32>, partition_view<tile=(64x64), tensor_view<8192x128xf32,
  strides=[128,1]>>, tile<i32> -> token

```

完整示例列表请参见 `cuda_tile.store_view_tko_0`。

## 8.12 杂项

**Tile IR** 中的杂项操作集是指那些没有特定类别的操作。

### 8.12.1 `cuda_tile.assume`

*Attach static information to an SSA value*

```
cuda_tile.assume %value %predicate
```

#### 参数

- **value** ( Any ) - 要附加谓词的值。 [13.1](#)
- **predicate** ( AssumePredicate ) - 要附加到该值的谓词。 [13.1](#)
- **result** ( Any ) - 带有附加谓词的值。 [13.1](#)

**assume** 操作将 **value** 作为结果传递，并为其附加一个谓词。假定的谓词是 **result** 的一个属性。

此操作可用于向编译器注入静态信息，从而可能实现更高效的代码生成。

**predicate** 必须实现 **AssumePredicateAttrInterface**。

Note

**assume** 不会检查谓词的正确性。

不正确的谓词可能注入错误的静态信息并导致编译错误。如果将不正确的谓词附加到 SSA 值上，程序的行为将是未定义的。

#### AssumePredicate 实现者

##### 有界

```
#bounded<(lb|?), (ub|?)>
```

`bounded` 属性必须用作 `cuda_tile.assume` 的谓词。谓词化的值必须是一个整数切片。  
`bounded` 为谓词图块的所有元素（当解释为有符号整数时）指定了下界和上界。边界是可选的：可以省略指定边界，如汇编格式中的“?”所示。例如，`#bounded<0, ?>`。下界和上界都是包含的。下界必须小于或等于上界。一个下界/超出预测值有效值范围的上限是无效的。

```
%1 = cuda_tile.assume #cuda_tile.bounded<0, ?>, %0
      : !cuda_tile.tile<4x8xi16>
```

完整示例列表请参见 `cuda_tile.assume_example_Bounded_0`。

## DivBy

```
div_by< $divisor (, every $every^ along $along)?>
```

`div_by` 属性必须用作 `cuda_tile.assume` 操作的谓词。谓词化的值必须是整数或指针的 `tile`，或者是一个 `tensor_view`。

如果预测值是一个 `tile`，该属性表示 `tile` 的某些元素可以被 `divisor` 整除。如果预测值是一个 `tensor_view`，该属性表示 `tensor_view` 的基地址可以被 `divisor` 整除。`divisor` 必须是 2 的正整数次幂。

`every` 和 `along` 属性控制哪些元素被假定满足可除性属性。当沿着维度 `along` 将张量按大小为 `every` 的组进行分割时，假定每个组的第一个元素满足可除性属性。假定组内其他元素以 1 单调递增。对于指针的 `tile`，假定元素以指针所指向类型的字节宽度单调递增。最后一个组的大小可能小于 `every`。

`every` 和 `along` 属性是可选的。如果缺失，在 `tile` 的情况下，它们默认值分别为 1 和 0。

即，假定 `tile` 的所有元素都满足可除性。（在这种情况下，`along` 的值无关紧要。）如果谓词值是 `tensor_view` 或 0D `tile`，则不能使用 `every` 和 `along`。

`every` 和 `along` 必须同时使用。如果指定了其中一个，则另一个也必须被指定。

Note

如果预测值是一个整数瓦片，`every` 是整数值有符号解释的一个属性。否则，它是无符号整数解释的一个属性。例如，对于以下“i8”值序列（以二进制形式书写），`every = 4` 是不正确的，因为它们被解释为有符号整数时会环绕：`[01111110, 01111111, 10000000, 10000001]`。`every = 2` 将是正确的。下面的例子展示了满足假设性质的张量。

```
// Example 1: Each pointer is divisible by 16.
// [ 0x10, 0x20, 0x80, 0x10, 0x0, 0x120, ... ]
%0 = cuda_tile.assume #cuda_tile.div_by<16>, %ptrs
      : !cuda_tile.tile<128x!cuda_tile.ptr<f32>>
// Note: Equivalent to #cuda_tile.div_by<16, every 1 along 0>.
```

完整示例列表请参见 `cuda_tile.assume_example_DivBy_0`。

```
// Example 2: Each integer is divisible by 4.
// [ 16, 24, 8, 4, 12, 12, 0, 16, ... ]
%0 = cuda_tile.assume #cuda_tile.div_by<4>, %t
      : !cuda_tile.tile<128xi32>
```

请参阅 `cuda_tile.assume_example_DivBy_1` 获取完整示例列表。

```
// Example 3: Group size [4].
// [ 7, 8, 9, 10, 23, 24, 25, 26, 0, 1, 2, 3, ... ]
```

```
%0 = cuda_tile.assume #cuda_tile.div_by<1, every 4 along 0>, %t
      : !cuda_tile.tile<128xi32>
```

完整示例列表请参见 `cuda_tile.assume_example_DivBy_2`。

```
// Example 4: 2-d Group size [1, 4] with divisibility 4.
// [ [ 4, 5, 6, 7, 12, 13, 14, 15 ],
//   [ 8, 9, 10, 11, 24, 25, 26, 27 ],
//   [ 24, 25, 26, 27, 64, 65, 66, 67 ],
//   [ 0, 1, 2, 3, 4, 5, 6, 7 ] ]
%0 = cuda_tile.assume #cuda_tile.div_by<4, every 4 along 1>, %t
      : !cuda_tile.tile<4x8xi32>
```

完整示例列表请参见 `cuda_tile.assume_example_DivBy_3`。

```
// Example 5: 2-d Group size [4, 1] with divisibility 32.
// Note that the elements within each column are monotonically increasing
// by the byte width of the pointee type f32, e.g., 0x20, 0x24, 0x28, 0x2c.
// [ [ 0x20, 0x100, 0x40, 0x60, 0x40, 0x200, 0x340, 0x40 ],
//   [ 0x24, 0x104, 0x44, 0x64, 0x44, 0x204, 0x344, 0x44 ],
//   [ 0x28, 0x108, 0x48, 0x68, 0x48, 0x208, 0x348, 0x48 ],
//   [ 0x2c, 0x10c, 0x4c, 0x6c, 0x4c, 0x20c, 0x34c, 0x4c ] ]
%0 = cuda_tile.assume #cuda_tile.div_by<32, every 4 along 0>, %ptrs
      : !cuda_tile.tile<4x8x!cuda_tile.ptr<f32>>
```

完整示例列表请参见 `cuda_tile.assume_example_DivBy_4`。

## SameElements

```
#same_elements< $values >
```

`same_elements` 属性必须用作 `cuda_tile.assume` 的谓词。谓词值必须是一个整数或指针的张量。  
`same_elements` 为每个维度指定。对于大小为  $N$  的维度，值  $C$  表示将该维度划分为  $N/C$  个大小为  $C$  的组后，每个组由相同的元素组成。由于  $N/C$  可能无法整除，最后一组的元素数量可能少于  $C$ 。

如果“相同元素”属性在某个维度上不成立，则相应的值应设为 1。`#cuda_tile.same_elements<[1, 1, ..., 1]>` 对于任何整数或指针张量都是一个正确的谓词，其中 1 的数量与张量的秩相匹配。（大小为 1 的组总是具有相同的元素。）

```
// Integer tensor with same elements.
%0 = cuda_tile.constant <i16: [[0, 0, 0, 0, 10, 10, 10, 10],
                                [0, 0, 0, 0, 10, 10, 10, 10],
                                [5, 5, 5, 5, 93, 93, 93, 93],
```

```

                                [5, 5, 5, 5, 93, 93, 93, 93]]>

    : tile<4x8xi16>
%1 = cuda_tile.assume #cuda_tile.same_elements<[2, 4]>, %0

    : !cuda_tile.tile<4x8xi16>
// Pointer tensor with same elements.
%2 = cuda_tile.constant <i64: [[ 0,  0,  0,  0,  8,  8,  8,  8],
                                [ 0,  0,  0,  0,  8,  8,  8,  8],
                                [64, 64, 64, 64, 32, 32, 32, 32],
                                [64, 64, 64, 64, 32, 32, 32, 32]]>

    : tile<4x8xi64>
%3 = cuda_tile.bitcast %2

    : !cuda_tile.tile<4x8xi64>

    -> !cuda_tile.tile<!cuda_tile.ptr<f32>>
%4 = cuda_tile.assume #cuda_tile.same_elements<[2, 4]>, %3

    : !cuda_tile.tile<!cuda_tile.ptr<f32>>

```

完整示例列表请参见 `cuda_tile.assume_example_SameElements_0`。

## 约束条件

- `value` 和 `result` 必须具有相同的形状和元素类型 (Any)。
- 操作的结果类型可以从其操作数和属性中推断出来。

## 示例

```

// Assume that all integers are divisible by 32.
%int_tile = constant <i16: [32, 64, 0, 0, 32, -32, 1024, 0]> : tile<8xi16>
%div_by_1 = assume div_by<32>, %int_tile : tile<8xi16>

// Assume that every 4th element (starting with element 0) along
// dimension 0 is divisible by 32 that and all integers are
// monotonically increasing by 1 within each group of 4.
%int_tile_2 = constant <i16: [96, 97, 98, 99, 64, 65, 66, 67]> : tile<8xi16>
%div_by_2 = assume div_by<32, every 4 along 0>, %int_tile_2 : tile<8xi16>

// Assume that every rectangular chunk of size [1, 4, 2] has the same
// values.
%same_elem = assume same_elements<[1, 4, 2]>, %ptr_3d : tile<1x8x8xptr<f32>>

```

```
// Assume that every value is greater or equal to 5.

%int_tile_3 = constant <i16: [5, 9, 10, 11, 6, 5, 5, 7]> : tile<8xi16>

%bounded = assume bounded<5, ?>, %int_tile_3 : tile<8xi16>
```

完整示例列表请参见 `cuda_tile.assume_0`。

### 8.12.2 `cuda_tile.print_tko`

*Print a formatted string (token-ordered)*

```
cuda_tile.print_tko %str %args %token
```

#### 参数

- **str** ( 字符串 ) - 格式字符串。 [13.1](#)
- **args** ( 可变参数<tile> ) - 要格式化并打印的参数。 [13.1](#)
- **token** ( token ) - 用于操作顺序的可选令牌。 [13.2](#)
- **result\_token** ( token ) - 用于同步的结果令牌。 [13.2](#)

`print_tko` 操作会打印一个 C 语言 `printf` 风格的格式字符串，并与给定的操作数交错输出。格式表达式的数量

(以 `%` 字符开头) 必须与操作数的数量匹配。

如果格式表达式不适用于其相应的操作数，则输出是未定义的。

令牌排序的打印操作不受程序顺序的约束。编译器可能会对它们进行重新排序（即在程序中提前或推迟它们）。

unless further constrained by tokens.

此操作用于调试。其实现未针对性能进行优化，因此不应在生产模式下使用。打印操作不保证原子性。即，同时执行的打印输出可能会交错显示。

Note

该操作在 13.2 版本中从 `print` 更名为 `print_tko`，操作码未改变。

#### 约束条件

- 操作必须将可变参数操作数段的大小编码在属性中。
- 操作的结果类型可以从其操作数和属性中推断出来。

#### 示例

```
print_tko "Hello world: %f\n", %arg : tile<4xf32> -> token

print_tko "%+08.3f", %arg : tile<4xf32> -> token
```

完整示例列表请参见 `cuda_tile.print_tko_0`。



## 9 调试信息

调试信息对于理解和诊断 **Tile IR** 程序的行为至关重要。调试信息是一种元数据，它将生成的代码映射回原始源代码，为 **Tile IR** 用户提供更具有信息性和直观性的调试体验，并允许用户快速识别和修复 **Tile IR** 程序中的问题。本节介绍在使用 MLIR 方言时，如何在 **Tile IR** 程序中包含和使用调试信息。有关如何在字节码中直接生成此信息的说明，请参阅字节码部分。

**Tile IR** 支持基于控制流的未优化代码调试，具备断点、单步执行和检查堆栈帧等功能。**Tile IR** 的调试信息会公开描述构成 **Tile IR** 程序的文件、函数和块的元数据，以及该程序的编译信息。此元数据通过融合到位置信息中而包含在 **Tile IR** 程序中。具体细节和示例如下。**Tile IR** 不支持用户变量的值检查，目前仅支持上述基于控制流的调试。

### 9.1 位置信息

**Tile IR** 要求将作用域元数据附加到 **Tile IR** 模块内的所有操作，以生成任何类型的调试信息。除了简单的文件、行、列位置信息外，还需要作用域元数据。仅提供简单的文件、行、列位置信息而没有作用域元数据会导致用户体验极差。生成 DWARF 信息需要作用域元数据，而所有开发工具（包括调试器和性能分析器）都需要 DWARF 信息。

简单文件、行、列位置的示例：`loc("/tmp/foo.py":1:4)`

融合了范围元数据的位置示例：`#cuda_tile.di_loc<loc("/tmp/foo.py":1:4) in #subprogram`

#### 9.1.1 位置类型

**Tile IR** 支持两种位置类型：

1. 一个由 `#cuda_tile.di_loc` 属性表示的 **Tile IR** 位置，该属性将文件、行、列信息与范围元数据相结合，如上例所示。这是生成与范围元数据融合的位置信息的首选方法。
2. 一个 `CallSiteLoc`，其中被调用方和调用方都是受支持的位置类型。这表示一个函数调用，例如在 **Tile IR** 生产者支持内联的情况下。

任何包含受支持位置（如 `FusedLoc`、`NamedLoc` 或 `OpaqueLoc`）的位置都将转换为该受支持位置。如果未找到受支持的位置，该位置将转换为 `UnknownLoc`，表示位置未知或不可用。所有 **Tile IR** 全局变量都将具有 `UnknownLoc`，因为 `#cuda_tile.di_loc` 仅支持局部作用域，且不支持 `FileLineColLoc`。

| Location Type      | CudaTile Bytecode Support                                          |
|--------------------|--------------------------------------------------------------------|
| CudaTile DILocAttr | Supported                                                          |
| CallSiteLoc        | Supported if BOTH callee and caller are supported, else UnknownLoc |
| FusedLoc           | Converted to first supported sub location, else UnknownLoc         |
| NameLoc            | Converted to child location if supported, else UnknownLoc          |
| OpaqueLoc          | Converted to fallback location if supported, else UnknownLoc       |
| FileLineColLoc     | Unsupported; always treated as UnknownLoc                          |

#### 9.1.2 生成作用域元数据

以下是 **Tile IR** 程序中生成作用域元数据的选项、权衡和机制。

#### 9.1.3 选项

**Tile IR** 的生产者有两种生成与作用域元数据融合的位置信息的选择：

1. **Tile IR** 生产者可以直接使用 `#cuda_tile.di_loc` 生成融合了范围元数据的位置信息，如上例所示。
2. **Tile IR** 生产者可以生成包含文件、行、列位置信息的 `loc`，然后通过 **Tile IR** 传递（使用 `--synthesize-debug-info-scopes`）从该文件、行、列位置信息合成作用域元数据。

Note

> 文件、行、列位置信息必须在生成字节码之前转换为 **Tile IR** 位置，因为所有 `FileLineColLoc` 属性在字节码中都被编码为 `UnknownLoc` 属性。

### 9.1.4 权衡

在以上两种方案之间做选择时，**Tile IR** 的生成者应考虑以下权衡：

1. 为 **Tile IR** 生产者提供带有作用域元数据的位置信息可能更难实现，然而，**Tile IR** 生产者保留完全控制权，可以向编译器描述其源代码，这可能为最终用户带来更顺畅的调试体验。
2. 对于 **Tile IR** 生成器而言，提供简单的文件行号和位置信息非常容易实现，但这会将控制权让渡给处理过程，而该过程可能无法在所有情况下准确合成调试信息，最终可能导致终端用户的调试体验下降。例如，处理过程或许能够合成函数名称、推测链接名称，但在获取函数的行号或列号方面可能表现不佳。

### 9.1.5 机制

要合成作用域元数据，可以将 `SynthesizeDebugInfoScopes` 过程添加到任何 **Tile IR** 过程流水线中，或通过 Python 按如下方式调用：

```
pm = passmanager.PassManager(context=module.context)
full_pass_string = f"cuda_tile.module({\"synthesize-debug-info-scopes\"})"
pm = pm.parse(full_pass_string, context=module.context)
pm.run(module.operation.regions[0].blocks[0].operations[0])
```

## 9.2 作用域元数据

**Tile IR** 调试信息由以下表示为属性和字段的范围元数据组成：

### 9.2.1 编译单元

编译单元表示在特定编译单元内声明的所有对象的根作用域。它指定与编译单元关联的源文件，并包含所有其他调试信息元素，如文件、词法块和子程序。此作用域对于组织调试信息并将其与原始源代码关联至关重要，有助于实现高效调试。

Fields:

- **File**：与编译单元相关联的源文件。

以下字段由 **Tile IR** 编译器设置，列出以供参考：

- **Is Optimized?**：如果编译器选项 `--opt-level` 为 0 则设为 `false`，否则设为 `true`。
- **Emission Kind**：根据编译器选项设置：`--line-info` 仅生成行表，或使用 `--device-debug` 别名 `-g` 生成完整调试信息。

### 9.2.2 文件

一个文件代表编译过程中使用的源文件。它指定了源文件的文件名和目录。此信息用于将生成的代码映射回原始源文件，使开发人员能够有效地追踪和调试代码。

Fields:

- **Name**：文件的名称。
- **Directory**：包含该文件的目录。

### 9.2.3 词法块

一个词法块表示子程序中的一个嵌套作用域，指定了该作用域的范围、文件、行号以及可选的列号。词法块用于表示源代码中的各种嵌套作用域，例如条件语句、循环和其他代码块。这种详细的作用域信息有助于在调试过程中理解程序的结构和控制流。

Fields:

- **Scope**：定义词法块的作用范围。
- **File**：包含词法块的源文件。
- **Line**：词法块开始的行号。
- **Column**：词法块起始的列号。

### 9.2.4 子程序

子程序表示源语言中的一个函数。它指定了子程序的作用域、文件、行号、名称和链接名称。可选地，它可以包含作用域内的行号。子程序信息用于函数级调试，允许开发人员在特定函数内设置断点、逐步执行代码以及检查栈帧。

Fields:

- **File**: 包含子程序的源文件。
- **Line**: 子程序开始的行号。
- **Name**: 子程序的名称。
- **Linkage Name**: 子程序的链接名称。
- **Compile Unit**: 包含该子程序的编译单元。
- **Scope Line**: 子程序作用域开始的行号。[可选]

以下字段由 **Tile IR** 编译器设置，并列出以供参考：

- **Subprogram Type**: 设置为 `() -> ()`，意味着所有函数似乎都没有参数和返回值。

## 9.3 示例用法

以下是一个展示如何使用 **Tile IR** 调试信息的示例：

```
#file= #cuda_tile.di_file<"foo.py" in "/tmp/">

#compile_unit= #cuda_tile.di_compile_unit<
  file = #file
>

#subprogram= #cuda_tile.di_subprogram<
  file = #file,
  line = 1,
  name = "test_kernel",
  linkageName = "kernel",
  compileUnit = #compile_unit,
  scopeLine = 1
>

#block= #cuda_tile.di_lexical_block<
  scope = #subprogram,
  file = #file,
  line = 1,
  column = 4
>

#loc_fn= #cuda_tile.di_loc<loc("/tmp/foo.py":1:4) in #subprogram>
#loc_return= #cuda_tile.di_loc<loc("/tmp/foo.py":5:4) in #block>

cuda_tile.module @kernels attributes { } {
  entry @kernel() {
    return loc(#loc_return)
  } loc(#loc_fn)
}
```